THE JOURNEY OF A TINY MODEL (YOUR TLM) · EVERY NUMBER COMPUTED

Arithmetic Intelligence

User-friendly AI maths

Follow your TLM, a language model with 80 numbers: designed, started from random noise, trained on four sentences, fine-tuned, rewarded, wrapped in a chat harness. At every step, the same step in a real, openly released 8B-parameter model ("the 8B model"), and what it costs in compute, memory and GPUs.

EVERY CHAPTER HAS

- **Story bar**: which of the 5 stages we are in
- **Mental map**: the step highlighted
- **Comes in → Who does the work → Hands on**: how it connects
- **Our numbers**: computed, step by step
- **Python**: a short snippet you can run

PLUS

- **In a real model**: the 8B model
- **Scale it up**: compute, memory, GPUs
- **Insight**, **Watch out**, **Myth vs reality**

THE ONLY MATHS YOU NEED

Multiply, add, and turn numbers into percentages. Where something harder appears (an exponent, a logarithm, a square root), the guide gives the result and one line on why it's there.

START HERE

Why you should (or shouldn't) care

Love maths or hate it, AI decisions now land on your desk. A few multiplications decide the bill, the speed and the failure modes.

You don't like maths

You won't have to do any. Every calculation is done and shown. What you get is the ability to **predict**: why a long chat slows down, why a bigger model costs more, why the same question gets two answers.

You like maths

The tiny model is complete and real. Every number can be checked by hand or with the Python snippets, and each chapter connects one operation to gigabytes, GPU hours and tokens per second.

Decisions this guide helps you make

Decision	Common guess	What the arithmetic says	Ch
Which model size?	Biggest is safest	Memory = parameters × bytes. 8B at 16-bit needs ≈ 16 GB; 70B needs ≈ 140 GB (several GPUs). Speed and cost scale with it.	17
Why is my long chat slow and expensive?	The provider is throttling	Every new token reads every earlier token's cached keys and values. Memory and work grow with conversation length.	15
Same question, different answers?	It's broken	The model outputs percentages; a word is then *sampled*. Lower temperature for consistency.	14
Trust a confident answer?	95% sure means right	A share is the model's preference among words, not a measured chance of being right. Our model gives "rain" 95% in a situation it never trained on.	14
Add our documents: fine-tune or paste in?	Fine-tune first	RAG puts relevant text into the prompt, where the model can read it; fine-tuning changes the weights. They solve different problems.	6
Which GPU to rent or buy?	Compare compute (TFLOPS)	Chat speed is capped by memory bandwidth and capacity far more than by compute.	18
Run it on a laptop?	4-bit is junk	4-bit ≈ ¼ of the 16-bit memory, usually with a modest quality loss.	17
Budget per word?	Price per word	Billing is per token. In English, 100 tokens ≈ 75 words; other languages often need more tokens per word.	2

ONE DECISION, WORKED THROUGH

A team wants every chat to include a 100-page manual (≈ 50,000 tokens) on the 8B model. Extra KV-cache memory for those tokens (128 KiB each):

```
Pasting it in:   50,000 × 128 KiB        ≈ 6.1 GiB per user
                 10 users at once        ≈ 61 GiB
Retrieving the 3 relevant pages (≈ 1,500 tokens):
                 1,500 × 128 KiB         ≈ 0.18 GiB per user
```

An illustrative comparison of **document** context only: about 30× less for this part. The system prompt, conversation history and the answer still use context and cache either way. Total memory doesn't shrink by the same factor: the 16 GB of weights, working memory and overhead stay, and retrieval adds its own search and storage cost. Chapters 15 and 17 show where the 128 KiB comes from.

WHEN YOU CAN SKIP THIS

If you only chat casually, never choose models, never pay the bill and never act on AI output without checking, the myth boxes are enough. Everyone else: the scale tables are where the money is.

BEFORE WE START

Words you need

Every term used later, defined once, with its value in our tiny model and in the 8B model.

Word	Plain meaning	Our tiny model	The 8B model
Token	A piece of text (word or word-part) swapped for an id number	"love" = id 1	128,256 possible tokens
Vector (row)	A list of numbers. Each token is represented by one	4 numbers	4,096 numbers
Slot	**One position in a vector: the 1st number, the 2nd number… Also called a *dimension*. A slot has no name or meaning on its own; meaning lives in patterns across slots**	**4 slots**	**4,096 slots**
Embedding table	A lookup table with one row per token. The token id is the row number; the row is that token's starting vector	6 rows × 4 slots	128,256 rows × 4,096 slots
Matrix	A grid of numbers. Multiplying a vector by a matrix = many dot products at once	Detectors: 3 × 4	Feed-forward: 14,336 × 4,096 (×3)
Weight / parameter	One number set by training and saved in the model file. Same thing, two names	80	≈ 8.03 billion
Score	A dot product used to compare. *Attention scores*: a token's query vs an earlier token's key (Ch 6). *Vocabulary scores*: the final row vs every word (Ch 9)	2 × 2 attention grid; 6 vocabulary scores	n × n per head; 128,256 vocabulary scores
Attention head	One independent attention calculation with its own Q, K, V weights. Several run side by side in each layer	1 per layer	32 per layer (8 shared K/V groups)
Layer	One round of attention (all its heads) + feed-forward. Layers are stacked, each working on the result of the one below	1	32
Bytes per number	Storage per number. 16-bit = 2 bytes; 4-bit = ½ byte	80 × 2 = 160 bytes	≈ 16 GB at 16-bit
FLOP	One multiply or one add: the unit of compute	≈ 180 per token	≈ 16 billion per token
GPU memory / bandwidth	How much fits on the chip (GB) / how fast it can be read (TB/s)	—	Data-centre GPU: 80 GB / ≈ 3.35 TB/s
Training / inference	Setting the weights from data / using fixed weights to answer	Chapters 10–12 / 13–15	Weeks once / every request

THE FOUR COSTS TO WATCH IN EVERY CHAPTER

- **Compute** (FLOPs): how much multiplying. Drives training cost and prompt-reading speed.
- **Memory capacity** (GB): what must sit on the GPU at once. Decides which GPU, and how many users fit.
- **Memory bandwidth** (TB/s): how fast numbers can be read. Decides how fast the answer types out.
- **Number of GPUs**: needed when weights or cache don't fit on one, adding communication cost.

SETTING THE STAGE

What we are building, and how the story runs

We design a language model from nothing. No step is skipped and no number is invented.

THE GOAL

Goal	What it means	Done when
1. What the model must do	Continue text the way our training text does: after **"I love"**, say tea or coffee in the right proportion	tea ≈ 67%, coffee ≈ 33%
2. What training must do	Turn 80 random numbers into numbers that achieve goal 1	Loss close to its best possible value, 0.520
3. Your goal	See every number change on the way, and what each step costs at real scale	You can predict why a model is slow, costly or unsure

Start → finish: random model gives tea **18%** → after 500 training steps tea **67%**, coffee **33%**, rain **0.2%**.

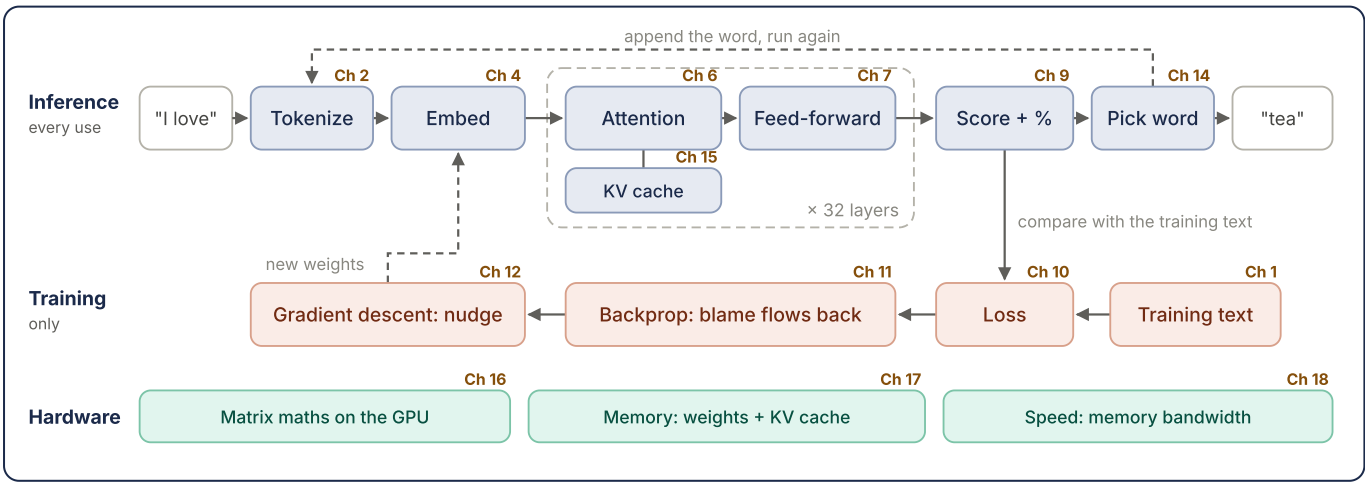
The story in six stages

Stage	Question	Who does the work	What comes out	Ch
1 Design	What should it learn, from what text, with what shape?	Us, the designers	Training text, vocabulary, the shape (80 empty slots for weights)	1–3
2 First run	What does a model with random weights do?	Embedding table, attention, feed-forward, scorer	A guess that is pure noise (tea 18%)	4–9
3 Train	How do random numbers become useful?	Loss, backprop, gradient descent	80 trained weights, saved and frozen	10–12
4 Use	What does the trained model do?	The same parts, now with trained weights, plus the picker and KV cache	"I love tea" (or coffee)	13–15
5 Scale	What changes at 8 billion weights?	GPU, memory, bandwidth	The bill: GB, GPUs, tokens per second, and smaller copies	16–19
6 Beyond	How do today's assistants go further?	Fine-tuning, reinforcement learning, the harness (incl. chat interface)	A model that follows a preference, inside a product, and how to pick one	20–23

Who does what: every part has one job

Part	Its one job	What it owns	Ch
Training text	Defines what "right" means	4 sentences	1
Tokenizer	Cuts text into pieces, swaps each for an id	A fixed word list (not learned)	2
Embedding table	Gives each id its starting row of numbers	24 weights	4
Attention	Lets a word pull information from earlier words	32 weights	6
KV cache	Keeps earlier words' keys and values so they aren't recomputed	Memory, no weights	15
Feed-forward	Adds information stored in its weights	24 weights	7
Scorer (softmax)	Turns the final row into a percentage per word	Reuses the embedding table	9
Picker	Chooses one word from the percentages	Settings: temperature, top-p	14
Loss	Measures how wrong the percentages were	Nothing; it compares	10
Backprop	Works out each weight's share of the blame	Nothing; it calculates	11
Gradient descent	Nudges every weight to reduce the loss	Setting: learning rate	12
GPU	Does all the multiplying; holds weights and cache in memory	Compute, memory, bandwidth	16–18
Number format (bits)	Sets how many bytes each stored weight takes	A setting: 16, 8 or 4 bits	19
Fine-tuning data	Targeted text that shifts behaviour	Examples (here 3 sentences)	20
Reward	Scores answers the model produced	A rule, tests, or a reward model	21
Harness	Code around the model: chat history, prompts, tools, loops, checks	Ordinary software	22

The mental map: one diagram for the whole guide



HOW TO USE THE MAP

Every chapter shows this map in miniature with its step highlighted in amber, so you always know where you are. Inference is the top lane. Training runs the whole top lane, then adds the bottom lane. The amber Ch labels are chapter numbers.

What is real here, and what is simplified

Part	Our tiny model	The 8B model	Same idea?
Embedding lookup, causal attention, shortcuts, softmax, cross-entropy loss	Yes, exactly as used in real models	Yes	✔ identical
Feed-forward switch	ReLU	SwiGLU (a gated, smoother switch)	✔ same purpose
Normalisation	None (numbers stay small enough)	RMSNorm before each sub-layer	✘ omitted
Word positions	None: order comes only from "look backwards only"	RoPE rotates Q and K by position	✘ omitted
Output scoring	Reuses the embedding table (as some well-known models do)	A separate output table	✔ both are used in practice
Tokenizer	Whole words	128K sub-word pieces (BPE)	✔ same job
Starting weights	Random, spread 0.5, rounded to 1 decimal	Random, spread 0.02	✔ same idea
Optimiser	Plain gradient descent, 500 steps	AdamW, ≈ 15 trillion tokens	✔ AdamW is a refined version
Gradients	Exact backprop, checked by nudging each weight (Ch 11)	Backprop via automatic differentiation	✔ same numbers

What this tiny model cannot show you

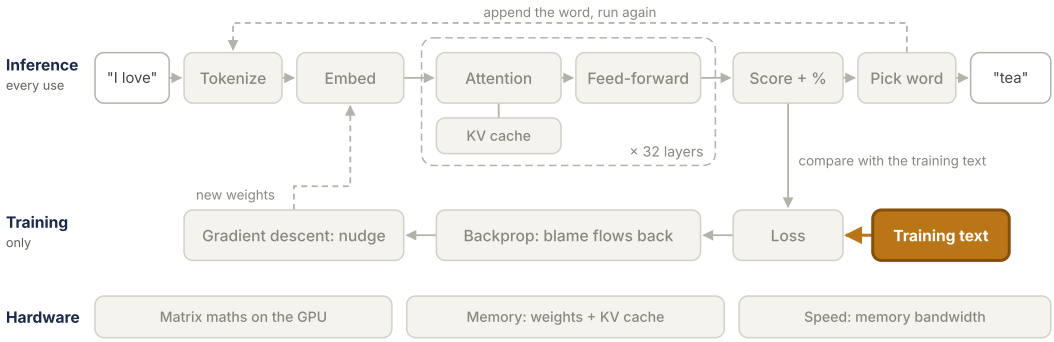
- **Capabilities at scale.** How large models generalise, follow instructions or reason. Four sentences can show the mechanism, not those abilities.
- **Post-training at scale.** Chapters 20–21 run fine-tuning and RL on the tiny model, but real instruction tuning and preference training use large curated datasets and human judgements.
- **Word order.** Without position encoding, our model knows order only through "look backwards only".
- **Where knowledge lives.** Our clean attention/feed-forward split is a mental model; in real models it's spread across many layers.
- **Real hardware speed.** GPU figures are idealised estimates, not benchmarks.
- **Understanding.** Similar rows show a useful statistical pattern, not evidence of understanding either way.

STAGE 1

Design

Before any maths runs, people decide three things: what the model should learn, from what text, and what shape it has.

STORY **1 Design** › 2 First run › 3 Train › 4 Use › 5 Scale › 6 Beyond



1 The goal and the training text

Pre-training teaches a language model to predict the next token. Along the way it can learn patterns that support grammar, facts and reasoning. The objective is simple; what emerges from it is not. Either way, the training text defines what it learns from.

What do we want the model to learn, and from what?

COMES IN
Our goal: continue "I love..."

→

WHO DOES THE WORK
Us, writing the training text

→

HANDS ON
4 sentences → the tokenizer (Ch 2)

IN ONE LINE The training text defines what “right” means; pre-training learns its statistics.

We write four sentences:

```
I love tea
I love tea
I love coffee
I hate rain
```

The model learns to predict **each next word** from the words before it. That gives 8 prediction tasks:

Seen so far	Next words in the text	The best possible prediction
"I"	love, love, love, hate	love 75%, hate 25%
"I love"	tea, tea, coffee	**tea 67%, coffee 33%**
"I hate"	rain	rain 100%

- The right answer is **not** "always tea". It's a **split**, copied from how often each word follows in the text.
- "rain" only ever follows "hate", so after "love" it should get nearly 0%.

Count what follows what: this is the target — Python

```python
from collections import Counter
text = ["I love tea", "I love tea", "I love coffee", "I hate rain"]
after_love = Counter(s.split()[2] for s in text if s.startswith("I love"))
print(after_love)   # Counter({'tea': 2, 'coffee': 1})  ->  67% / 33%
```

IN A REAL MODEL (8B)

The 8B model was pre-trained on **≈ 15 trillion tokens** of publicly available text (its published model card). The task is identical: predict each next token. What *pre-training* learns comes from patterns in that text. A deployed assistant's behaviour is also shaped by post-training, retrieval and tools (Stage 6).

SCALE IT UP: WHAT GROWS, AND WHAT IT COSTS

Knob	Toy → 8B model	Compute	Memory	Good	Bad
Training text	4 sentences → ≈ 15 trillion tokens	Training compute ≈ 6 × parameters × tokens ≈ 7×10^{23} operations for 8B; its model card reports 1.46 million data-centre GPU-hours	Text is streamed from storage, not held on the GPU	More text, more patterns learned	Cost grows with every token; data quality matters as much as size

INSIGHT

Data is one of the biggest factors in quality, alongside model size, architecture, training method and how the model is run. In our tiny case the link is direct: the text says tea 67%, so a perfect model says tea 67%. The objective rewards matching the text, not checking truth.

WATCH OUT

Bias and errors in the text become bias and errors in the model. Nothing in training checks whether the text is true.

MYTH

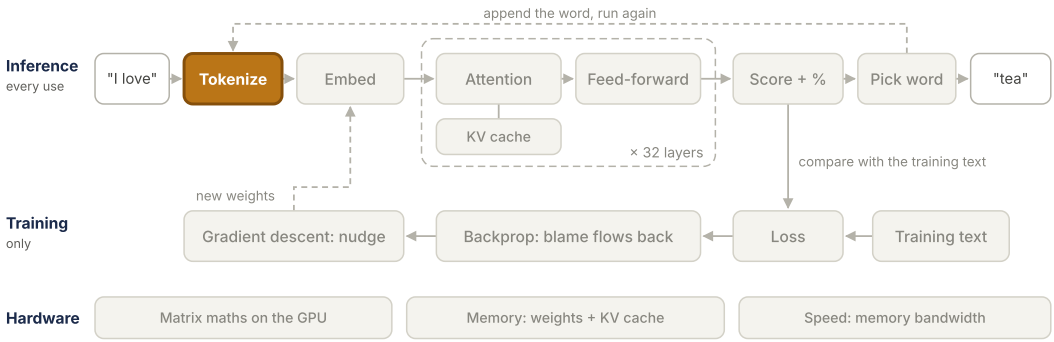
The model is taught facts and rules.

REALITY

Pre-training's only objective is predicting the next token. Large models trained this way on diverse text do pick up grammar, facts and reasoning patterns. The objective is simple; the capabilities that emerge from it at scale are not, and our 4-sentence model can't show them.

WATCH OUT

Chat assistants get a second phase after pre-training: instruction tuning and preference training (e.g. RLHF). Open models are usually released in both forms: a base model and an *instruct* (post-trained) version. Chapters 20–21 show both on our tiny model.



2 Tokenizer: building the vocabulary

A chip can only do arithmetic. The very first design job is a fixed list that turns text into numbers.

How does text become something a model can compute with?

COMES IN	WHO DOES THE WORK	HANDS ON
The training text (Ch 1) →	Tokenizer: fixed before model training	→ Token ids → the embedding table (Ch 4)

IN ONE LINE Models count tokens, not words, and so do prices and limits.

We list every distinct word in the text, in order of first appearance, and number them from 0. That list is the **vocabulary**; the numbers are **token ids**.

Token id	0	1	2	3	4	5
Word	I	love	tea	coffee	hate	rain

#	What happens	Our numbers
1	Text arrives	"I love"
2	Cut into pieces (tokens)	"I", "love"
3	Swap each for its id	0, 1

The tokenizer is **fixed before model training** and isn't updated by the model's gradient descent. Real tokenizers are themselves built by an algorithm run over text (BPE learns which pieces to merge), but that happens beforehand. Ours simply lists the words.

Our tokenizer, and a real one — Python

```
words = "I love tea I love tea I love coffee I hate rain".split()
vocab = {w: i for i, w in enumerate(dict.fromkeys(words))}
print([vocab[w] for w in "I love".split()])   # [0, 1]

from transformers import AutoTokenizer        # pip install transformers
tok = AutoTokenizer.from_pretrained("path/to/an-8b-model")  # a real sub-word vocabulary
print(tok.encode("I love tea"))                # a few ids, one per piece
```

IN A REAL MODEL (8B)

Real tokenizers split text into **sub-word pieces** (BPE), so rare words become several tokens. The 8B model's 128K-token vocabulary averages about 4 characters per token on English text.

SCALE IT UP: WHAT GROWS, AND WHAT IT COSTS

Knob	Toy → 8B model	Compute	Memory	Good	Bad
Vocabulary	6 → 128,256	Final scoring is one dot product per vocabulary entry: 4,096 × 128,256 ≈ 525 M multiply-adds per token	Embedding table 525 M numbers; output table another 525 M (≈ 1 GB each at 16-bit)	Bigger vocabulary → fewer tokens per sentence → cheaper, faster, more text fits in context	Bigger tables; rare tokens learned poorly
Tokens per text	—	Every token is one more pass	Every token adds to the KV cache	English is cheap: 100 tokens ≈ 75 words	Languages that split into more pieces per word cost more for the same text

WATCH OUT

Prices, context limits ("128K") and speed are all in tokens, not words.

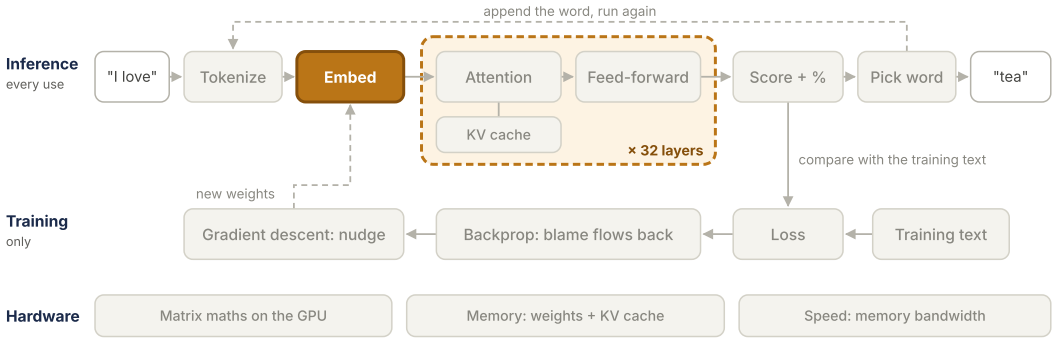
MYTH

The model reads letters like we do.

REALITY

It receives only ids. It never sees spelling, which is why counting letters in a word is surprisingly hard for it.

STORY **1 Design** › 2 First run › 3 Train › 4 Use › 5 Scale › 6 Beyond



3 The shape: slots, layers and our 80 weights

Before training, someone fixes the model's shape. Training can change the numbers but never the shape.

How big should each part be, and why did we pick these sizes?

COMES IN
The vocabulary: 6 tokens (Ch 2)

→ WHO DOES THE WORK
Us: choosing the sizes

→ HANDS ON
80 empty places for weights → filled at random (Ch 4)

IN ONE LINE The shape is fixed before training; the size knobs set the cost.

Choice	Ours	Why this number	The 8B model	What we give up
Slots per token	4	The smallest size that does three jobs: • enough room for 6 words to settle into different positions • attention can shrink rows to a smaller size (4 → 2), as the 8B model does (4,096 → 128) • a dot product stays at 4 multiplications, doable by hand **2 slots:** too crowded, and the shrink can't be shown. **8 slots:** double the arithmetic; tables stop fitting the page.	4,096	Room to separate 128K tokens and their many meanings
Layers	1	One complete pass with every number shown. The 8B model's 32 layers repeat the same steps with different weights	32	No layer-by-layer refinement
Attention heads	1, size 2	One head shows the whole mechanism; size 2 makes Q, K, V visibly smaller than the token row	32 query heads × 128; 8 key/value heads	Can't look for several patterns at once
Feed-forward detectors	3	Enough to show detectors that fire and one that's switched off	14,336 per layer	Real feed-forward is 3.5× *wider* than the row; ours is narrower to keep tables short

Where the 80 weights live

Part	Shape	Count	Same part in the 8B model	Ch
Embedding table	6 tokens × 4 slots	24	128,256 × 4,096 ≈ 525 M	4
Q, K, V matrices	3 × (4 → 2)	24	≈ 42 M per layer	6
Attention output	2 → 4	8		6
Detectors + write rows	3 × 4 + 3 × 4	24	≈ 176 M per layer	7
Total		**80**	**≈ 8.03 billion (incl. separate 525 M output table)**	

What "8B" means: count the cells

The **B** in "8B" is **billion parameters**: the number of cells in all the weight grids. Ours has 80. It is *not* the number of slots: slots are one of the knobs that make the count grow.

Knob	Ours → 8B model	Effect on the count
Slots (width)	4 → 4,096	Most grids are slots × slots, so 2× slots ≈ 4× weights
Layers	1 → 32	2× layers ≈ 2× weights
Detectors (feed-forward)	3 → 14,336	The biggest share: ≈ 80% of each layer
Vocabulary	6 → 128,256	Adds to the embedding and output tables only

B counts numbers, not how precisely each is written; that is bits (Ch 19). Memory = B × bytes per number (Ch 17).

Count the weights, ours and the 8B model's — Python

```
ours = 6*4 + 3*(4*2) + 2*4 + 3*4 + 3*4            # = 80
d, L, ff, vocab, kv = 4096, 32, 14336, 128256, 1024
attn  = d*d + 2*(d*kv) + d*d                       # Q, K, V, output
ffn   = 3 * d * ff                                 # SwiGLU uses 3 matrices
real8b = L*(attn + ffn) + 2*vocab*d                # ~8.0e9 (norm weights add ~0.3M)
print(ours, f"{real8b:,}")
```

SCALE IT UP: WHAT GROWS, AND WHAT IT COSTS

Knob	Toy → 8B model	Compute	Memory	Good	Bad
Width (slots)	4 → 4,096	Most matrices are width × width, so 2× width ≈ 4× compute per layer	≈ 4× the weights; KV cache 2×	More room to separate meanings	Cost grows with the square of width
Layers	1 → 32	∝ layers	∝ layers (weights and KV cache)	More rounds of combining information	Layers run one after another for each token: more layers, more delay

INSIGHT

"8B" is just this table filled in with big numbers. Every size knob in the 8B model is one we chose here, turned up.

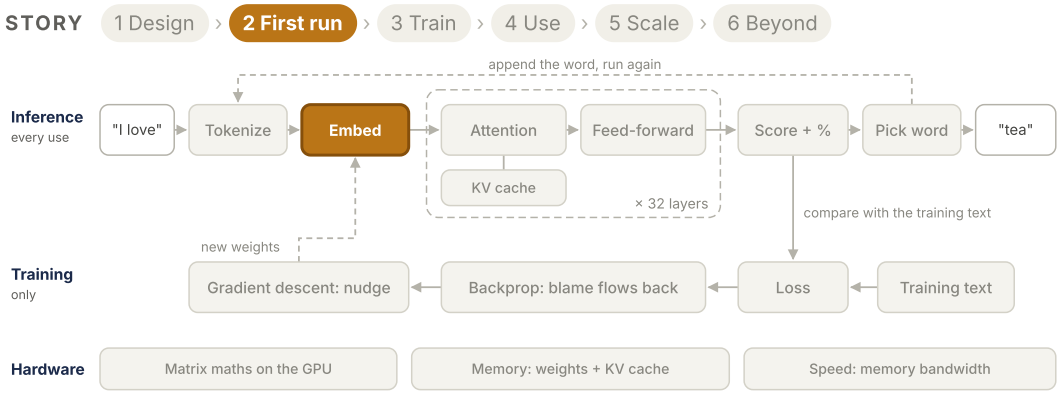
WATCH OUT

The shape is fixed once training starts. Changing slots or layers later generally means substantial retraining or special adaptation methods; training can't simply "grow" a model.

STAGE 2

First run: a model with random weights

We fill all 80 weights with random numbers and run "I love" through it. Every step works; none of it means anything yet. This is exactly where every real model starts.



4 Random birth and the embedding table

The newborn model is 80 random numbers. The first 24 form the embedding table.

What is the embedding table, and what's in it before training?

COMES IN
Token ids 0, 1 (Ch 2)

→

WHO DOES THE WORK
Embedding table: a lookup, no maths

→

HANDS ON
2 rows of 4 numbers → attention (Ch 6)

IN ONE LINE The embedding table is a learned lookup: one row per token.

- A **lookup table** with one row per token in the vocabulary.
- Each row is that token's **starting vector** (4 slots).
- The **token id is the row number**: "love" = id 1 = row 1.
- Using it is **pure lookup**: fetch the row. Its numbers are weights, so training will change them.

The newborn's embedding table (random, spread 0.5, rounded to 1 decimal)

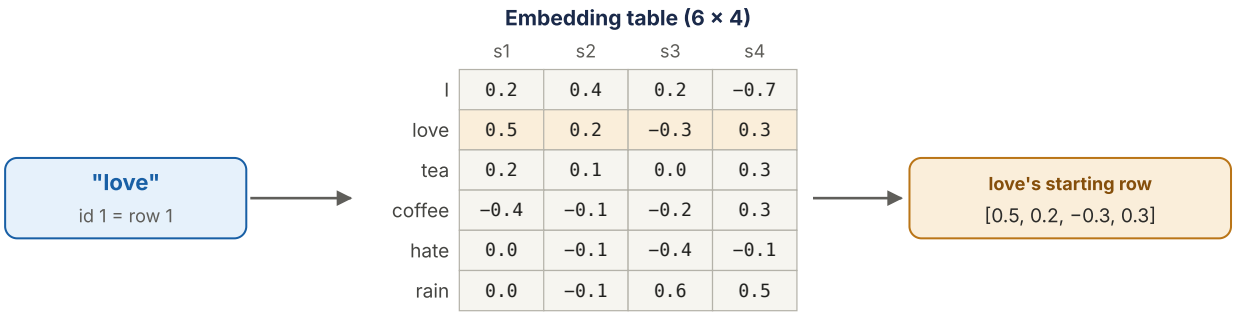
Word	Token id (row)	Slot 1	Slot 2	Slot 3	Slot 4
I	**0**	**0.2**	**0.4**	**0.2**	**−0.7**
love	**1**	**0.5**	**0.2**	**−0.3**	**0.3**
tea	2	0.2	0.1	0.0	0.3
coffee	3	−0.4	−0.1	−0.2	0.3
hate	4	0.0	−0.1	−0.4	−0.1
rain	5	0.0	−0.1	0.6	0.5

Looking up "I love" fetches rows 0 and 1: `I [0.2, 0.4, 0.2, −0.7]`, `love [0.5, 0.2, −0.3, 0.3]`.

In plain words: the embedding table is a lookup

Think of it as a numbered list. The tokenizer gives each word a number (its id); that number is simply the row to copy.

FROM A WORD TO ITS ROW



No multiplying, no adding: the word's id says which row to copy. Only the numbers inside the rows are learned.

WHAT IT SHOWS

The word "love" has id 1, so the model copies row 1 of the table: 4 numbers. Nothing is calculated.

WHY IT MATTERS

This row is the word's only form inside the model. Every later step (attention, feed-forward, scoring) works on rows like this one.

- **What it is:** a table with one row per vocabulary word and one column per slot
- **Who decides:** we choose the size (6 words × 4 slots); training sets the numbers inside
- **How it is used:** copy a row by its id. The same table is reused at the very end to score words (Ch 9), so it does two jobs

If each word were…	tea / coffee / rain	What goes wrong or right
Just its id number	2 / 3 / 5	One number, set by the order we met the words. tea looks "next to" coffee only by accident, and nothing can move
A row of 4 learned numbers	three rows of 4	Training can move rows. tea–coffee similarity went 0.00 → 0.92; tea–rain went 0.48 → 0.22

Random birth and lookup — Python

```
import numpy as np
rng = np.random.default_rng(1)
E = rng.normal(0, 0.5, size=(6, 4)).round(1)   # the embedding table
ids = [0, 1]                                    # "I love"
X = E[ids]                                      # lookup = row indexing
```

IN A REAL MODEL (8B)

The 8B model's table is 128,256 rows × 4,096 slots ≈ 525 million numbers (≈ 1 GB at 16-bit). It starts random too: its config sets the initial spread (initializer_range) to 0.02.

SCALE IT UP: WHAT GROWS, AND WHAT IT COSTS

Knob	Toy → 8B model	Compute	Memory	Good	Bad
Rows × slots	6 × 4 → 128,256 × 4,096	Lookup costs almost no compute	≈ 525 M numbers ≈ 1 GB at 16-bit	Every token gets a rich starting point	Must sit in GPU memory even though each token uses one row

INSIGHT

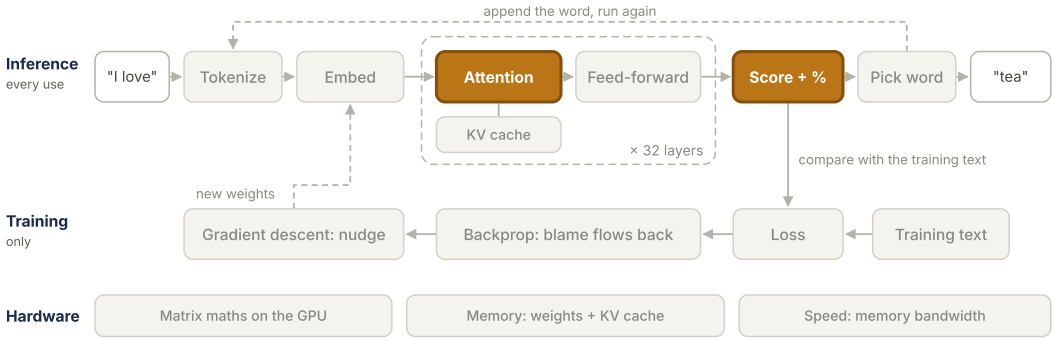
Slots have no names. Slot 3 doesn't mean anything; it's just position 3. Any meaning appears only after training (Ch 13).

MYTH

Someone designs the embedding table.

REALITY

It begins as random noise. Training shapes it.



5 Dot product: the one operation behind everything

Attention and scoring both rely on one question: how much do two rows agree?

How does the model put a number on "how related are these two rows?"

COMES IN	WHO DOES THE WORK	HANDS ON
Rows from the embedding table (Ch 4)	Dot product: multiply slot by slot, add up	A score → used by attention (Ch 6) and the scorer (Ch 9)

IN ONE LINE A dot product scores agreement, and most of the model is dot products.

Multiply two rows slot by slot, then add. One number comes out. It's a compatibility score for direction: big positive = they point the same way; near zero = no shared direction; negative = opposite. Near zero doesn't always mean "unrelated in meaning". It means these particular numbers don't line up.

NEWBORN NUMBERS: TEA VS COFFEE

```
tea    :  0.2   0.1   0.0   0.3
coffee : −0.4  −0.1  −0.2   0.3
product: −0.08 + −0.01 + 0.00 + 0.09  =  0.00
```

Newborn tea and coffee score **0.00**: unrelated, purely by chance. After training they score **1.23** (Ch 13). Nobody will tell the model they're similar; training will find it.

Dot product and cosine similarity — Python

```python
import numpy as np
def dot(a, b): return float(np.dot(a, b))          # multiply pairs, add
def cos(a, b): return dot(a, b) / (np.linalg.norm(a) * np.linalg.norm(b))
tea, coffee = np.array([0.2, 0.1, 0.0, 0.3]), np.array([-0.4, -0.1, -0.2, 0.3])
print(dot(tea, coffee), cos(tea, coffee))         # 0.0  0.0
```

IN A REAL MODEL (8B)

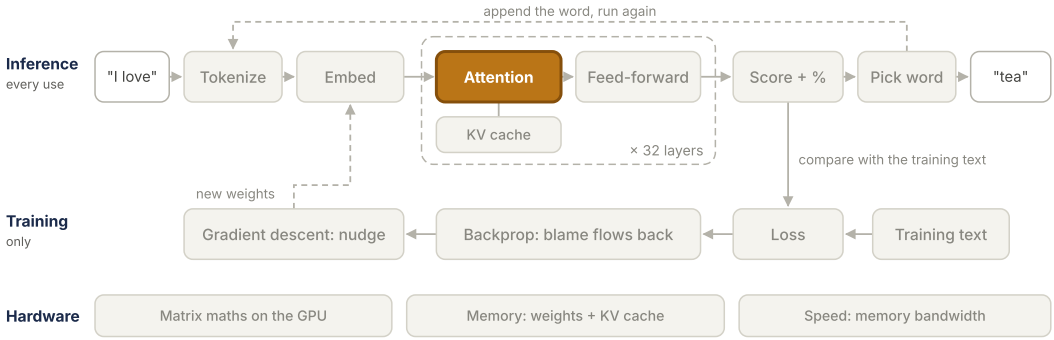
In every layer, each query is compared with every earlier key: one dot product of 128 numbers per head per earlier token. The final scoring is 128,256 dot products of 4,096 numbers, for every token.

SCALE IT UP: WHAT GROWS, AND WHAT IT COSTS

Knob	Toy → 8B model	Compute	Memory	Good	Bad
Slots per dot product	4 → 4,096 (128 inside a head)	Cost ∝ slots	None extra	Richer comparisons	Tens of billions of them per token
Number precision	16-bit → 8-bit	GPUs run roughly 2× the 8-bit operations per second	½ the bytes	Faster and smaller	Small rounding errors

WATCH OUT

Length inflates the score. Dividing by both lengths gives **cosine similarity** (−1 to 1), which compares direction only.



6 Attention: a word pulls information from earlier words

This is the only step where words exchange information. Everything else handles each word alone.

How does "love" pull in information from "I"?

COMES IN	WHO DOES THE WORK	HANDS ON
Rows for "I" and "love" (Ch 4)	Attention: Q, K, V weights (32)	Updated "love" row → feed-forward (Ch 7)

IN ONE LINE Attention is the only step where tokens read each other.

Each token makes three smaller rows by multiplying its row by three weight matrices:

Row	Its job	"love" (newborn)	"I" (newborn)
Q query	Compared with every key: decides what this token pulls in	[−0.42, −0.70]	—
K key	What each token is matched on	[0.08, 0.83]	[−0.05, 0.09]
V value	What gets copied if matched	[−0.30, −0.34]	[−0.12, −0.05]

#	What happens	Our numbers
1	Q of "love" · each K, divided by √2 (√ of the head size) to keep scores in range	with I: −0.03 with love: −0.43
2	Softmax the scores into shares (Ch 9 explains softmax)	I 60%, love 40%
3	Blend the V rows by those shares	0.60 × V(I) + 0.40 × V(love) = [−0.19, −0.17]
4	An output matrix turns 2 numbers back into 4, which are added to love's row	[0.5, 0.2, −0.3, 0.3] + [−0.12, −0.03, −0.10, 0.16] = **[0.38, 0.17, −0.41, 0.46]**

Look backwards only: a token may only attend to itself and earlier tokens (the **causal mask**). "I" sees only itself (100%). This is what makes it a next-word predictor: it can't peek at the answer.

Follow the arrays: what each grid holds

Everything in attention is a small grid of numbers. **Rows are always tokens** ("I", "love"); columns are slots or other tokens. Here are the real newborn numbers, step by step.

1 · STACK THE ROWS

Embedding table (6 rows × 4 slots)

	slot 1	slot 2	slot 3	slot 4
I	0.2	0.4	0.2	−0.7
love	0.5	0.2	−0.3	0.3
tea	0.2	0.1	0.0	0.3
coffee	−0.4	−0.1	−0.2	0.3
hate	0.0	−0.1	−0.4	−0.1
rain	0.0	−0.1	0.6	0.5

copy rows 0, 1 →

X: rows stacked for "I love" (2 × 4)

	slot 1	slot 2	slot 3	slot 4
I	0.2	0.4	0.2	−0.7
love	0.5	0.2	−0.3	0.3

Each **row** = one token.
Each **column** = one slot.
8B model: n tokens × 4,096 slots.

WHAT IT SHOWS

The embedding table holds one row of 4 numbers for each of the 6 words. For "I love", rows 0 and 1 are copied and stacked into a grid X: one row per token, one column per slot.

WHY IT MATTERS

Everything after this works on grids where rows are tokens. Keeping that one rule in mind makes every later grid readable.

2 · MAKE Q, K, V (ONE LEARNED MATRIX EACH)

X (2 × 4)

I	0.2	0.4	0.2	−0.7
love	0.5	0.2	−0.3	0.3

×

Wq (4 × 2): random for now

−1.4	−0.9
−0.1	−0.2
0.1	0.1
1.1	−0.6

=

Q (2 × 2)

I	−1.07	0.18
love	−0.42	−0.70

K = X · Wk (2 × 2)

I	−0.05	0.09
love	0.08	0.83

V = X · Wv (2 × 2)

I	−0.12	−0.05
love	−0.30	−0.34

Same X, two other weight matrices (Wk, Wv).

Each Q number = dot product of one X row with one Wq column (4 multiply-adds), e.g.
Q[love, 1] = 0.5×(−1.4) + 0.2×(−0.1) + (−0.3)×0.1 + 0.3×1.1 = −0.42

WHAT IT SHOWS

X is multiplied by three weight matrices (Wq, Wk, Wv; random here, set by training in Ch 12) to make Q, K and V, three smaller grids with one row per token. The worked example shows one Q cell: 4 multiply-adds.

WHY IT MATTERS

Q, K and V aren't new information. They're three views of the same rows. Training (Ch 12) shapes each matrix for its job: Q asks, K is matched against, V is what gets copied.

3 · SCORES → SHARES → BLEND → UPDATED ROWS

Scores = Q · Kᵀ ÷ √2

	vs I	vs love
I asks	0.05	hidden
love asks	−0.03	−0.43

→

Shares (softmax per row)

	from I	from love
I	100%	hidden
love	60%	40%

→

Blend = shares · V

I	−0.12	−0.05
love	−0.19	−0.17

H = X + (blend · Wo): updated rows (2 × 4)

	slot 1	slot 2	slot 3	slot 4
I	0.14	0.37	0.16	−0.63
love	0.38	0.17	−0.41	0.46

Same shape as X: 2 tokens × 4 slots, now each row carries information from earlier rows. → feed-forward (Ch 7)

WHAT IT SHOWS

Scores: every row of Q against every row of K (look-ahead cells hidden). Shares: each score row turned into percentages that add to 100%. Blend: shares × V. The output H has the same shape as X, but each row now carries information from earlier rows.

WHY IT MATTERS

This is the only place where rows exchange information. The scores grid is tokens × tokens, which is why attention cost grows with the square of the context length.

Grid	Shape (ours)	Shape (8B model, per head)	What one cell means
X (stacked rows)	2 × 4	n × 4,096	One slot of one token
Q, K, V	2 × 2 each	n × 128 each	One number of that token's query / key / value
Scores	2 × 2	n × n	How well token A's query matches token B's key (a dot product, ÷ √ head size)
Shares	2 × 2	n × n	The percentage of token B that token A pulls in; each row adds to 100%
H (updated rows)	2 × 4	n × 4,096	One slot of one token, after attention

TWO DIFFERENT "SCORES" IN THIS GUIDE

Attention scores (this page): one token's query against an earlier token's key, so they decide *who to pull from*. **Vocabulary scores** (Ch 9, the "Score + %" box on the map): the final row against every word in the vocabulary, so they decide *which word comes next*. Same operation (dot product), different job.

INSIGHT

The scores grid is n × n per head, per layer. For 1,000 tokens that's 1,000,000 scores, × 32 heads × 32 layers. That's why long prompts get expensive, and why the cache stores K and V (the columns) so new rows can be added one at a time.

The hidden cell: a locked door, not an empty box

In the scores grid above, one cell says **hidden**: "I asks" × "vs love". It is not missing data and it is not zero. It is **locked**. Here is what that means, with no maths needed.

WHICH CELLS ARE LOCKED, AND WHY

Scores grid for "I love tea" (3 × 3)

	vs I	vs love	vs tea
I asks	✔	locked	locked
love asks	✔	✔	locked
tea asks	✔	✔	✔

One pass = three guessing tasks

At	Can see	Must guess
I	I	love
love	I, love	tea
tea	I, love, tea	end (nothing)

Each word may look at itself and earlier words (green). Cells that look at a later word are locked (grey).

WHAT IT SHOWS

Left: for a 3-word sentence, every cell where a word would look at a *later* word is locked. Right: because of those locks, one pass over "I love tea" gives three separate guessing tasks, each using only the past.

WHY IT MATTERS

During training the whole sentence goes in at once. Without the locks, "I" could simply read "love" from the next row and copy the answer. The locks keep every guess honest.

- **What is locked:** only cells in the **scores grid**. Q, K and V are made normally for every word; nothing is removed from the input.
- **Who decides:** we do, when designing the model. It is a fixed rule, like choosing 4 slots. No weight controls it and training never changes it.
- **How it is done:** one line of code puts −∞ (minus infinity) in every locked cell before softmax. Softmax turns −∞ into exactly 0%.

If the hidden cell held…	"I" row shares (from I / from love)	Result
0 (an empty box)	51.2% / 48.8%	Leaks: "I" still pulls almost half its information from the future word
−∞ (a locked door)	100% / 0%	Blocked: "I" sees only itself

Real newborn numbers: "I" scores 0.049 against itself. A score of 0 still earns a share in softmax, which is why "empty" is not enough.

WATCH OUT: TWO KINDS OF "HIDE AND GUESS"

Our TLM and chat models (like the 8B): the *future* is locked; each word guesses the next one using only the past. This is what lets a model write text, one word at a time.
Fill-in-the-blank models: random words in the input are hidden ("I [blank] tea") and guessed using words on *both* sides. Good for understanding and search, not for writing.

INSIGHT

At use time the locks change nothing: the next word hasn't been written yet, so there is nothing to peek at. They exist so training matches use. A bonus: since a row never looks ahead, an earlier word's K and V rows never change once computed. That is exactly what makes the KV cache (Ch 15) possible.

One attention head in NumPy — Python

```python
def attention(X, Wq, Wk, Wv, Wo):
    Q, K, V = X @ Wq, X @ Wk, X @ Wv
    s = Q @ K.T / np.sqrt(K.shape[1])               # every query vs every key
    s += np.triu(np.full(s.shape, -np.inf), 1)      # causal mask: no peeking ahead
    w = np.exp(s - s.max(1, keepdims=True)); w /= w.sum(1, keepdims=True)
    return (w @ V) @ Wo                             # blend values, back to 4 slots
```

IN A REAL MODEL (8B)

32 query heads per layer, each with rows of 128 numbers. Keys and values use only 8 heads, each shared by 4 query heads (grouped-query attention), which shrinks the cache 4×. The 8B model also rotates Q and K by position (RoPE), which our model omits.

SCALE IT UP: WHAT GROWS, AND WHAT IT COSTS

Knob	Toy → 8B model	Compute	Memory	Good	Bad
Context length	2 → up to 131,072 tokens	Every token scores every earlier one: 16× more tokens ≈ 256× more score work	KV cache grows in step with tokens (128 KiB each)	Model can use more of your document	Long prompts are slow and pricey
K/V heads	1 → 8 (shared by 32 query heads)	Barely changes	4× smaller cache than 32 separate K/V heads	More users and longer context per GPU	A trade-off chosen at training time

INSIGHT

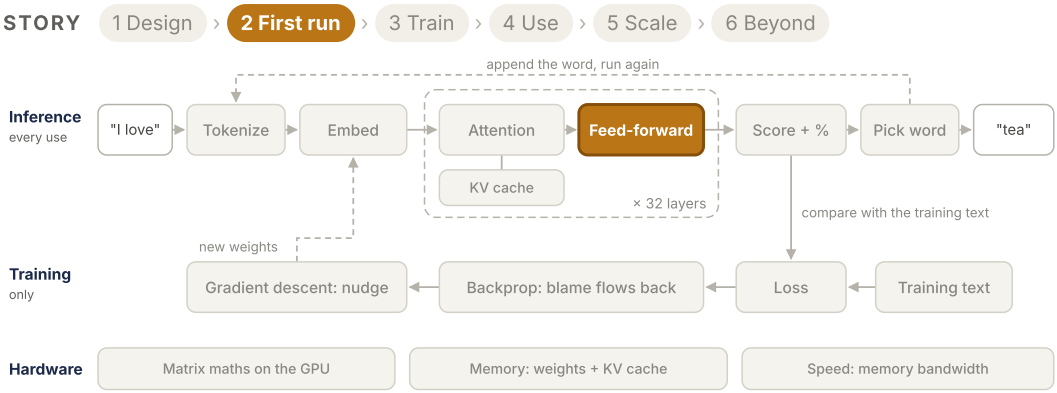
Attention is the only place where one token reads another, so text in your prompt influences the answer through attention. That's the in-model half of RAG. RAG as a system has three steps: **retrieve** relevant passages, **insert** them into the prompt, **generate**. Inserting text makes it available; it doesn't guarantee the model uses it correctly.

MYTH

Attention means the model focuses like a person.

REALITY

It's a weighted average: here 60% of one value row and 40% of another, blended and added.



7 Feed-forward: detectors, a switch, and a write

Attention mixes words. Feed-forward adds information from the weights themselves, one word at a time.

What happens to "love" after attention?

COMES IN	WHO DOES THE WORK	HANDS ON
Updated "love" row (Ch 6)	Feed-forward: 3 detectors (24 weights)	A change to add to "love" → shortcut (Ch 8)

IN ONE LINE Feed-forward transforms each token on its own, using stored weights.

#	What happens	Our numbers
1	**Detect**: each detector takes a dot product of the row with its own weights	[0.38, 0.17, −0.41, 0.46] → scores [0.34, −0.06, 0.15]
2	**Switch** (ReLU): negative → 0, positive unchanged	[0.34, 0.00, 0.15]: detector 2 is off
3	**Write**: each detector adds its own stored row, scaled by how strongly it fired	change = [−0.30, −0.07, 0.19, −0.06]

A single detector is what textbooks call a **neuron**: a row of weights and one dot product.

Why the switch matters: without it, "detect" and "write" collapse into one fixed table, so stacking 32 layers would be no better than one. With it, *which* detectors fire depends on the input. Chapter 13 measures the difference in the trained model.

Follow the arrays: feed-forward, row by row

Same rule as attention: **rows are tokens**. The difference is that each row is handled **on its own**: no row looks at another row here.

1 · EVERY TOKEN ROW MEETS EVERY DETECTOR ROW

H: rows after attention (2 × 4)

	slot 1	slot 2	slot 3	slot 4
I	0.14	0.37	0.16	−0.63
love	0.38	0.17	−0.41	0.46

·

Detectors W1 (3 × 4): random for now

	slot 1	slot 2	slot 3	slot 4
D1	0.5	0.0	−0.6	−0.2
D2	0.0	0.1	−0.5	−0.6
D3	0.1	−0.2	0.1	0.4

Every token row is dotted with every detector row: 2 tokens × 3 detectors = 6 dot products of 4 numbers.
e.g. love · D1 = 0.38×0.5 + 0.17×0 + (−0.41)×(−0.6) + 0.46×(−0.2) ≈ 0.34

WHAT IT SHOWS

Each token row (from attention) is compared with each detector row by a dot product, giving 2 tokens × 3 detectors = 6 scores.

WHY IT MATTERS

After training (Ch 12), each detector's row is a pattern it learned; its score says how strongly a token's row matches that pattern. Here the weights are still random, so the scores mean nothing yet.

2 · SCORE → SWITCH → WRITE

Detector scores (2 × 3)

	D1	D2	D3
I	0.10	0.34	−0.29
love	0.34	−0.06	0.15

→

After the switch (ReLU)

	D1	D2	D3
I	0.10	0.34	hidden
love	0.34	hidden	0.15

Negative → 0 (grey cells: that detector is off for that token).

Write rows W2 (3 × 4): random for now

	slot 1	slot 2	slot 3	slot 4
D1	−0.8	0.1	0.6	−0.1
D2	−0.4	0.4	0.1	0.4
D3	−0.2	−0.7	−0.1	−0.2

→

Change = switched scores · W2 (2 × 4)

	slot 1	slot 2	slot 3	slot 4
I	−0.22	0.14	0.10	0.12
love	−0.30	−0.07	0.19	−0.06

Each token adds up the write rows of its active detectors, scaled by how strongly each fired. e.g. love: 0.34 × D1 + 0.15 × D3.

WHAT IT SHOWS

Negative scores are switched off (grey cells). Each remaining detector adds its own write row, scaled by how strongly it fired, giving a change for each token.

WHY IT MATTERS

Different tokens switch on different detectors. That input-dependent switching is what lets stacked layers do more than a single layer could.

3 · ADD THE CHANGE TO THE ROWS

H (2 × 4)

I	0.14	0.37	0.16	−0.63
love	0.38	0.17	−0.41	0.46

+

change (2 × 4)

	−0.22	0.14	0.10	0.12
	−0.30	−0.07	0.19	−0.06

=

final rows H2 (2 × 4)

−0.07	0.51	0.25	−0.50
0.08	0.10	−0.22	0.40

Shortcut: add, don't replace. The "love" row (bottom) goes on to scoring (Ch 9); in a real model, to the next layer.

WHAT IT SHOWS

The change is added to the rows that came in (H + change = H2). Nothing is overwritten.

WHY IT MATTERS

Adding instead of replacing keeps the original information alive through every layer. In the 8B model, H2 becomes the input of the next layer.

Grid	Shape (ours)	Shape (8B model)	What one cell means
H (rows in)	2 × 4	n × 4,096	One slot of one token
Detectors W1	3 × 4	14,336 × 4,096 (plus a gate of the same size)	One weight of one detector
Detector scores	2 × 3	n × 14,336	How strongly one detector fires for one token
Write rows W2	3 × 4	14,336 × 4,096	What one detector adds to one slot
Change / H2	2 × 4	n × 4,096	One slot of one token, after feed-forward

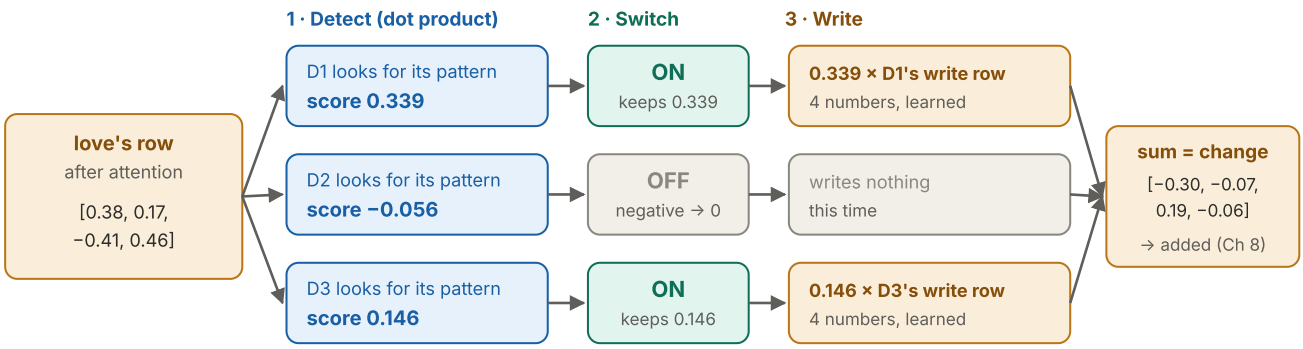
INSIGHT

Compare the grids: attention has an n × n grid (tokens × tokens), so rows mix. Feed-forward only has n × detectors grids, so each row is processed alone. That's why feed-forward cost grows in a straight line with tokens, while attention scores grow with tokens².

In plain words: three detectors, a switch, a write

Feed-forward is three small machines working on one word's row. Each one asks "does my pattern appear here?", stays quiet if the answer is no, and writes its stored row if the answer is yes.

INSIDE FEED-FORWARD, FOR "LOVE" (RANDOM WEIGHTS)



WHAT IT SHOWS

Each detector scores love's row with one dot product. The switch keeps positive scores and turns negative ones into 0. Each detector that is on writes its own 4-number row, scaled by its score. The three writes are summed into one change.

WHY IT MATTERS

Different words switch different detectors on, by different amounts. That is how one set of weights can treat "love" and "hate" differently.

- **Detect:** W1 holds one row of 4 numbers per detector: the pattern it looks for. Learned
- **Switch:** a fixed rule we chose (ReLU): below 0 becomes 0. Not learned
- **Write:** W2 holds one row of 4 numbers per detector: what it adds when on. Learned
- **Who decides:** we choose 3 detectors and the switch rule; training sets all 24 numbers in W1 and W2

Case	D1	D2	D3	What it means
Trained, "I love"	2.32 on	−0.81 off	3.11 on	Same detectors, different strengths, so a different change is written for each word
Trained, "I hate"	5.21 on	−0.37 off	3.64 on	

WHAT IF THERE WERE NO SWITCH?

Without the switch: detect + write = one fixed 4 × 4 table

	s1	s2	s3	s4
s1	−0.42	−0.02	0.29	−0.07
s2	0.00	0.18	0.03	0.08
s3	0.66	−0.33	−0.42	−0.16
s4	0.32	−0.54	−0.22	−0.30

Every row would get the same treatment: multiply by this one table. No "if".

Stack 32 such layers and they still equal one table. The switch prevents that.

WHAT IT SHOWS

Without the switch, "detect then write" is the same as multiplying by this single 4 × 4 table ($W1^T \cdot W2$, real newborn numbers). With the switch, newborn love's change is [−0.30, −0.07, 0.19, −0.06]; without it, D2's negative score leaks in: [−0.28, −0.09, 0.18, −0.09].

WHY IT MATTERS

The switch is the only step in the model that can say "do this for some inputs and not others". Everything else is multiply and add.

HONEST NOTE

In our one-layer model, removing the switch from the trained model only moves "I love" → tea from 67% to 65%. The switch matters most when layers stack: it is what stops 32 layers collapsing into one.

Feed-forward in NumPy — Python

```python
def feed_forward(H, W1, W2):
    scores = H @ W1.T                # detect: one dot product per detector
    on = np.maximum(scores, 0)       # switch: ReLU
    return on @ W2                   # write: scaled sum of write rows
```

IN A REAL MODEL (8B)

14,336 detectors per layer, 32 layers. The 8B model's switch is SwiGLU: a second set of 14,336 weights acts as a gate, so it uses 3 matrices instead of 2.

SCALE IT UP: WHAT GROWS, AND WHAT IT COSTS

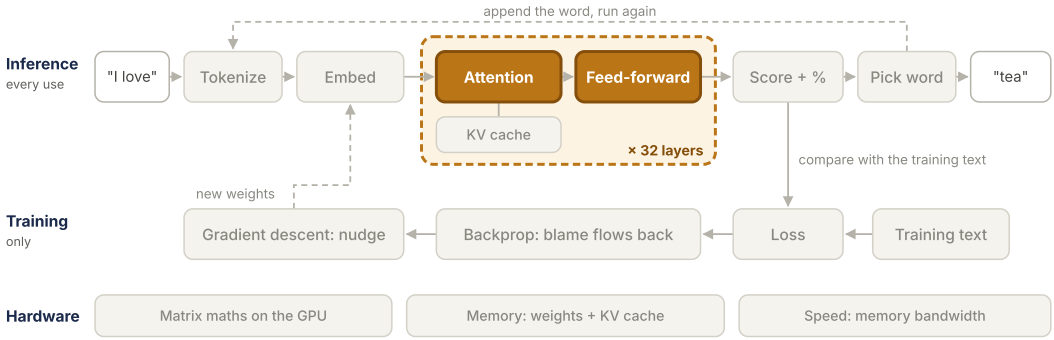
Knob	Toy → 8B model	Compute	Memory	Good	Bad
Detectors	3 → 14,336 per layer	Every token runs every detector: ≈ 80% of each layer's work	≈ 80% of the weights (≈ 176 M of ≈ 218 M per layer)	Room for many stored patterns	Every token pays for all of them
Switch type	ReLU → SwiGLU	+50% feed-forward compute	+50% feed-forward weights	Better quality per weight (found by experiment)	An extra matrix

INSIGHT

A useful mental model: attention **moves** information between tokens; feed-forward **transforms** each token using its weights. Research finds much factual recall involves feed-forward layers, but knowledge and computation are spread across both, and across many layers. Treat the split as a guide, not a complete map.

WATCH OUT

Real detectors rarely mean one clean idea. Each responds to a mix of patterns, and each idea is spread over many detectors (superposition).



8 One layer: add, don't replace

Attention and feed-forward each compute a change. The layer adds both changes to the original row.

How do the pieces combine inside one layer?

COMES IN
Changes from attention (Ch 6) and feed-forward (Ch 7)

→ WHO DOES THE WORK
Shortcut connections: plain addition

→ HANDS ON
Final "love" row → scorer (Ch 9)

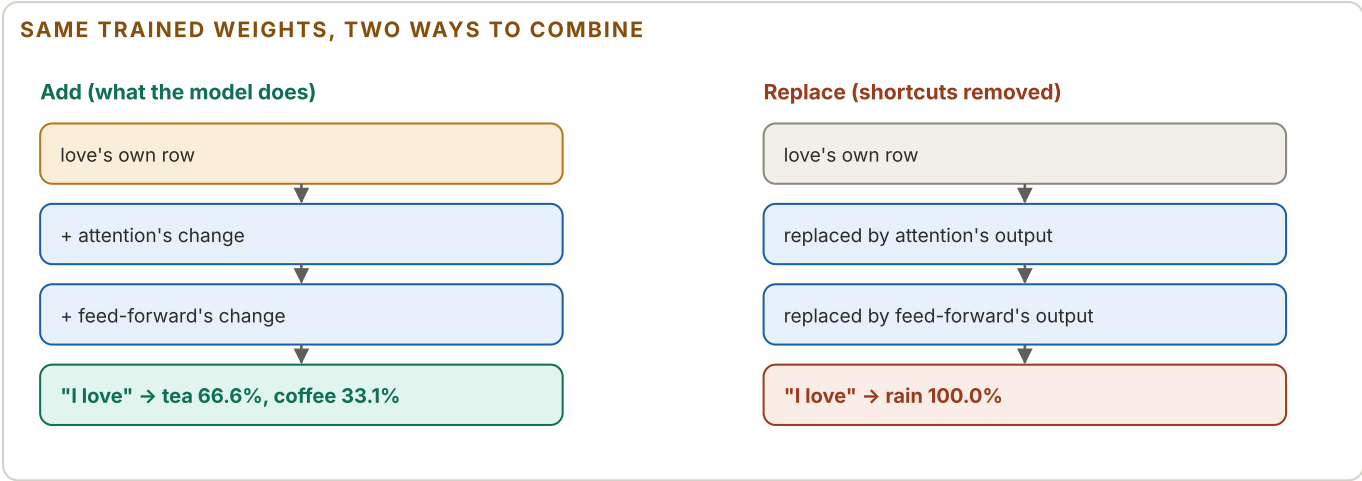
IN ONE LINE Each layer adds changes; it never replaces the row.

Stage	Operation	"love" row (newborn)
Start	Embedding lookup	`[0.50, 0.20, −0.30, 0.30]`
Attention	Change from earlier tokens	`+[−0.12, −0.03, −0.10, 0.16]`
Shortcut	Add	`=[0.38, 0.17, −0.41, 0.46]`
Feed-forward	Detect, switch, write	`+[−0.30, −0.07, 0.19, −0.06]`
Shortcut	Add	**`=[0.08, 0.10, −0.22, 0.40]`**

These added paths are called **residual shortcuts**. The original word is never overwritten, only adjusted. Without them, 32 layers of rewriting would wash out the original token.

In plain words: add, don't replace

Each layer keeps a running total for every word. Attention and feed-forward each suggest a change; the change is added to the total, never written over it.



WHAT IT SHOWS

Left: the real model, where love's own row stays in the total. Right: the same trained weights, but each step overwrites the row instead of adding to it.

WHY IT MATTERS

With the original row thrown away, the model no longer "knows" the word was love, and the answer collapses to rain. The shortcut is what carries the word itself through every layer.

- **What it is:** a plain + after attention and after feed-forward
- **Who decides:** us, in the design. It has no weights and is never trained
- **Bonus for training:** blame (Ch 11) flows back through a + unchanged, which is a big reason 32-layer models can be trained at all

Fair warning: this model was trained *with* shortcuts, so removing them afterwards is a harsh test. It still shows how much the final row depends on the word's own row.

One full layer — Python

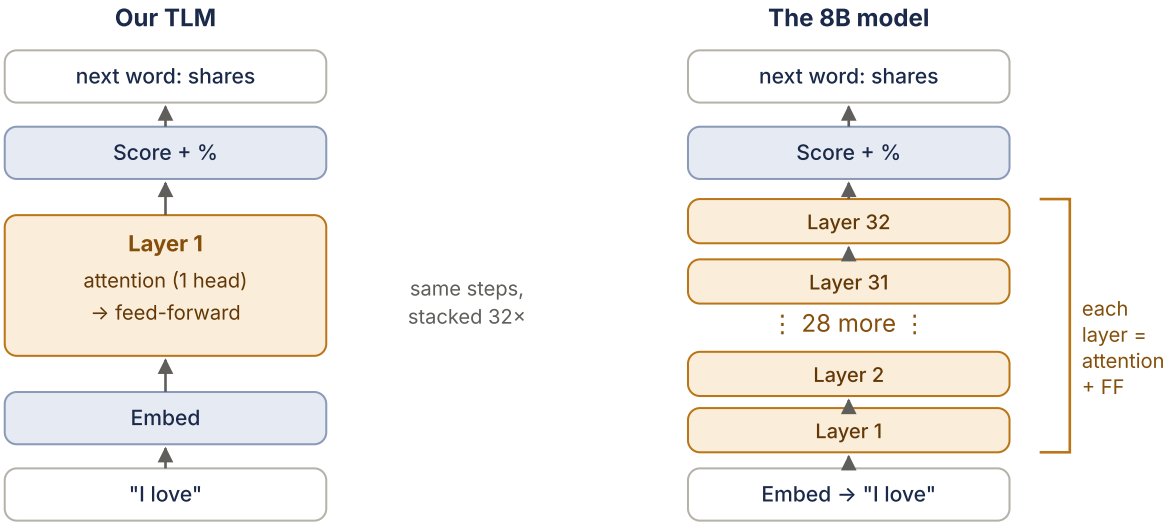
```
H  = X + attention(X, Wq, Wk, Wv, Wo)     # add attention's change
H2 = H + feed_forward(H, W1, W2)          # add feed-forward's change
```

Zoom out: what "heads" and "layers" are

Our TLM has **1 layer** with **1 attention head**. The 8B model has the same parts, repeated in two directions:

- **Layers** are stacked **vertically**: the output row of layer 1 is the input of layer 2, and so on up to 32.
- **Heads** sit **side by side inside one layer**: several independent attention calculations run in parallel, each with its own Q, K, V weights.

ZOOM LEVEL 1: THE WHOLE MODEL



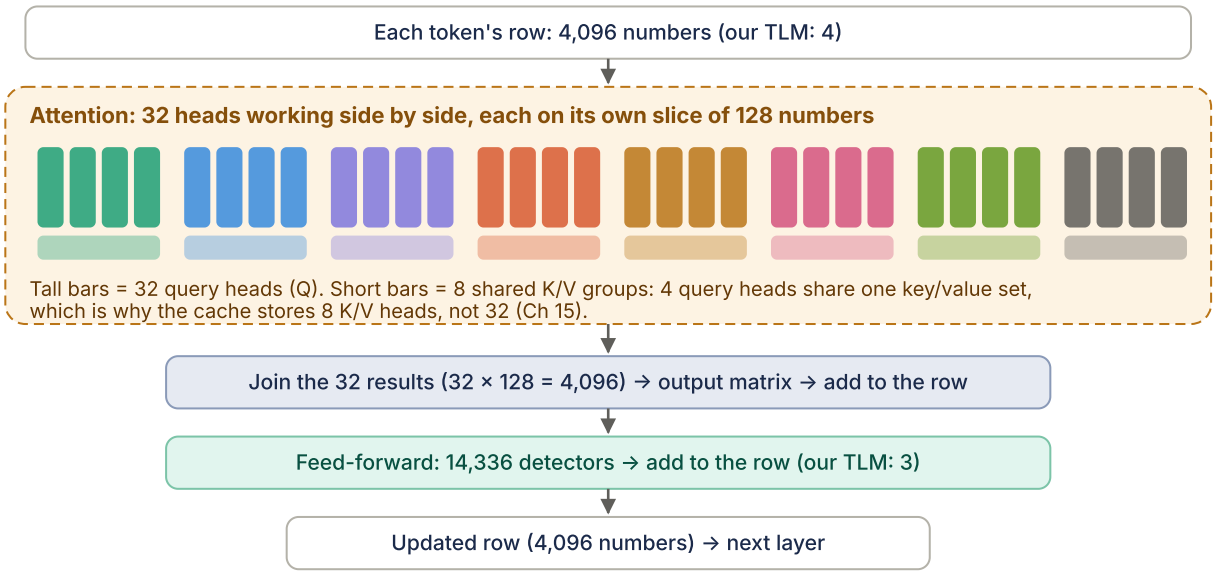
WHAT IT SHOWS

The whole model, bottom to top. Text becomes rows (Embed), passes through layers, and the top row is scored against every word. Our TLM has 1 layer; the 8B model stacks the same layer design 32 times.

WHY IT MATTERS

A layer is one repeated unit, not a different idea. Each one works on the output of the one below, and every added layer adds its weights, its compute and its KV cache.

ZOOM LEVEL 2: INSIDE ONE LAYER (8B MODEL)



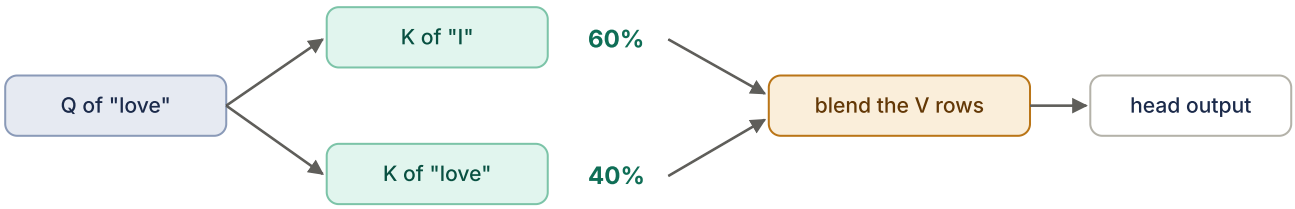
WHAT IT SHOWS

One layer of the 8B model opened up. Each token's 4,096 numbers are cut into 32 slices of 128, and 32 heads work on them at the same time. Every 4 query heads share one key/value set (same colour). The results are joined back to 4,096 numbers, then feed-forward runs.

WHY IT MATTERS

Heads let one token pull in many differently weighted blends at once for about the same cost as one big head. Sharing K/V across 4 heads is why the cache stores 8 heads instead of 32, cutting its memory 4×.

ZOOM LEVEL 3: INSIDE ONE HEAD (OUR TLM'S NUMBERS, CH 6)



WHAT IT SHOWS

A single head, with our TLM's real numbers: the query of "love" is compared with each key, the comparison becomes shares (60% / 40%), and the shares blend the value rows.

WHY IT MATTERS

This small calculation is all one head does. The 8B model runs it 32 heads × 32 layers = 1,024 times for every token.

Chapter 6 shows this head as grids of real numbers, from stacked rows to scores, shares and updated rows.

	Our TLM	The 8B model
Layers (stacked)	1	32
Heads per layer (side by side)	1	32 query heads, 8 shared K/V groups
Numbers each head works with	2	128 (= 4,096 ÷ 32)
Attention calculations per token	1 × 1 = 1	32 × 32 = 1,024
Feed-forward per token	1 layer × 3 detectors	32 layers × 14,336 detectors

Why several heads?

- One head produces **one** set of shares: one way of deciding which earlier words matter.
- With 32 heads, a token can pull in 32 differently weighted blends at once, because each head has its own Q, K and V weights and so learns to match on different patterns.
- Splitting 4,096 numbers into 32 slices of 128 keeps the cost about the same as one big head.

Why several layers?

- Each layer works on the **result** of the layer below, so later layers can combine what earlier layers gathered.
- Each layer adds a change to the running row (shortcut), so information from the start survives all 32 layers.

WATCH OUT

Nobody assigns jobs to heads or layers. Training shapes them. Researchers have found some heads with readable patterns (for example, one that mostly looks at the previous token), but many heads have no simple description.

INSIGHT

Cost follows the count: KV cache per token = 2 × **layers** × **K/V heads** × head size × bytes. That's why the 8B model's 32 layers and 8 K/V heads give 128 KiB per token (Ch 15).

IN A REAL MODEL (8B)

32 layers in a row, same steps, different weights. Before each sub-step the 8B model rescales the row to a standard size (RMSNorm) to keep numbers stable over 32 layers; our model omits it.

SCALE IT UP: WHAT GROWS, AND WHAT IT COSTS

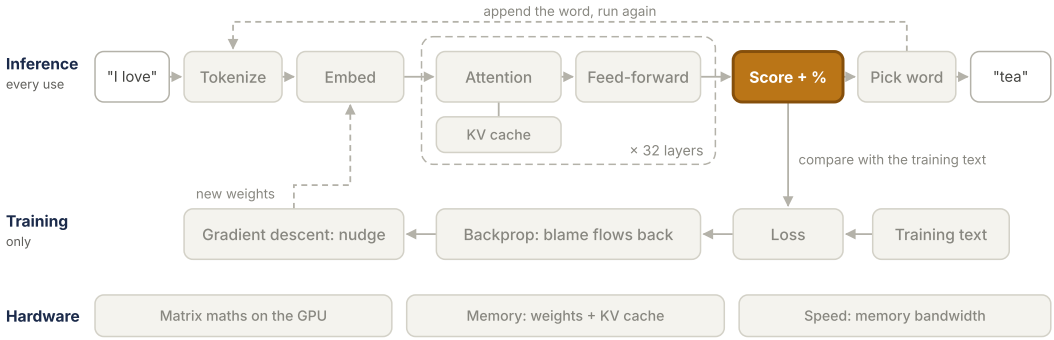
Knob	Toy → 8B model	Compute	Memory	Good	Bad
Layers	1 → 32	∝ layers	∝ layers	More rounds of combining	Each token waits for all 32 in turn

MYTH

Each layer replaces the word with something new.

REALITY

Each layer adds a correction to a running total. After 32 layers the row is the original embedding plus 64 additions.



9 Scores to percentages: the newborn's guess

The final row now votes for each word in the vocabulary. For a random model, the vote is noise.

How does a row of numbers become "x% tea"?

COMES IN
Final "love" row (Ch 8)

→ WHO DOES THE WORK
Scorer: dot products + softmax

→ HANDS ON
Percentages → loss (Ch 10) or picker (Ch 14)

IN ONE LINE The output is a percentage for every token, not a word.

#	What happens	Our numbers
1	**Score** each word: dot product of the final row with that word's embedding row. Raw scores are called **logits**	I −0.26, love 0.24, tea 0.14, coffee 0.12, hate 0.04, rain 0.06
2	**Softmax**: raise e (≈ 2.72) to each score, then divide by the total so all shares add to 100%	see table

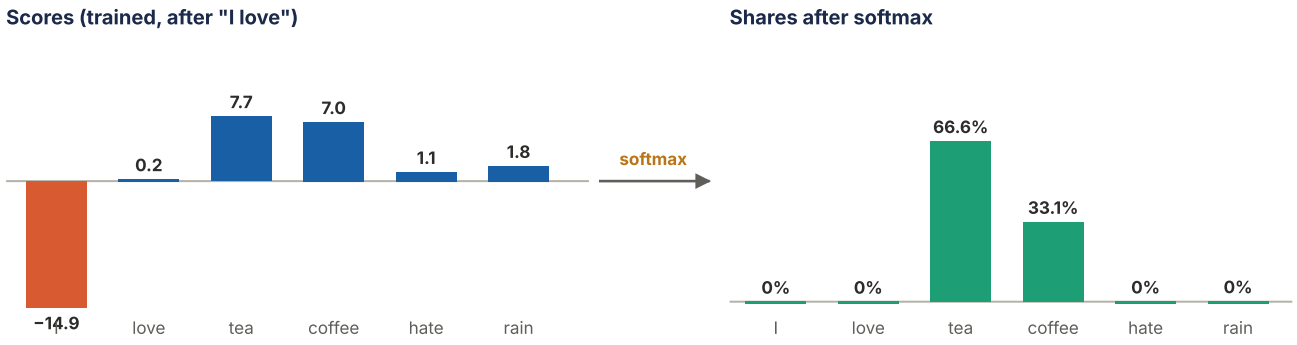
	I	love	tea	coffee	hate	rain
Newborn share	12.0%	19.8%	18.0%	17.6%	16.1%	16.5%

Every word gets 12–20%. Tea is at **18%**; the target is 67%. **Stage 2 ends here:** every part works, but the weights are random, so the answer is noise. Stage 3 fixes the weights.

In plain words: why softmax, not just "divide by the total"

Scores can be any size and any sign. Shares must be 0–100% and add to 100%. Softmax is the conversion: raise e (≈ 2.72) to each score, then divide by the total.

FROM SCORES TO SHARES (TRAINED MODEL)



WHAT IT SHOWS

Left: the six raw scores; "I" is very negative (red). Right: the shares. Negative scores become tiny shares, and the 0.7 gap between tea (7.7) and coffee (7.0) becomes a 2 : 1 split.

WHY IT MATTERS

Shares are what the loss scores (Ch 10) and what the picker samples from (Ch 14). Both need real percentages.

- **What it is:** e to the power of each score, divided by the total of those
- **Who decides:** a fixed rule in the design, no weights
- **Why e:** e raised to anything is positive, and each +1 in score multiplies the share by 2.72, so gaps between scores become ratios between shares

Word	Score	Score ÷ total (2.85)	Softmax share
I	−14.9	-523%	0.0%
love	0.2	8%	0.0%
tea	7.7	269%	66.6%
coffee	7.0	245%	33.1%
hate	1.1	38%	0.1%
rain	1.8	63%	0.2%

Dividing by the total gives "I" a negative share and tea more than 200%: not percentages at all. Softmax always gives 0–100%, adding to 100%.

Softmax — Python

```
def softmax(z):
    e = np.exp(z - z.max())      # subtracting the max avoids overflow; same result
    return e / e.sum()
```

IN A REAL MODEL (8B)

The final row has 4,096 numbers and is scored against 128,256 rows, giving 128,256 percentages per token.

SCALE IT UP: WHAT GROWS, AND WHAT IT COSTS

Knob	Toy → 8B model	Compute	Memory	Good	Bad
Vocabulary scored	6 → 128,256 per token	≈ 525 M multiply-adds per token	Small for one user; in training, scores for millions of tokens at once need careful chunking	The model can say any of 128K tokens	Big output step

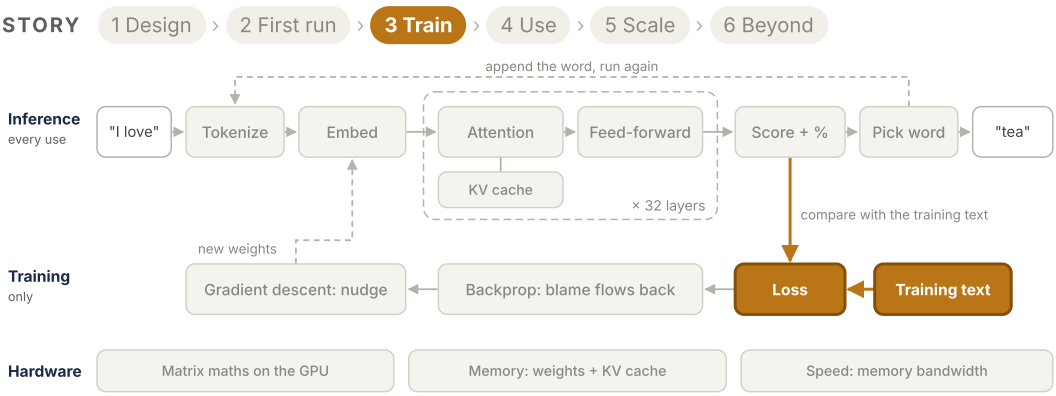
WATCH OUT

A share is a **preference**, not a probability of being correct. A model can put 95% on a wrong answer (Ch 14).

STAGE 3

Train: turning noise into knowledge

Training repeats one loop: run the model, measure the mistake, find who is to blame, nudge every weight. Our run: 500 loops.



10 Loss: one number for "how wrong"

To improve, the model needs a single number to push down. Everything in training serves that number.

How do we score percentages against the training text?

COMES IN
Percentages (Ch 9) + the training text (Ch 1)

→ WHO DOES THE WORK
Loss: cross-entropy

→ HANDS ON
One number → backprop (Ch 11)

IN ONE LINE Loss measures surprise; training pushes it down.

The loss used for language models is **cross-entropy**: **how surprised was the model by the word that actually came next?** It's −ln(share given to the actual next word). The model is scored on all 8 predictions from Chapter 1 and the results averaged.

Share given to the actual next word	Loss	Meaning
100%	0	No surprise
67%	0.40	Mild surprise
18.0% (newborn, "I love" → tea)	**1.72**	**Very surprised**
1%	4.61	Confidently wrong

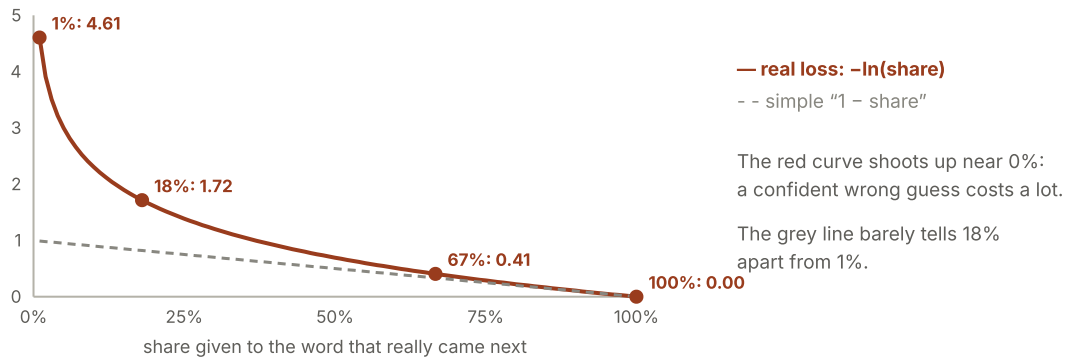
Average over all 8 predictions	Loss
Newborn model	1.864
Guessing evenly among 6 words	1.792
Best possible for this text (the 75/25 and 67/33 splits)	**0.520**

The best possible loss is **not 0**: after "I love" the text itself is uncertain (tea or coffee), so even a perfect model is a little surprised.

In plain words: why minus log, not "1 − share"

The loss turns "how much did you give the right word?" into a penalty. The shape of that penalty decides what the model learns.

PENALTY FOR EACH SHARE GIVEN TO THE REAL NEXT WORD



WHAT IT SHOWS

The penalty for each share: red is the real loss, grey is a simpler idea. Dots mark 1%, the newborn's tea share, 67% and 100%.

WHY IT MATTERS

The steep red curve pushes hardest on the worst mistakes, and its best answer for "I love" is exactly the 67 / 33 split in the text.

- **What it is:** −ln(share the model gave the word that actually came next), averaged over all 8 predictions
- **Who decides:** us: we pick the loss. It has no weights
- **What it rewards:** honest odds. Our text says tea 2 times and coffee once after "I love", and the lowest possible loss is at 67% / 33%

Share given to the right word	1 − share	−ln(share), the real loss
100%	0	0
67%	0.33	0.40
18.0% (newborn tea)	**0.82**	**1.72**
1%	0.99	4.61
0%	1	infinite

Why not just "was the top word right?" A model that always says tea 100% would be "right" 2 times in 3, but on "I love coffee" it gave coffee 0%, so its loss is infinite. Only the 67 / 33 split scores best.

Cross-entropy loss — Python

```
p = softmax(logits)          # the model's shares
loss = -np.log(p[target])    # target = id of the actual next word
```

IN A REAL MODEL (8B)

A model guessing evenly among 128,256 tokens would score ln(128,256) ≈ 11.8. Training on ≈ 15 trillion tokens pulls the average loss far below that.

SCALE IT UP: WHAT GROWS, AND WHAT IT COSTS

Knob	Toy → 8B model	Compute	Memory	Good	Bad
Predictions scored	8 → trillions	Each token in the training text is one prediction	Text is streamed	More predictions, more learned	Training cost ∝ tokens

INSIGHT

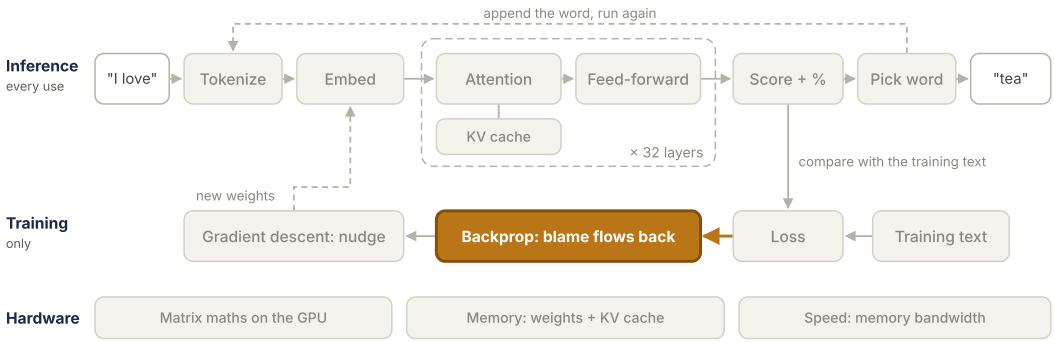
Confident mistakes cost most. 1% on the right word costs 4.61, while 67% costs only 0.40. Training fixes confident errors first.

MYTH

Low loss means the model is right.

REALITY

Low loss means it predicts its training text well. That is related to, but not the same as, giving correct or useful answers on new questions.



11 Backprop: finding each weight's share of the blame

80 weights contributed to the mistake. Training needs to know, for each one, which way to move.

How do we know whether to increase or decrease one particular weight?

COMES IN	WHO DOES THE WORK	HANDS ON
The loss (Ch 10) →	Backprop: the chain rule, run backwards	→ A gradient for every weight → gradient descent (Ch 12)

IN ONE LINE Backprop gives every weight a direction to move.

The **gradient** of a weight = how much the loss changes if that weight rises a tiny bit. **Backprop** computes every gradient by working backwards from the loss, multiplying effects along the way (the **chain rule**).

Example: detector 1's write into slot 3 (currently 0.6), for the single prediction "I love" → tea.

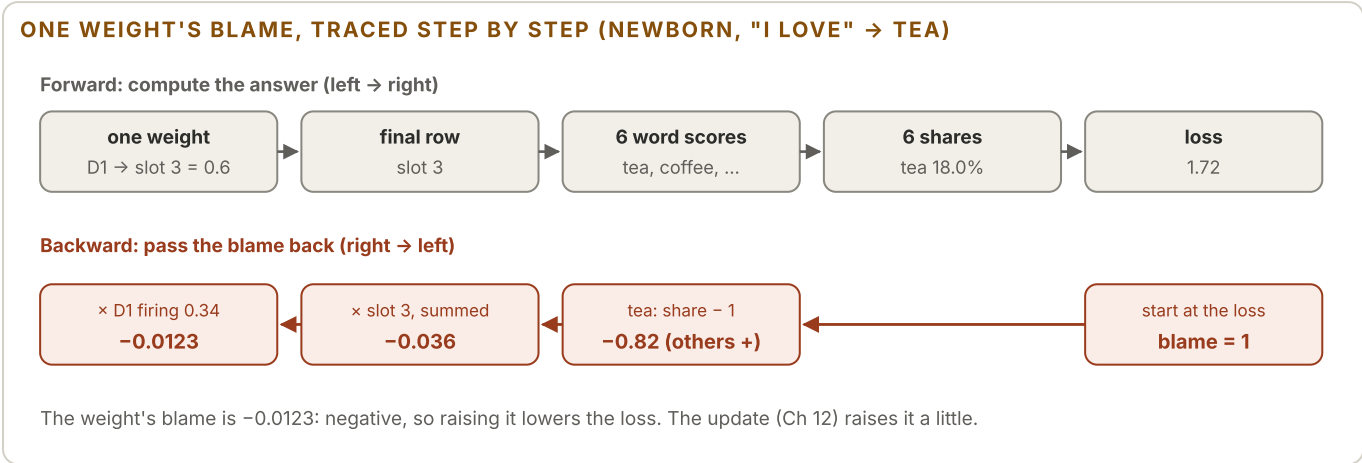
#	What happens	Our numbers
1	**Blame on each word's score** = its share − (1 if it's the actual next word, else 0). This simple rule comes out of softmax + cross-entropy	tea 18.0% − 1 = **−0.82**; every other word = its own share, e.g. love +0.20
2	**Blame on slot 3 of the final row**: every score used slot 3, so add up (word's blame × that word's slot 3)	= **−0.036**
3	**Blame on the weight**: its effect on slot 3 was scaled by detector 1's firing	0.339 × −0.036 = **−0.0123**

Check: we also nudged the weight by 0.00001, re-ran the model and measured the change in loss: **−0.0123**. Same number.

A negative gradient means "raise this weight and the loss falls".

In plain words: blame flows backwards

Training must know, for each of the 80 weights, "if I raise you a little, does the loss go up or down?". Backprop answers that for all of them in one backward sweep.



WHAT IT SHOWS

Top: the normal forward pass. Bottom: the blame, starting at the loss and multiplied by each step's effect on its way back to one weight.

WHY IT MATTERS

Every weight gets its own blame number in the same sweep. The sign says which way to move it; the size says how much it mattered.

- **What it is:** the chain rule: multiply the effects along the path back from the loss
- **Who decides:** nobody chooses the blame. It follows from the model and the loss
- **When it runs:** training only. Using a model (Ch 13 onwards) never runs it

Way to find every weight's blame	Our 80 weights	The 8B model
Nudge each weight up and down, re-run each time	160 runs per step	≈ 16 billion runs per step: impossible
Backprop: one forward + one backward sweep	**≈ 3 runs' worth**	**≈ 3 runs' worth**

The nudge method is still useful as a check: Chapter 11 used it to confirm one backprop number.

Check a gradient by nudging (how we verified it) — Python

```
def numeric_grad(loss_fn, W, i, j, eps=1e-5):
    old = W[i, j]
    W[i, j] = old + eps; up = loss_fn()
    W[i, j] = old - eps; down = loss_fn()
    W[i, j] = old
    return (up - down) / (2 * eps)
# PyTorch does backprop for you:  loss.backward(); W.grad
```

IN A REAL MODEL (8B)

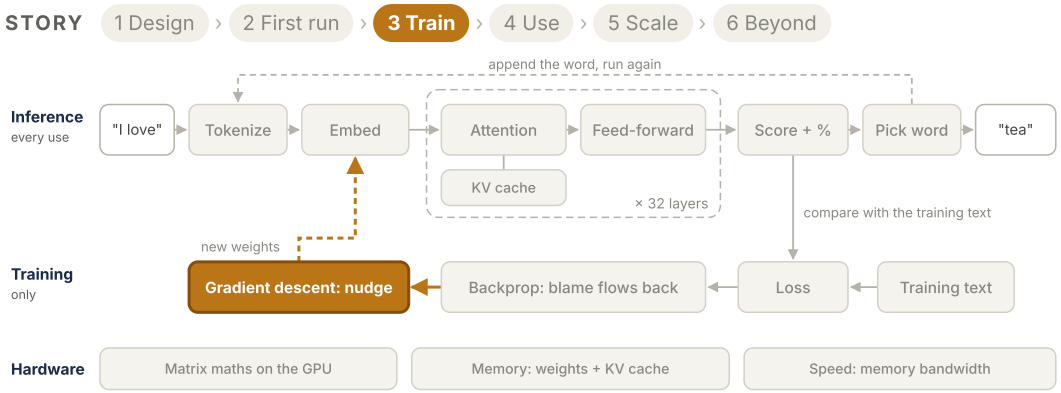
One backward pass computes gradients for all ≈ 8 billion weights at about twice the cost of the forward pass, so a training step costs ≈ 3× a forward pass (the source of the 6 × parameters × tokens rule).

SCALE IT UP: WHAT GROWS, AND WHAT IT COSTS

Knob	Toy → 8B model	Compute	Memory	Good	Bad
Backward pass	—	≈ 2× forward	Intermediate values from the forward pass must be kept: grows with batch × context × layers	Exact gradients for every weight in one sweep	Memory-heavy; a common fix recomputes some values instead of storing them

INSIGHT

Look at step 1: blame is negative only for the actual next word (push it up) and positive for every other word (push down, in proportion to the share it took).



12 Gradient descent: the training run

Now we repeat the loop 500 times and watch random numbers become a model.

How do the weights change, and what do we get at the end?

COMES IN	WHO DOES THE WORK	HANDS ON
Gradients for all 80 weights (Ch 11) →	Gradient descent: weight − learning rate × gradient	→ 80 trained weights, saved and frozen → use (Ch 13)

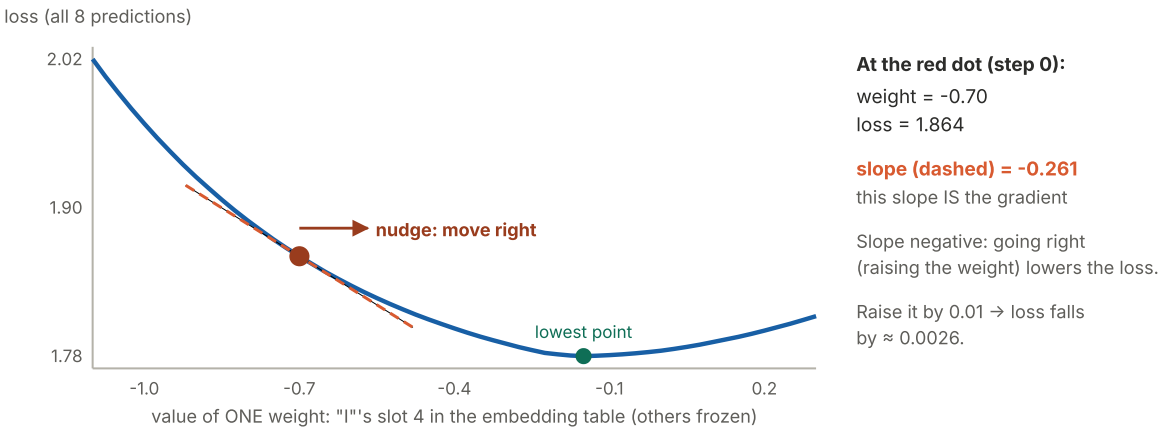
IN ONE LINE Training is measure → blame → nudge, repeated.

Update rule: new weight = old weight − **learning rate** × gradient. Ours: learning rate 0.1, all 8 predictions per step, 500 steps.

In plain words: gradient, descent, nudge

- **Gradient** = the slope of the loss for one weight: "if I raise this weight a tiny bit, how much does the loss change, and which way?" (Ch 11 computes it.)
- **Descent** = move the weight the *opposite* way to the slope, so the loss goes down.
- **Nudge** = the actual change: −learning rate × gradient. Small on purpose.

1 · THE GRADIENT IS A SLOPE (ONE REAL WEIGHT)



At the red dot (step 0):
weight = -0.70
loss = 1.864

slope (dashed) = -0.261
this slope IS the gradient

Slope negative: going right (raising the weight) lowers the loss.

Raise it by 0.01 → loss falls by ≈ 0.0026.

WHAT IT SHOWS
The real loss (all 8 predictions) as we slide just one weight, "I"'s slot 4 in the embedding table, and keep the other 79 at their random start. The red dot is where it starts; the dashed line is the slope there.

WHY IT MATTERS
The slope (-0.261) is negative, so moving right lowers the loss. Training never sees this whole curve; it only knows the slope at the current point, so it takes a small step and checks again.

2 · THE ARITHMETIC OF ONE NUDGE



Result for this one weight: loss 1.8642 → 1.8576 (drop 0.0066; predicted ≈ learning rate × gradient² = 0.0068)

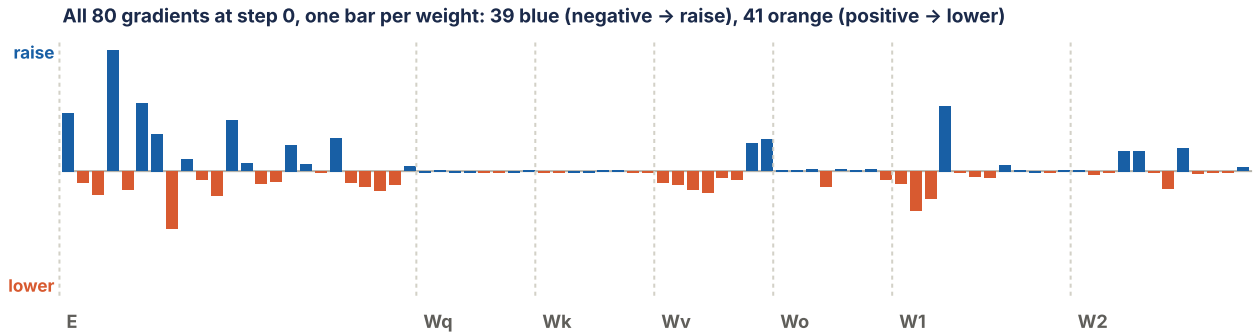
WHAT IT SHOWS

The update rule as five boxes: slope × learning rate = step; subtract it (descent) to get the new weight. The loss drops by almost exactly what the slope predicts.

WHY IT MATTERS

That is all "learning" is at the level of one number: a subtraction. The minus sign is the "descent".

3 · EVERY STEP NUDGES ALL 80 WEIGHTS AT ONCE



Tall bar = this weight strongly affects the loss, so it gets a big nudge. Tiny bar = it barely matters right now, so it barely moves.

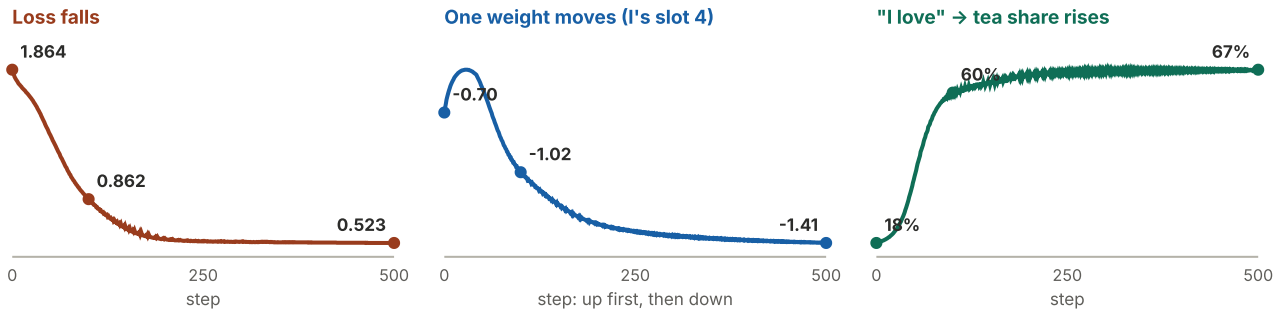
WHAT IT SHOWS

The gradient of every weight at step 0, grouped by grid. Blue bars point up (raise the weight), orange bars point down (lower it).

WHY IT MATTERS

One backprop gives all 80 slopes; one step applies all 80 nudges together. The 8B model does the same for 8 billion weights per step.

4 · THE IMPACT: 500 SMALL NUDGES ADD UP



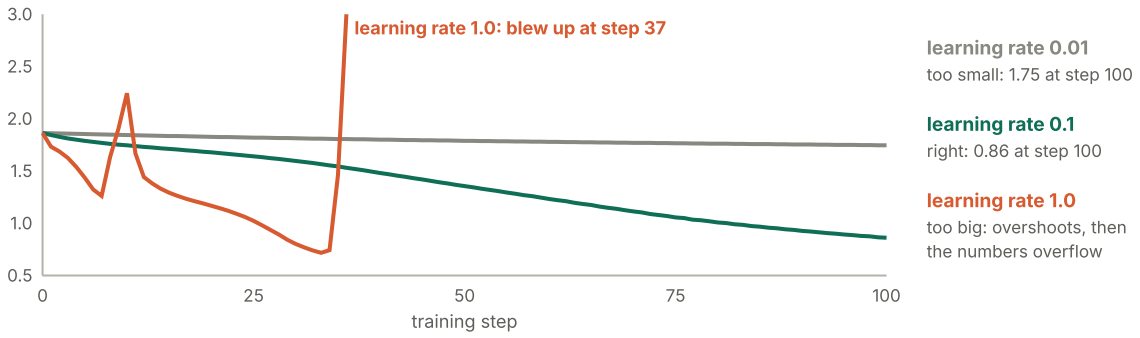
WHAT IT SHOWS

Over 500 steps: the loss falls, our example weight moves, and the tea share after "I love" climbs to the 2 : 1 target. The weight first rises (as figure 1 predicted), then turns and falls.

WHY IT MATTERS

The slope is *local*: as the other 79 weights move, each weight's best direction changes. That is why training takes many small steps instead of one big jump.

5 · HOW BIG A NUDGE? THE LEARNING RATE



WHAT IT SHOWS

The same training with three learning rates, first 100 steps. Too small barely moves; 0.1 falls steadily; 1.0 jumps too far, overshoots the low point, and the numbers eventually overflow.

WHY IT MATTERS

The learning rate is a setting we choose, not learned. Large runs adjust it on a schedule and watch for exactly this blow-up.

Step	Loss	"I love" → tea	→ coffee	→ rain	"I" → love / hate	tea–coffee similarity
0	1.864	18%	18%	16.5%	14% / 15%	0.00
10	1.745	19%	18%	16.0%	18% / 16%	0.20
25	1.641	22%	19%	14.4%	21% / 14%	0.40
50	1.356	37%	22%	8.4%	32% / 9%	0.61
100	0.862	60%	26%	8.3%	70% / 6%	0.81
150	0.630	63%	30%	4.4%	75% / 14%	0.88
200	0.559	65%	31%	1.1%	80% / 18%	0.90
300	0.534	66%	32%	0.4%	81% / 19%	0.91
500	**0.523**	**67%**	**33%**	**0.2%**	**77% / 23%**	**0.92**
Target	*0.520*	*67%*	*33%*	*0%*	*75% / 25%*	—

- The loss falls from 1.864 to 0.523, almost the best possible (0.520).
- The model learned the **split**, not just "tea": 67% / 33%.
- **Nobody told it tea and coffee belong together.** Their cosine similarity went from 0.00 to 0.92, because they appear in the same place in the text. That's a useful statistical pattern, not proof of understanding.

Learning rate: too small, right, too large

Learning rate	Loss at step 0	Step 10	Step 50	Step 100
0.01	1.864	1.844	1.789	1.746 (barely moving)
0.1	**1.864**	**1.745**	**1.356**	**0.862**
1.0	1.864	2.245 (worse)	blew up: the numbers overflowed	

Where "learned" happens: watch the weights move

Every grid called a weight in Chapters 4–9 (embedding table, Wq, Wk, Wv, Wo, W1, W2) starts random. **This chapter's loop is the only place they change.** Each step nudges every cell a little, in the direction its

gradient points.

1 · WQ BEFORE AND AFTER TRAINING

Wq at step 0 (random)

−1.4	−0.9
−0.1	−0.2
0.1	0.1
1.1	−0.6

→

Wq at step 500 (trained)

−1.39	−0.84
−0.08	−0.16
0.09	0.06
1.11	−0.57

Same shape, same positions: training only changes the numbers in the cells. Every cell here moved by 0.06 or less.

WHAT IT SHOWS

The same 4 × 2 grid from Chapter 6, at step 0 and at step 500. Every cell moved, but only slightly (0.06 or less), because Wq had little effect on the loss for this tiny text.

WHY IT MATTERS

"Learned" means exactly this: numbers in fixed cells, adjusted by the update rule. Each weight moves only as much as its gradient says: a small gradient means a small change.

2 · FEED-FORWARD WRITE ROWS BEFORE AND AFTER

W2 at step 0 (random)

D1	−0.8	0.1	0.6	−0.1
D2	−0.4	0.4	0.1	0.4
D3	−0.2	−0.7	−0.1	−0.2

→

W2 at step 500 (trained)

D1	−1.49	−0.55	1.20	0.17
D2	−0.34	0.41	0.07	0.45
D3	−0.59	−0.98	−0.21	0.22

WHAT IT SHOWS

The write rows (W2) from Chapter 7. D1's write into slot 3 grew from 0.6 to 1.20: the weight whose blame Chapter 11 computed for one prediction.

WHY IT MATTERS

Chapter 11 found its blame for "I love → tea" was negative, meaning "raise this weight". Over all 8 predictions, and as the other weights moved, it rose to 1.20.

Step	Loss	D1's write to slot 3	tea's embedding row	coffee's embedding row
0	1.864	0.60	`[0.20, 0.10, 0.00, 0.30]`	`[−0.40, −0.10, −0.20, 0.30]`
10	1.745	0.60	`[0.21, 0.10, −0.05, 0.40]`	`[−0.39, −0.12, −0.22, 0.35]`
50	1.356	0.63	`[0.13, 0.16, −0.25, 0.86]`	`[−0.41, −0.18, −0.26, 0.48]`
100	0.862	0.81	`[−0.08, −0.09, −0.24, 1.07]`	`[−0.43, −0.23, −0.25, 0.53]`
200	0.559	1.06	`[−0.28, −0.25, −0.14, 1.09]`	`[−0.54, −0.26, −0.30, 0.67]`
500	0.523	1.20	`[−0.36, −0.28, −0.15, 1.15]`	`[−0.60, −0.28, −0.35, 0.76]`

Watch slot 4: tea's and coffee's rows both grow there, which is how their similarity rises from 0.00 to 0.92.

The whole training loop — Python

```
for step in range(500):
    grads = gradients(weights, text)       # backprop (Ch 11)
    for name in weights:
        weights[name] -= 0.1 * grads[name] # nudge every weight downhill
# then save: the weights are frozen for use
```

IN A REAL MODEL (8B)

The 8B model used AdamW (a refined gradient descent), a peak learning rate of 3×10^{-4} for 8B, ≈ 15 trillion tokens, and 1.46 million data-centre GPU-hours (published model card and technical report).

SCALE IT UP: WHAT GROWS, AND WHAT IT COSTS

Knob	Toy → 8B model	Compute	Memory	Good	Bad
Training memory per weight	Using: 2 bytes → training: ≈ 16 bytes	Each step: forward + backward + update	Mixed-precision Adam: 2 (weight) + 2 (gradient) + 12 (fp32 copy + 2 running averages) = 16 bytes × 8 B ≈ 128 GB, before intermediate values	Stable, fast learning	Training 8B needs several 80 GB GPUs; using it needs one
Batch size	8 predictions → millions of tokens per step	Big matrices keep the GPU busy	More intermediate values held	Steadier gradients, higher throughput	Memory

INSIGHT

No step in the loop is labelled “understand”. Training is this rule repeated: measure, find blame, nudge. Useful structure, like tea and coffee getting similar rows, comes out of it. Whether large models trained this way “understand” is debated; the mechanism itself is not.

WATCH OUT

Learning rate is a real risk. Too small wastes compute; too large makes training blow up, and large runs watch for exactly this.

MYTH

Training stores the text in the model.

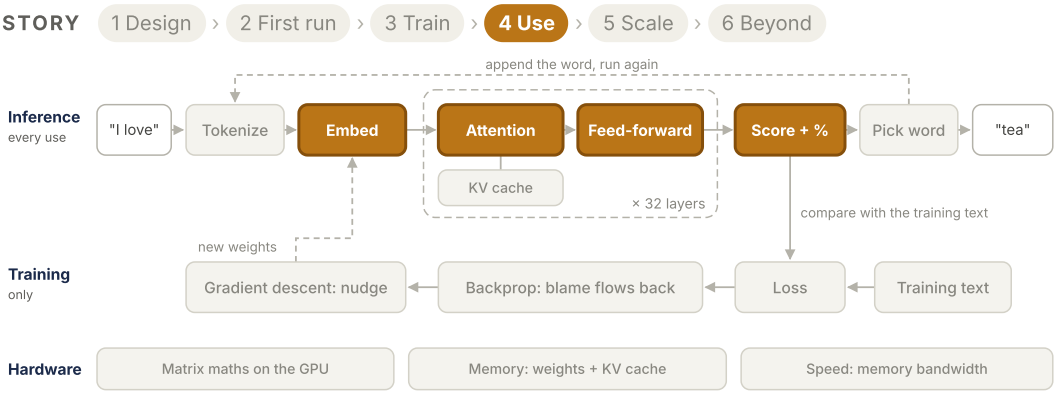
REALITY

Training adjusts 80 numbers. Our model holds no sentences, only numbers that reproduce the 67/33 pattern.

STAGE 4

Use: the trained model at work

Same steps as Stage 2, same arithmetic. Only the 80 numbers are different. This is inference: weights are read, never changed.



13 The trained model answers "I love"

Run exactly the same pipeline as Chapter 4–9, with trained weights.

What changed inside, step by step?

COMES IN
Trained weights (Ch 12) + "I love"

→

WHO DOES THE WORK
Every inference part, now trained

→

HANDS ON
Percentages → picker (Ch 14)

IN ONE LINE Same code, different numbers: a trained model is a file of weights.

Step	Newborn	Trained	Ch
Embedding row for "love"	`[0.5, 0.2, −0.3, 0.3]`	`[0.91, 1.44, −0.77, 0.70]`	4
Attention shares (I / love)	60% / 40%	60% / 40%	6
Row after attention	`[0.38, 0.17, −0.41, 0.46]`	`[2.13, 2.75, −4.56, 3.92]`	6
Detector scores	`[0.34, −0.06, 0.15]`	`[2.29, −0.77, 3.12]`	7
Final row	`[0.08, 0.10, −0.22, 0.40]`	`[−3.13, −1.58, −2.46, 4.98]`	8
Scores: tea / coffee / rain	0.14 / 0.12 / 0.06	7.69 / 6.99 / 1.80	9
Shares: tea / coffee / rain	**18% / 18% / 16%**	**67% / 33% / 0.2%**	**9**

What all of it achieves: the end result

FROM THE FINAL ROW TO THE NEXT WORD

Embedding table (trained), 6 × 4

I	1.39	1.37	0.58	−1.41
love	0.91	1.44	−0.77	0.70
tea	−0.36	−0.28	−0.15	1.15
coffee	−0.60	−0.28	−0.35	0.76
hate	1.40	−0.23	−1.17	0.45
rain	−0.76	−1.35	1.21	0.05

Final "love" row (1 × 4)

love	−3.14	−1.58	−2.46	4.98

The row that came out of attention + feed-forward (Ch 6–8), with trained weights.

It is compared with every word's row: 6 dot products, one per word.

Vocabulary scores: final row · each word row (1 × 6)

I	love	tea	coffee	hate	rain
−14.9	0.2	7.7	7.0	1.1	1.8

Shares after softmax (1 × 6)

I	love	tea	coffee	hate	rain
0.0%	0.0%	66.6%	33.1%	0.1%	0.2%

→ pick: "tea" (greedy),
or coffee about 1 time in 3

WHAT IT SHOWS

The final "love" row is scored against every word's row in the trained embedding table. tea and coffee score high (7.7, 7.0) because training made their rows point the same way as this row; "I" scores very low. Softmax turns the scores into shares.

WHY IT MATTERS

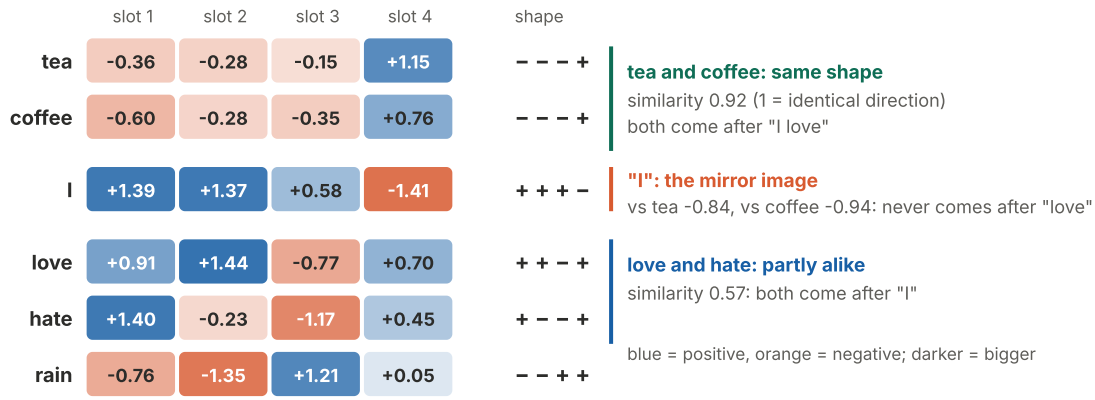
This is the single number the whole pipeline exists to produce: the shares for the next word. The result, tea 66.6% and coffee 33.1%, is the 2 : 1 split in the training text.

Step	What it contributed to "tea 66.6%"
Embedding table	Gave each word a row; training placed tea and coffee close together and far from "I"
Attention	Mixed information from "I" into the "love" row (60% / 40%)
Feed-forward	Added a learned change that pushes the row toward where tea and coffee sit
Vocabulary scores + softmax	Measured how close the final row is to each word, then turned that into shares
Training (Ch 12)	Set every weight above so the shares match the text: 67% / 33%

What the patterns are, and how they pick the word

"Meaning lives in patterns across the slots" has a concrete meaning: the **shape** of a row, which slots are positive or negative and how big each is relative to the others. No single slot means anything.

1 · THE SHAPES TRAINING PRODUCED (TRAINED EMBEDDING TABLE)



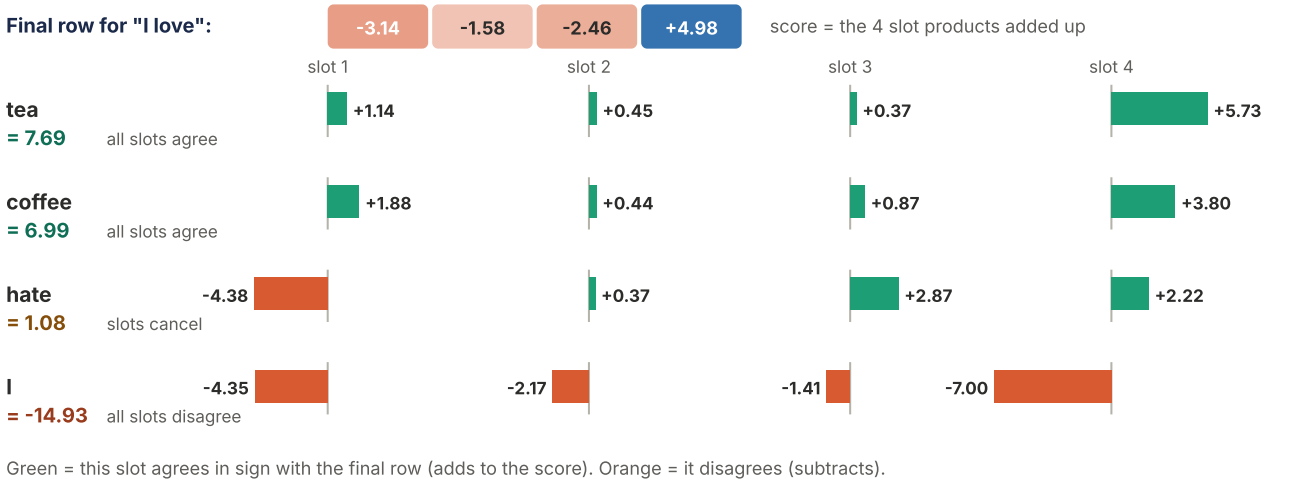
WHAT IT SHOWS

Each word's 4 numbers as coloured cells, with its sign shape. tea and coffee share the shape − − − +. "I" has the opposite shape. love and hate are partly alike.

WHY IT MATTERS

Nobody designed these shapes. Words used in the same place in the text ended up with the same shape, and that shape is what the scoring step compares.

2 · HOW A SHAPE BECOMES A SCORE: SLOT BY SLOT

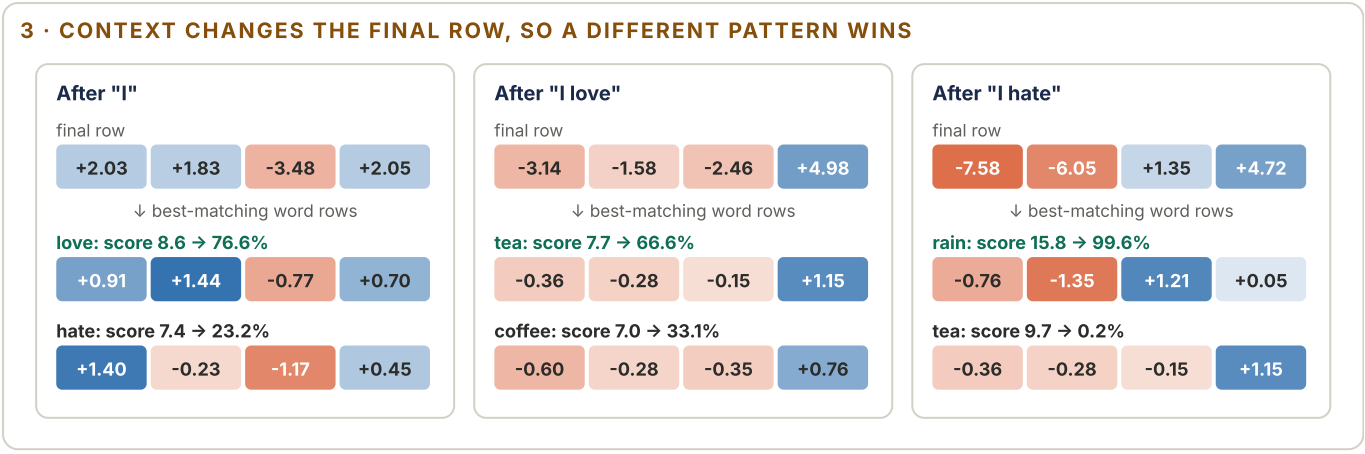


WHAT IT SHOWS

The final row for "I love" has the shape − − − +. Multiplying it slot by slot with each word's row: tea and coffee agree in every slot, "hate" agrees in some and cancels out, "I" disagrees in every slot.

WHY IT MATTERS

Slot 4 alone would rank love (3.47) above hate (2.22) and close to coffee (3.80). Only all four slots together rule love out: the meaning is the whole pattern.



WHAT IT SHOWS

Three prompts, three final rows, three different winners: after "I" the shape matches love and hate; after "I love" it matches tea and coffee; after "I hate" it matches rain.

WHY IT MATTERS

Attention and feed-forward (Ch 6–8) exist to build a final row with the right shape for the context. Scoring then just finds the word rows with the same shape.

- **Where the shapes come from:** the blame rule (Ch 11). Each time tea is the right next word, training pulls tea's row toward that final row and pushes the other rows away. Coffee is pulled toward the same final row because it appears in the same place, so the two end up alike. "I", never used there, is pushed the other way.
- **In real models:** the same mechanism with 4,096 slots. Words used in similar ways point in similar directions: drinks near drinks, verbs near verbs, a shared direction for plural vs singular.

WATCH OUT

Real slots almost never map to one clean idea. Each direction mixes many patterns, and each idea is spread over many slots (Ch 7). Patterns are learned from how words are *used* in the text, which is useful for prediction but not proof of understanding.

A useful pattern appeared in the embedding table

Pair	Newborn cosine	Trained cosine
tea – coffee	0.00	**0.92**
tea – rain	0.48	0.22
love – hate	0.24	0.57

Tea and coffee became neighbours because both follow "I love". Slots still have no names; the pattern is in their *relative* positions. This is a learned statistical representation: useful for prediction, but cosine similarity alone isn't evidence of understanding.

The switch, measured

Trained detector scores [2.29, −0.77, 3.12]: detector 2 is negative and is switched off. With the switch, the write is [−5.27, −4.33, 2.10, 1.05]; with the switch removed, it would be [−5.01, −4.65, 2.04, 0.71]. Different input words switch on different detectors; that input-dependence is what makes stacked layers useful.

Use the trained model — Python

```
X  = E[[0, 1]]                          # "I love"
H  = X + attention(X, Wq, Wk, Wv, Wo)
H2 = H + feed_forward(H, W1, W2)
p  = softmax(H2[-1] @ E.T)              # tea 0.66, coffee 0.33
```

INSIGHT

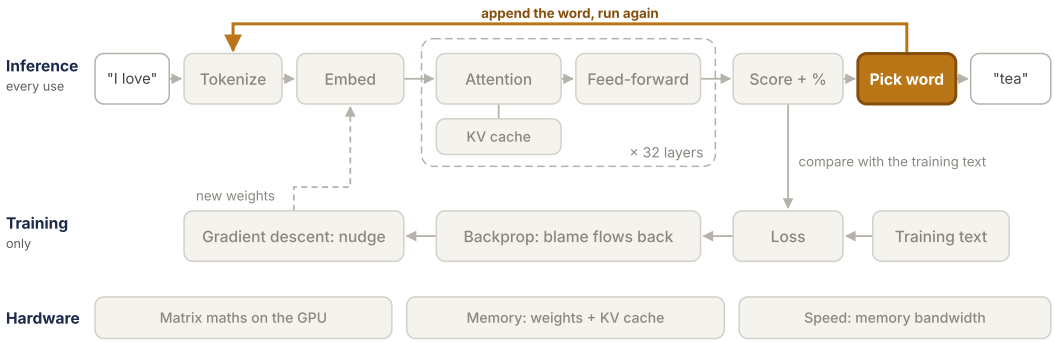
Nothing new was added between Stage 2 and Stage 4. Same code, same arithmetic, different 80 numbers. A trained model is just a file of numbers.

MYTH

The model looks up the answer.

REALITY

It computes scores for every word and ranks them. "Knowing" means the right words score highest.



14 Picking the word, then going again

The model outputs percentages. Something else chooses a word, appends it, and runs the model again.

How does a list of percentages become text, and why can answers differ?

COMES IN
Percentages (Ch 13)

→

WHO DOES THE WORK
Picker: sampling with temperature / top-p

→

HANDS ON
One word, appended → next step (Ch 15)

IN ONE LINE The picker adds randomness, and a share isn't a measure of reliability.

Temperature: divide the scores before softmax

Temperature	tea	coffee	rain	Behaviour
0.5	80%	20%	0.0%	Sharper, more repetitive
1	**67%**	**33%**	**0.2%**	**As trained**
2	55%	39%	2.9%	Flatter, riskier

Method	Rule	Our result
Greedy	Always the top word	tea, every time
Sampling	Weighted random pick	tea about 2 times in 3, coffee about 1 in 3
Top-p 0.9	Keep the top words until they cover 90%, sample among them	tea + coffee ≈ 100% → rain is never picked

The loop, and a warning

We append "tea" and run again on "I love tea". Our training text **never continued after "tea"**, so nothing was learned there. The model still answers, and confidently: **rain 95%**.

WATCH OUT

A model always produces percentages, even where it learned nothing. Here a 95% share comes from a situation the training text never covered. That's one mechanism behind confident errors, not a full explanation of hallucination in large models. Keep three things apart: a **token share** (the model's preference), **confidence in a whole answer**, and **measured reliability** (how often answers like this are right). They're related, but not interchangeable. Real models also learn an end-of-text token, so they know when to stop.

Sampling, ours and with the transformers library — Python

```
rng = np.random.default_rng()
next_id = rng.choice(len(p), p=softmax(logits / 0.7))   # temperature 0.7

from transformers import AutoTokenizer, AutoModelForCausalLM
name = "path/to/an-8b-model"                            # any open causal LM
tok = AutoTokenizer.from_pretrained(name)
model = AutoModelForCausalLM.from_pretrained(name, torch_dtype="auto", device_map="auto")
ids = tok("I love", return_tensors="pt").to(model.device)
out = model.generate(**ids, max_new_tokens=20, do_sample=True, temperature=0.7, top_p=0.9)
print(tok.decode(out[0]))
```

SCALE IT UP: WHAT GROWS, AND WHAT IT COSTS

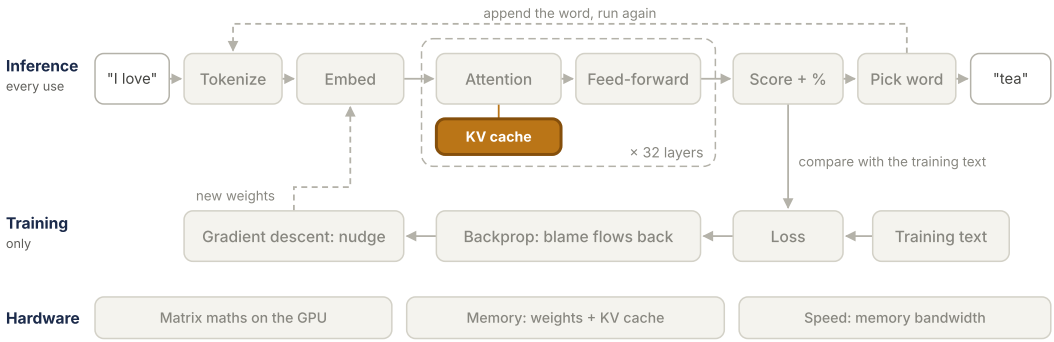
Knob	Toy → 8B model	Compute	Memory	Good	Bad
Output length	1 word → hundreds or thousands	Each output token is one pass of the new token through all layers, reusing cached K and V; tokens come one after another	Each adds 128 KiB of cache	Longer, fuller answers	Output tokens are slow; APIs typically price them higher than input
Hidden reasoning tokens	—	Reasoning models may write thousands of tokens before answering	Same cache growth	Better on hard problems	You pay for tokens you may never see

MYTH

Temperature 0 makes the model accurate.

REALITY

It makes it consistent. If its top choice is wrong, it's wrong every time.



15 KV cache: keep the keys and values

To handle "I love tea", attention needs keys and values for all three words. Two of them were already computed last step.

What is saved between steps, and does it change the answer?

COMES IN	WHO DOES THE WORK	HANDS ON
New token "tea" (Ch 14)	KV cache: memory, no weights	Cache [I, love, tea] → next step

IN ONE LINE The cache saves computing, not reading or memory.

Because tokens only look backwards, earlier tokens' K and V rows **never change** when new tokens arrive. So they're computed once and stored.

TRAINED MODEL, STEP 3: NEW TOKEN "TEA"

```
Compute fresh for "tea" only:  Q = [1.79, −0.31]  K = [−0.06, −0.47]  V = [0.15, 0.61]

token   K               V               status
I       [0.21, 1.64]     [−1.11, −2.82]   reused
love    [0.48, 1.90]     [−1.42, −2.52]   reused
tea     [−0.06, −0.47]   [0.15, 0.61]     added now

Shares for tea's query:  I 29%   love 38%   tea 33%
```

Check: recomputing everything from scratch gives the same shares to every decimal (yes). The cache changes the cost, never the answer.

	Kept?	Why
K and V of earlier tokens	✔	Never change; every future query needs them
Q of earlier tokens	✘	Only used once, when that token was newest
Weights	n/a	Read again in full every step (Ch 18)

Follow the arrays: the cache grows one row per token

The cache is just the K and V grids from Chapter 6, kept between steps. **Rows are tokens**; each new token appends one row to each grid.

1 · APPEND ONE ROW TO K AND TO V

K cache after step 2

I	0.21	1.64
love	0.48	1.90

V cache after step 2

I	−1.11	−2.82
love	−1.42	−2.52

+

New token "tea": compute only this

Q tea	1.79	−0.31
K tea	−0.06	−0.47
V tea	0.15	0.61

→

K cache after step 3

I	0.21	1.64
love	0.48	1.90
tea	−0.06	−0.47

V cache after step 3

I	−1.11	−2.82
love	−1.42	−2.52
tea	0.15	0.61

Grey rows: reused from earlier steps, not recomputed. Coloured row: added now. Q of "tea" is used once and not stored.

WHAT IT SHOWS

Before "tea" arrives, the cache holds the K and V rows of "I" and "love". For "tea", only its own Q, K, V are computed, and its K and V rows are appended. Grey rows are reused unchanged.

WHY IT MATTERS

Earlier rows never change (tokens only look backwards), so computing them again would be wasted work. The price is memory: one more row per token, in every layer and K/V head.

2 · ONLY THE NEWEST ROW OF THE SCORE GRID IS NEW WORK

Shares grid for "I love tea" (3 × 3)

	from I	from love	from tea
I asks	100%	hidden	hidden
love asks	60%	40%	hidden
tea asks	29%	38%	33%

→

Rows "I asks" and "love asks" were computed in steps 1 and 2 (grey).

tea's blend = shares · V

tea	−0.81	−1.57

Step 3 computes only the "tea asks" row: 1 query against all 3 cached keys.

WHAT IT SHOWS

The full 3 × 3 shares grid for "I love tea". The first two rows were computed in earlier steps (grey); step 3 computes only the "tea asks" row, then blends the cached V rows.

WHY IT MATTERS

The cache turns each step from recomputing the whole grid into computing one row. That one row still reads every cached key, so long contexts remain slower per word.

Grid	Ours, after 3 tokens	8B model, after n tokens	Grows by
K cache	3 × 2 (1 layer, 1 head)	n × 128, for each of 8 K/V heads × 32 layers	1 row per token
V cache	3 × 2	n × 128, × 8 × 32	1 row per token
Score row computed per step	1 × 3	1 × n per query head, × 32 heads × 32 layers	1 longer each step

INSIGHT

The cache's size is just the row count: **tokens × (2 grids × layers × K/V heads × 128 numbers × 2 bytes)** = tokens × 128 KiB for the 8B model. Each step adds one row, and each new query reads every row, so a longer conversation means more memory and more reading.

KV cache with transformers (generate uses it automatically) Python

```
out = model(**ids, use_cache=True)
past = out.past_key_values                          # cached K and V, every layer
next_id = out.logits[:, -1].argmax(-1, keepdim=True)
out = model(input_ids=next_id, past_key_values=past, use_cache=True)  # only 1 new token
```

IN A REAL MODEL (8B)

Per token: 2 (K and V) × 32 layers × 8 K/V heads × 128 numbers × 2 bytes = 131,072 bytes = **128 KiB**. 8,192 tokens = 1 GiB; the full 131,072-token context = **16 GiB**, as much as the weights.

SCALE IT UP: WHAT GROWS, AND WHAT IT COSTS

Knob	Toy → 8B model	Compute	Memory	Good	Bad
Cached tokens × users	3 rows → 128 KiB × tokens × users	Saves recomputing K and V for every earlier token	1 user × 131K tokens = 16 GiB; 10 users × 32K = 40 GiB	Each step computes only the new token	The cache, not the weights, limits how many users fit

INSIGHT

The cache removes repeated **computing**, not repeated **reading**. The new query still compares with every stored key.

MYTH

The KV cache stores the conversation text.

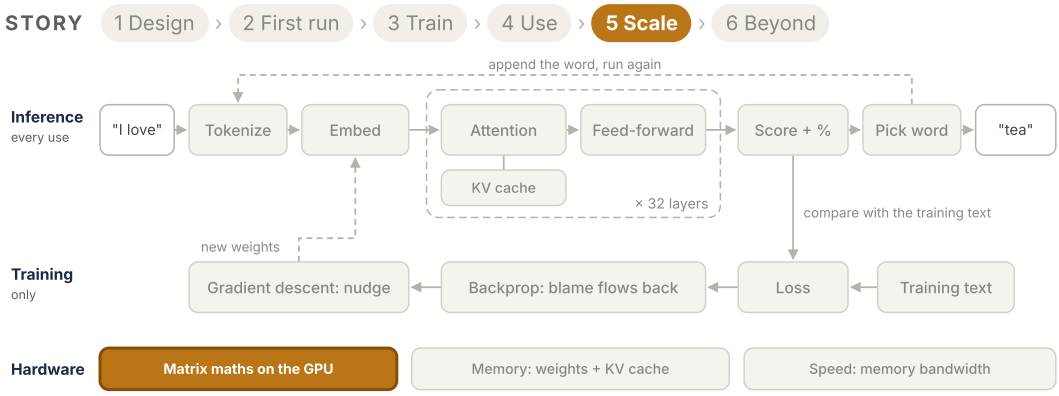
REALITY

It stores K and V numbers per layer, per head, per token. The text itself is just a short list of ids.

STAGE 5

Scale: same maths, 100 million times bigger

Our 80 weights fit in 160 bytes. The 8B model needs a data-centre GPU. Here is where the cost comes from.



16 Matrices and GPUs

Every step in this guide was a batch of dot products. A GPU's job is doing thousands of them at once.

Why does AI run on GPUs?

COMES IN	WHO DOES THE WORK	HANDS ON
All the dot products (Ch 5–13) →	GPU: thousands of multiply-add units	→ Speed limits → memory (Ch 17)

IN ONE LINE GPUs win because the dot products are independent.

A **matrix multiplication** does many independent dot products at once. Both tokens through all 3 detectors = 6 dot products in one operation. A GPU runs thousands of such operations in parallel.

	Our model	The 8B model
Operations per token (≈ 2 × weights)	≈ 180	≈ 16 billion
Reading a 1,000-token prompt	≈ 4 million (attention dominates in a model this small)	≈ 16 trillion (attention adds only a few %)
GPU peak (dense 16-bit)	≈ 989 trillion operations per second	

Many dot products, one call — Python

```
scores = H @ W1.T     # (tokens x 4) @ (4 x detectors): every pair at once
# On a GPU with PyTorch the same line runs on thousands of cores:
# scores = H.cuda() @ W1.cuda().T
```

SCALE IT UP: WHAT GROWS, AND WHAT IT COSTS

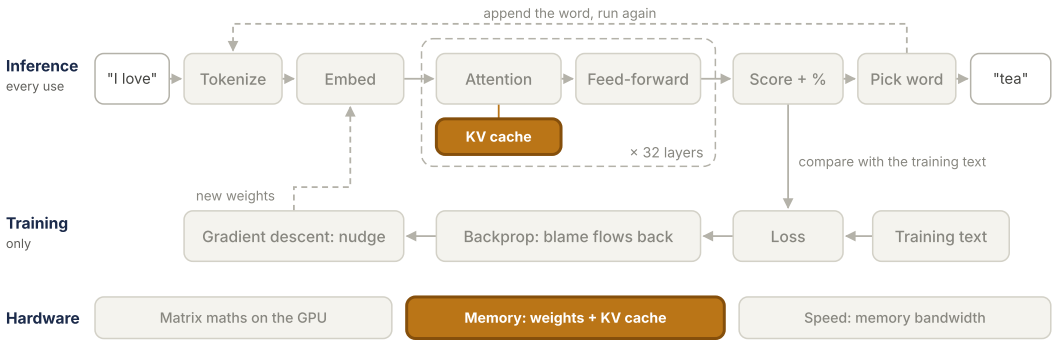
Knob	Toy → 8B model	Compute	Memory	Good	Bad
Rows per matrix (tokens or users together)	2 → thousands	Big matrices keep the GPU busy	More intermediate values	High throughput	Each user may wait longer
Model bigger than one GPU	—	Split across GPUs that exchange results every layer	Spread across GPUs	Run models that don't fit one chip	Needs fast GPU-to-GPU links

INSIGHT

GPUs win on three things: massive parallelism (detector 1 on "I" doesn't wait for detector 2 on "love"), very high memory bandwidth, and hardware built specifically for matrix multiplication.

WATCH OUT

One-token-at-a-time generation is a one-row matrix, too small to keep the chip busy (Ch 18).



17 Memory: weights plus cache

Before speed comes fit. Weights and cache must sit in GPU memory.

How much memory does a model need?

COMES IN
The model file + live caches

→

WHO DOES THE WORK
GPU memory (80 GB on a data-centre GPU)

→

HANDS ON
What must be read every step → bandwidth (Ch 18)

IN ONE LINE Weights are a fixed cost; the cache grows per user and per token.

Storage	Bytes per number	The 8B model weights
16-bit (BF16)	2	8.03 B × 2 ≈ **16 GB**
8-bit	1	≈ 8 GB
4-bit	0.5	≈ 4–5 GB (some parts kept at higher precision)

Units: cache sizes use binary units (1 KB = 1,024 bytes, 1 GB = $1{,}024^3$ bytes, strictly KiB/GiB); weight sizes use decimal GB (10^9 bytes). All figures are rounded engineering estimates.

KV CACHE, THE 8B MODEL AT 16-BIT

```
per token  = 128 KiB
1 user,   8,192 tokens  =  1 GiB
1 user, 131,072 tokens  = 16 GiB  (≈ the weights)
10 users, 32,768 tokens = 40 GiB  (≈ 2.5× the weights)
```

Decision matrix: which fix for which problem

Problem	Fix	What it changes	Cost
Weights don't fit	Quantisation	Fewer bytes per weight	Some quality loss, more at 4-bit
Cache too big per token	GQA (shared K/V) / MLA (compressed K/V)	Fewer numbers stored per token	Decided at training time
Cache too big overall	KV quantisation	Store cache in 8-bit	Small quality loss
Very long input	Sliding window / offload	Keep only recent tokens, or park old ones in CPU memory or storage	Forgets, or slower
Same prompt reused	Prefix caching	Reuse the cache across requests	Memory to hold it
Memory wasted in gaps	PagedAttention	Cache stored in small fixed-size blocks	Engine complexity

Memory arithmetic, and loading in 4-bit — Python

```
weights_gb = 8.03e9 * 2 / 1e9                      # ~16 GB at 16-bit
kv_per_token = 2 * 32 * 8 * 128 * 2                # 131,072 bytes
print(weights_gb, kv_per_token * 32768 * 10 / 2**30)   # 10 users x 32K: 40 GiB

from transformers import BitsAndBytesConfig        # pip install bitsandbytes
q = BitsAndBytesConfig(load_in_4bit=True)
model = AutoModelForCausalLM.from_pretrained(name, quantization_config=q)
```

SCALE IT UP: WHAT GROWS, AND WHAT IT COSTS

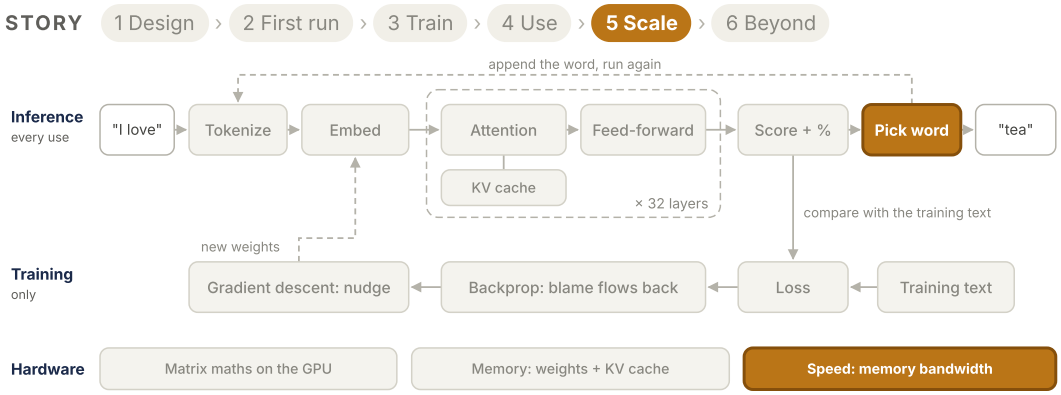
Knob	Toy → 8B model	Compute	Memory	Good	Bad
Bytes per weight	2 → 1 → 0.5	Fewer bytes to read → faster decoding	16 → 8 → ≈ 4–5 GB	Fits smaller GPUs and laptops	Quality loss grows at low bits
Parameters	8 B → 70 B → 405 B	∝ parameters	≈ 16 → 140 → 810 GB at 16-bit	Generally more capable	Several GPUs; slower and costlier

INSIGHT

Weights are a fixed cost. The cache is per user, per token. At scale the cache often decides how many users fit on a GPU.

WATCH OUT

Real inference memory = weights + KV cache + temporary working values + framework overhead. Training memory is far larger again (Ch 12: ≈ 16 bytes per weight before working values).



18 Compute vs memory: why answers type out word by word

An data-centre GPU can do ≈ 989 trillion operations a second. One token of the 8B model needs ≈ 16 billion. So why isn't it instant?

What limits the speed of an answer?

COMES IN
Weights + cache in memory (Ch 17)

→

WHO DOES THE WORK
Memory bandwidth: ≈ 3.35 TB/s

→

HANDS ON
Tokens per second → you

IN ONE LINE Single-user decode speed is mostly limited by bytes moved per token.

ONE TOKEN, ONE USER, THE 8B MODEL ON ONE DATA-CENTRE GPU

```
Maths:           16 billion ops    ÷ ~989 trillion/s   ≈ 0.016 ms
Weights to read: 16.06 GB (10⁹)    ÷ ~3.35 TB/s (10¹²) ≈ 4.8 ms    ← the real wait
Upper bound ≈ 1 ÷ 4.8 ms ≈ 209 tokens/s, i.e. roughly 200
```

A bandwidth-bound upper-bound estimate in decimal units, before cache traffic. Real throughput depends on effective bandwidth, weight format, cache traffic and serving setup.

For one user, each new token needs **all the weights** read from memory, plus the cache, but only a little maths per number read. The calculators wait on memory.

WHAT THIS ESTIMATE IS, AND ISN'T

An **idealised ceiling**: one user, 16-bit weights, no batching, perfect use of bandwidth, cache traffic ignored. Real throughput depends on batching, quantisation, KV-cache size, kernel efficiency and the serving software. Use it to reason about *what limits speed*, not to predict a benchmark.

	Prefill (reading your prompt)	Decode (writing the answer)
Tokens per pass	All at once	One
Weights read	Once for the whole prompt	Once per token
Limited by	**Compute**	**Memory bandwidth**
You feel it as	Wait before the first word	Speed of the words

Batching fixes most of it: read the weights once and apply them to many users' next tokens together. The catch: each user brings a separate cache.

The speed ceiling in three lines — Python

```python
bytes_per_token = 16.06e9            # all weights, 16-bit
bandwidth = 3.35e12                  # data-centre GPU SXM, bytes per second
print(bandwidth / bytes_per_token)   # ~209 tokens/s ceiling for one user
```

SCALE IT UP: WHAT GROWS, AND WHAT IT COSTS

Knob	Toy → 8B model	Compute	Memory	Good	Bad
Users per batch	1 → 32	One weight read serves 32 tokens: far more total throughput	32 caches	Lower cost per token	Memory fills; each user may slow down
Memory bandwidth	—	Directly sets the decode ceiling	—	Faster typing-out	The expensive part of the chip

INSIGHT

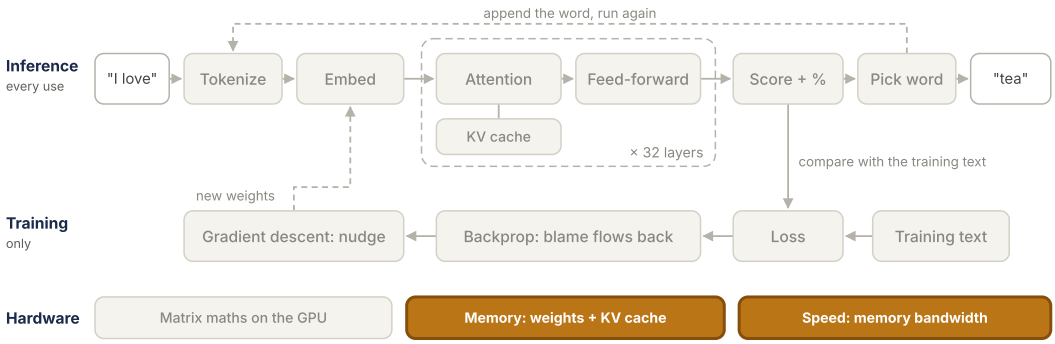
Almost every inference trick (quantisation, GQA/MLA, batching, KV compression) is about **moving fewer bytes per token**, not doing less maths.

MYTH

Models type word by word for effect.

REALITY

Each word genuinely needs a full pass that reads every weight.



19 Fewer bits: a smaller copy of the same model

The same trained weights can be stored with fewer bits each. You get a smaller, faster copy that answers almost the same, until it doesn't.

What changes when you pick the 8-bit or 4-bit version of a model, and which parts suffer first?

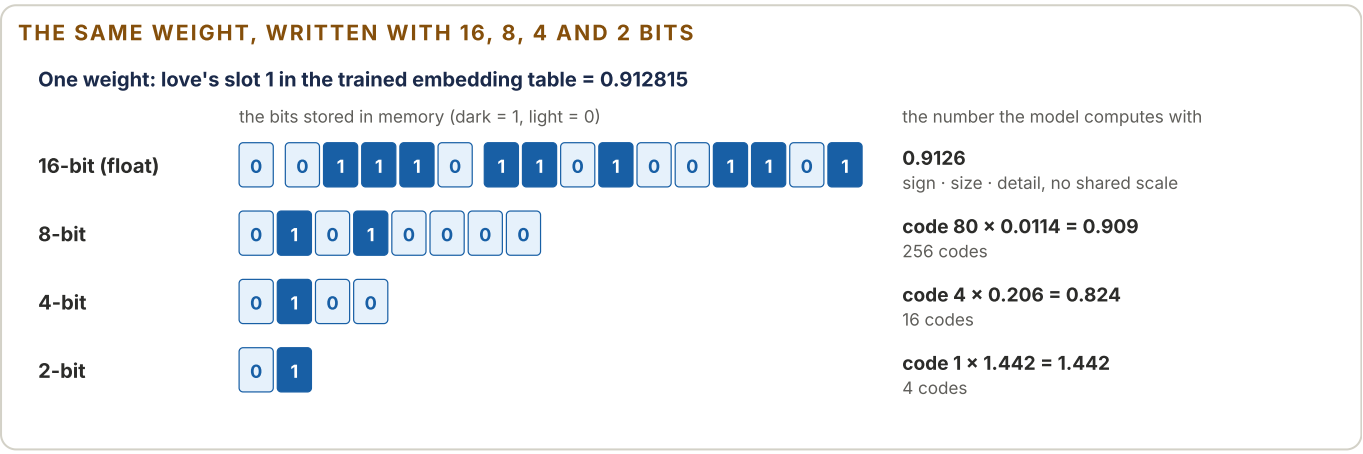
COMES IN	WHO DOES THE WORK	HANDS ON
Trained weights (Ch 12–13)	Rounding: each weight snapped to a coarser ruler	A smaller copy → memory (Ch 17) and speed (Ch 18)

IN ONE LINE Fewer bits = a smaller, faster copy of the same model; quality holds, then drifts, then breaks.

A trained model is a file of numbers (Ch 13). Each number can be stored with more or fewer **bits** (on/off digits). Fewer bits = a coarser ruler. The model's shape and arithmetic stay the same; only the stored values get rounded. This is **quantisation**, and it is what the "8-bit" and "4-bit" versions of the same model are.

What a bit is, in our model's terms

- A **bit** is one on/off switch: 0 or 1. Each extra bit doubles the number of different codes: 2 bits → 4, 4 bits → 16, 8 bits → 256, 16 bits → 65,536.
- Every **cell** in every grid you met (the embedding table, Wq, Wk, Wv, Wo, W1, W2: the 80 weights of Ch 3) is one stored number. Its bits are simply how that number is written in memory.
- **Our model uses 64 bits per weight** (NumPy's default number format): 80 weights × 64 bits = 5,120 bits = **640 bytes**. Real models are usually trained with a mix of 16- and 32-bit numbers and published at 16-bit (2 bytes per weight).
- The shape, the steps and the maths don't change. Only how precisely each cell is written.
- **The name:** rounding stored numbers onto a fixed set of allowed values (fewer bits = fewer values) is called **quantisation**. To quantise means to force a value onto fixed steps, like a ruler with marks. "An 8-bit model" or "a 4-bit model" is a quantised copy: same weights, fewer bits each.



WHAT IT SHOWS

One real weight written four ways. 16-bit floats split their bits into a sign, a size and the detail. The small formats store a whole-number code; the model multiplies it by the row's shared scale to get the value back.

WHY IT MATTERS

Fewer bits = fewer possible values. 0.913 survives at 16 and 8 bits, becomes 0.824 at 4 bits and 1.442 at 2 bits. Every dot product that uses this cell now uses the new value.

Comparison: bits per weight, our model and the 8B

Bits per weight	Codes per weight	Our 80 weights	The 8B model's weights	Bytes read per token (8B)	Decode ceiling (Ch 18)
64 (ours, as trained)	**≈ 18 billion billion**	**640 bytes**	**64.2 GB**	**64.2 GB**	**≈ 52 tokens/s**
32	≈ 4.3 billion	320 bytes	32.1 GB	32.1 GB	≈ 104 tokens/s
16	65,536	160 bytes	16.1 GB	16.1 GB	≈ 209 tokens/s
8	256	80 bytes	8.0 GB	8.0 GB	≈ 417 tokens/s
4	**16**	**40 bytes**	**4.0 GB (+ scales)**	**4.0 GB**	**≈ 834 tokens/s**
2	4	20 bytes	2.0 GB	2.0 GB	≈ 1,670 tokens/s

Size = weights × bits ÷ 8. Ceiling = 3.35 TB/s ÷ bytes read per token: the idealised upper bound from Ch 18, before scales, unpacking and cache reads.

How bits reach the answer: rounding → dot products → scores → shares

Nothing new happens in the maths. The same dot products (Ch 5) run on rounded cells, so the vocabulary scores (Ch 9) move a little, and softmax turns the **gap** between scores into shares.

Weights stored at	tea score	coffee score	Gap (tea − coffee)	tea / coffee share
64-bit (as trained)	7.69	6.99	+0.70	66.6% / 33.1%
8-bit	7.66	6.98	+0.69	66.4% / 33.3%
4-bit	8.16	7.27	+0.89	70.1% / 28.8%
2-bit	**5.62**	**6.64**	**-1.02**	**26.2% / 72.7%**

INSIGHT

Only the gap matters. A gap of +0.70 means tea gets $e^{0.70} \approx 2$ times coffee's share: the 2 : 1 split. 4-bit rounding widened the gap to +0.89 (a bit too sure); 2-bit rounding pushed it to −1.02, and the answer flipped. Bits are a **storage** choice; their effect arrives through the same arithmetic you already know.

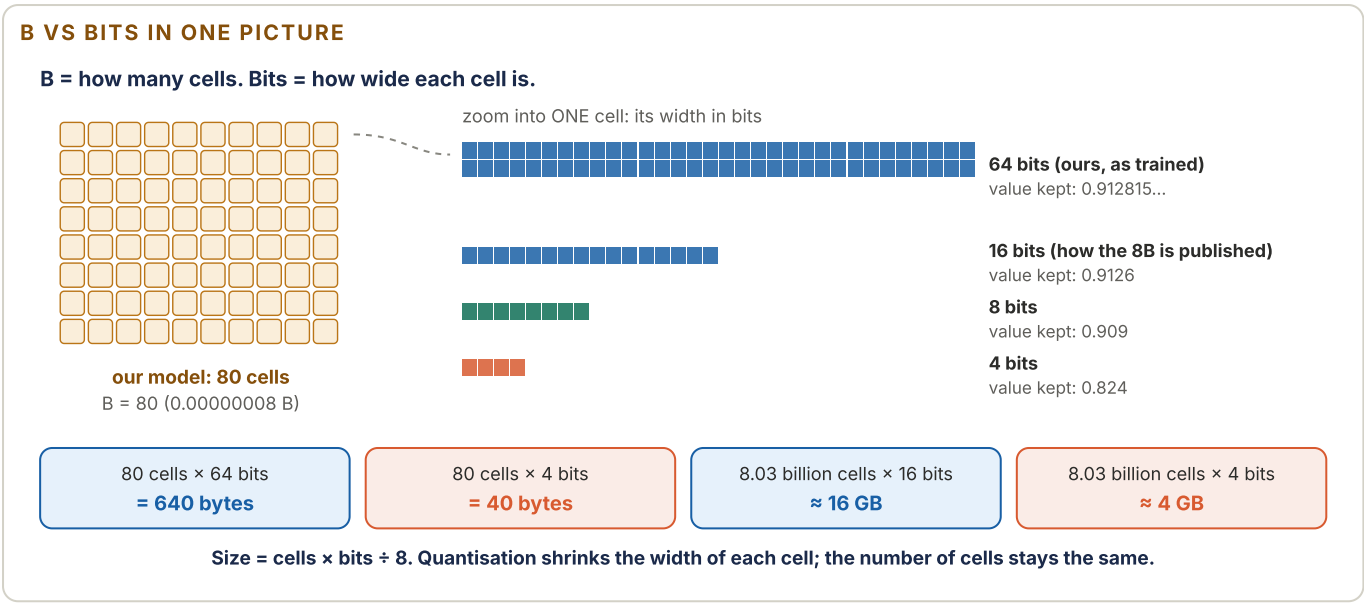
B or bits? Two different numbers

	B (as in "8B")	Bits (as in "4-bit")
Measures	**How many** numbers the model has: every cell in every weight grid	**How precisely** each number is written
Ours	80 (Ch 3)	64 as trained
The 8B model	8.03 billion	16 as published
Set by	The shape: slots, layers, detectors, vocabulary	A storage choice, made after training
To change it	Design and train a different model	Round the same weights (quantisation, minutes)
Changes the answers?	Yes: a different model	Barely at 8-bit, a little at 4, a lot at 2

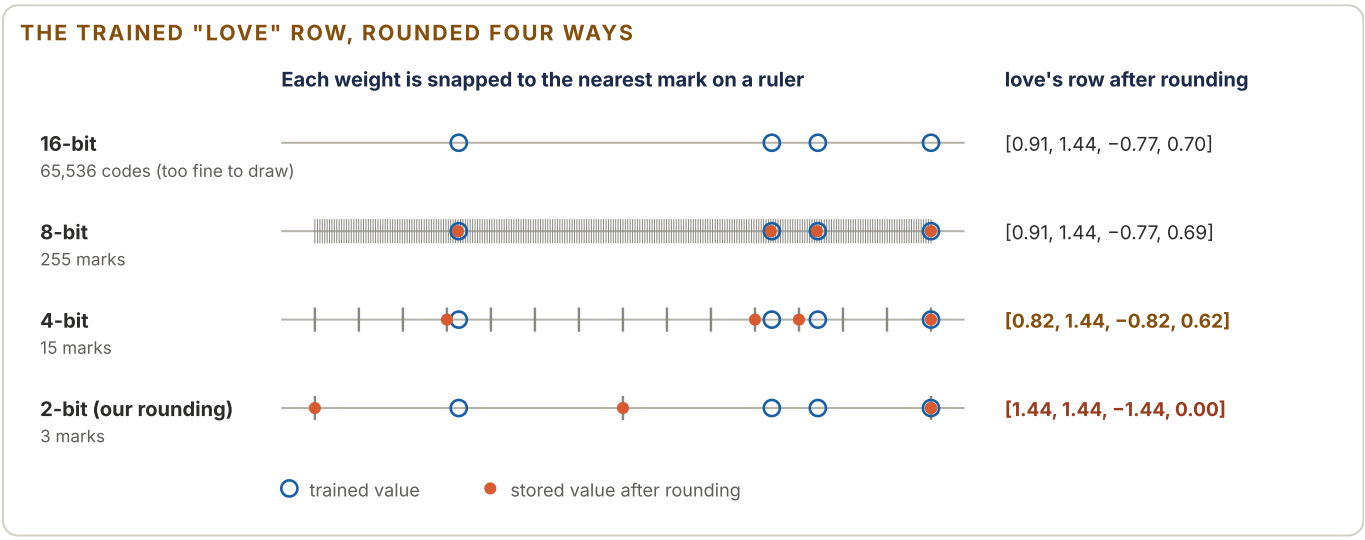
Size = B × bits ÷ 8 bytes. Ours: 80 × 64 ÷ 8 = 640 bytes. The 8B model: 8.03 billion × 16 ÷ 8 ≈ 16 GB; at 4-bit, still 8.03 billion numbers, ≈ 4 GB. **Quantisation changes the bits, never the B.**

WATCH OUT: FOUR LOOK-ALIKE UNITS

B in "8B" = billion, a *count* of weights. **bit** = one 0/1 switch. **byte** = 8 bits. **GB** = a billion bytes, a *size* in memory. So "an 8B model at 4-bit" = 8 billion numbers × 4 bits ≈ 4 GB.



In plain words: a ruler with fewer marks



WHAT IT SHOWS

Each line is a ruler from −1.44 to +1.44 (the row's largest value). Blue rings are the trained values; red dots are where rounding puts them. At 8-bit the marks are so close that nothing visibly moves. At 4-bit values shift by up to 0.09. At 2-bit there are only 3 marks, so 0.91 becomes 1.44 and 0.70 becomes 0.

WHY IT MATTERS

Each mark needs a code. 8 bits give 256 codes, 4 bits give 16, 2 bits give 4. Fewer codes = smaller file and fewer bytes to read per token (Ch 18), but coarser numbers.

- **What it is:** store each weight as a small whole number (which mark), plus one shared scale per row (how far apart the marks are).
- **Who decides:** whoever publishes or runs the model, usually *after* training. The weights are not retrained; they are rounded.
- **When it runs:** once, when the copy is made. Using the copy is the same pipeline (Ch 13–15), reading rounded numbers.

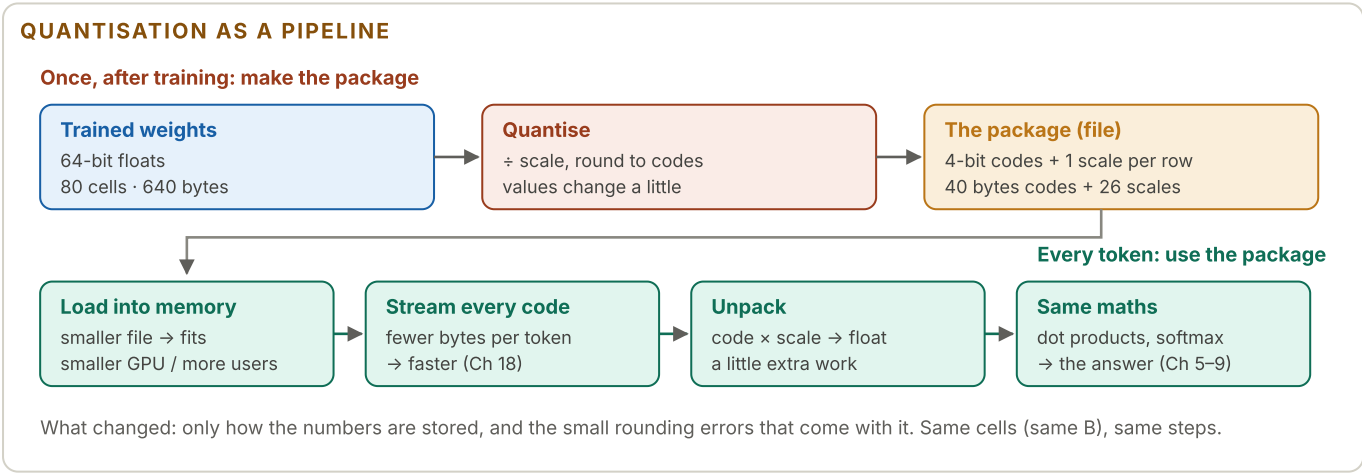
How quantisation is done, step by step (love's row, 4-bit)

Step	What happens	Our numbers
1 · Find the biggest	The largest value in the row, ignoring the sign	1.442
2 · Count the codes	4 bits give whole numbers −7 … +7	7 steps each side
3 · Make the scale	Biggest ÷ 7 = the gap between marks	**0.206**
4 · Weight → code	Divide by the scale, round to a whole number	0.913 ÷ 0.206 = 4.43 → **4** 1.442 ÷ 0.206 = 7.00 → **7** -0.769 ÷ 0.206 = -3.73 → **-4** 0.697 ÷ 0.206 = 3.38 → **3**
5 · Store	4 codes of 4 bits, plus one 16-bit scale	codes 4, 7, -4, 3; scale 0.206
6 · Use	code × scale, then the usual dot products	[0.824, 1.442, -0.824, 0.618]

	slot 1	slot 2	slot 3	slot 4	Bits for this row
Original (64-bit)	0.913	1.442	-0.769	0.697	4 × 64 = 256
Rebuilt (4-bit)	**0.824**	**1.442**	**-0.824**	**0.618**	**4 × 4 + 16 = 32 (8× smaller)**
Rounding error	-0.089	+0.000	-0.055	-0.079	

Do this for every row of every grid and you have "the 4-bit model". Real tools add refinements: a scale per small group (32–128 weights) instead of per row, sensitive parts kept at 16-bit, sample text used to pick the rounding that changes the outputs least, and, for very low bits, training with the rounding applied from step 1.

So is it just packaging? Mostly: lossy packaging



What changed: only how the numbers are stored, and the small rounding errors that come with it. Same cells (same B), same steps.

WHAT IT SHOWS

Top: done once, after training. The trained 64-bit weights are rounded into small codes plus one scale per row: the package. Bottom: every token, the package is streamed through the chip, unpacked back to numbers, and the usual maths runs.

WHY IT MATTERS

Unlike a zip file, you don't get the exact numbers back, so answers can shift. But a smaller package means less memory (what fits) and fewer bytes per token (speed). Like exporting a video at a lower bitrate: same film, smaller file, fine detail lost first.

The whole model at each bit width (real runs of our trained model)

Bits per weight	Our 80 weights	Loss (best 0.520)	"I love" → tea / coffee	"I" → love (text: 75%)	"I hate" → rain
64-bit (as trained)	640 bytes	0.523	66.6% / 33.1%	77%	99.6%
16-bit	160 bytes	0.523	66.6% / 33.1%	77%	99.6%
8-bit	80 bytes	0.523	66.4% / 33.3%	77%	99.6%
4-bit	**40 bytes**	**0.529**	**70.1% / 28.8%**	**77%**	**99.9%**
3-bit	30 bytes	0.559	59.7% / 36.8%	81%	86.6%
2-bit	20 bytes	1.201	26.2% / 72.7%	100%	100.0%

- **8-bit:** no visible change. Loss 0.523, split 66 / 33.
- **4-bit:** half the size of 8-bit; loss 0.529. The split drifts to 70% / 29%: still the right preference.
- **2-bit:** the preference **flips**: coffee 73%, tea 26%; loss 1.201, more than double.

The pattern: **nothing, nothing, a little, then a cliff.** Quality does not fall in a straight line with the bits.

Which part feels it first, and the side effect

We rounded one part at a time to 2-bit and kept the rest exact.

Rounded to 2-bit (rest kept as trained)	Weights	Loss	tea / coffee	"I" → love	What breaks first
Embedding table (E)	24	0.725	23% / 67%	82%	Which word wins. In our model the same table also scores the answer (Ch 9), so rounding hits twice: at the start and at the end
Attention (Wq Wk Wv Wo)	32	0.784	63% / 31%	98%	The finer odds: "I → love" drifts from 75% toward 98%. The model gets overconfident
Feed-forward (W1 W2)	24	0.835	64% / 30%	99%	Biggest loss rise; again the 75 / 25 split collapses toward 99 / 1
Nothing rounded	**80**	**0.523**	**67% / 33%**	**77%**	—

Rounded to 3-bit	Loss	tea / coffee	"I" → love
Embedding table (E)	0.576	74% / 26%	54%
Attention (Wq Wk Wv Wo)	0.583	64% / 34%	91%
Feed-forward (W1 W2)	0.538	61% / 39%	83%

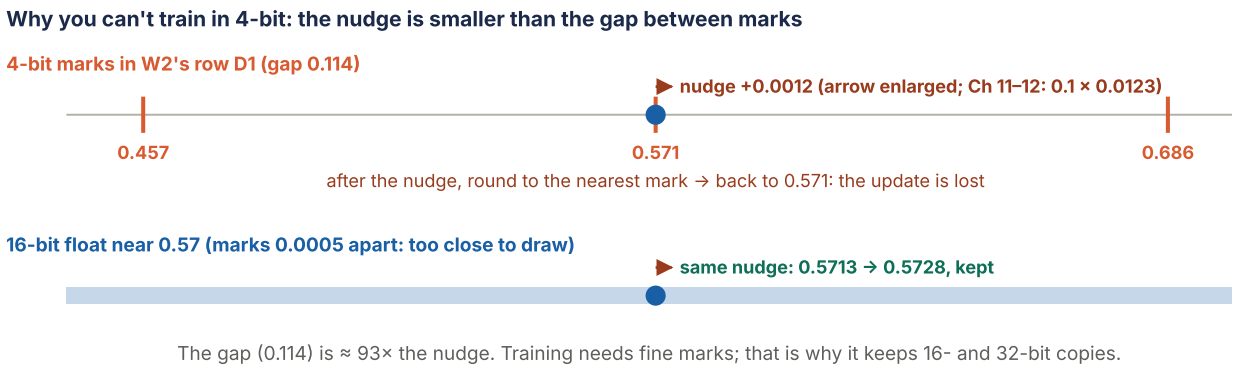
- **Embedding table:** its rounding changes *which* word wins (coffee overtakes tea). It is used twice, so its error lands directly on the final scores.
- **Attention and feed-forward:** their rounding mostly destroys the *odds*: honest 75 / 25 becomes a near-certain 98 / 2. A rounded model can sound more sure while being less faithful to its training text.
- **Side effects you would notice:** different answers to the same prompt, overconfidence, and errors on the less common cases first (here the 1-in-4 "I hate").

HONEST NOTE

These are 80 weights and 3–4 sentences, so single percentages jump around. Treat the **pattern** as the lesson (fine at 8-bit, drift at 4, break at 2; different parts fail differently), not the exact numbers.

Why not always use fewer bits?

THE BIGGEST DRAWBACK: TRAINING CANNOT HAPPEN IN LOW BITS



The gap (0.114) is ≈ 93× the nudge. Training needs fine marks; that is why it keeps 16- and 32-bit copies.

WHAT IT SHOWS

The weight from Ch 11 (0.6, stored as 0.571 at 4-bit). Its training nudge is +0.0012. On the 4-bit ruler the next mark is 0.114 away, so rounding throws the nudge away. On a 16-bit ruler the marks are 0.0005 apart, so the same nudge survives.

WHY IT MATTERS

Training is millions of tiny nudges. That is why models are trained in 16/32-bit and quantised *after* training, and why 1.58-bit models train with hidden high-precision copies.

Drawback	What happens	Our numbers
Fewer marks	Every weight moves to the nearest mark	4-bit: up to 0.09; 2-bit: 0.70 → 0
Errors add up	Each score sums many rounded products; 32 layers feed each other	Ours: 4 products per score. 8B: 4,096
Softmax magnifies	Shares follow e^{gap}, so a small score shift moves shares a lot	Gap +0.70 → −1.02 at 2-bit: answer flips
One big weight coarsens the row	The largest value sets the scale for all others	Max 1.442 makes 2-bit marks 1.44 apart
Rare cases go first	Fine odds collapse; the model sounds surer	"I → love" 77% → 99% (feed-forward at 2-bit)
Unpacking costs work	Codes must become numbers before the maths; hardware favours 16/8-bit	4× fewer bytes gives less than 4× speed
Can't train in it	**Nudges are smaller than the gap between marks**	**0.0012 vs 0.114**

The same choice for the 8B model

Version	Weights in memory	Decode ceiling (Ch 18)	Typical fit	Typical quality
16-bit	16.1 GB	≈ 209 tokens/s	One data-centre GPU, lots of room for cache	The reference
8-bit	8.0 GB	≈ 417 tokens/s	Data-centre GPU with 2× the cache room; large consumer GPUs	Usually very close to 16-bit
4-bit	**≈ 4.0 GB + scales (≈ 4.5 GB)**	**≈ 834 tokens/s**	**8 GB consumer GPU, laptop**	**Small, measurable loss; check your own tasks**
2–3-bit	≈ 2–3 GB	higher still	Phones, very small devices	Often a clear drop unless special methods are used

Ceilings are the idealised Ch 18 estimate (bytes of weights ÷ 3.35 TB/s). Real gains are smaller: low-bit numbers must be unpacked, and the cache is read too. Scales cost extra: one 16-bit scale per group of 32 weights adds 0.5 bit per weight.

Decision matrix: which version should you pick?

Your situation	Pick	Why	Watch
Quality matters most, memory is fine	16-bit	No rounding at all	Most memory, slowest per token
Serving many users on data-centre GPUs	8-bit	Half the memory goes to KV cache, so more users fit (Ch 17)	Check results match on your prompts
Laptop or one consumer GPU	4-bit	The only way the model fits	Hard tasks (maths, long reasoning, rare languages) are often reported to drop first
Tiny device	Compare 2–3-bit of this model vs 4-bit of a smaller model	Either can win; it depends on the task	Test both on your own examples
Fine-tuning on a small GPU	Frozen 4-bit base + 16-bit LoRA (Ch 20)	Only the small add-on is trained, at full precision	The add-on learns on top of rounded weights

Round a weight matrix to n bits (what we ran above) — Python

```
def quantise(W, bits):
    levels = 2 ** (bits - 1) - 1                # 8-bit: 127, 4-bit: 7, 2-bit: 1
    scale = abs(W).max(axis=1, keepdims=True) / levels   # one scale per row
    codes = np.round(W / scale)                 # small whole numbers: what is stored
    return codes * scale                        # what the model computes with
```

IN A REAL MODEL (8B)

The common 4-bit formats round small groups of weights (often 32–128) with their own scale, and many keep the embedding and output tables, plus the small normalisation weights, at higher precision. Those parts hold few of the 8 billion weights but, as in our test, their errors land directly on the answer. The KV cache can be stored in 8-bit too: 128 KiB → 64 KiB per token, so twice the context or users, at the cost of attention reading rounded keys and values.

SCALE IT UP: WHAT GROWS, AND WHAT IT COSTS

Knob	Toy → 8B model	Compute	Memory	Good	Bad
Bits per weight	32 → 16 → 8 → 4	Fewer bytes read per token: faster decode, up to the bandwidth ceiling	16 → 8 → ≈ 4.5 GB for 8B	Fits smaller hardware; more room for KV cache	Coarser numbers; drift, then a cliff
Group size for scales	row → 32 weights	Slightly more work to unpack	More scales: +0.5 bit per weight at 32	Finer rounding where values differ	Bigger file than the bits suggest

INSIGHT

Quantisation changes the **numbers**, not the **model**: same shape, same steps, same 80 (or 8 billion) weights. That is why a 4-bit copy can be made in minutes without any training, and why it can quietly behave a little differently.

MYTH

4-bit is 4× worse than 16-bit.

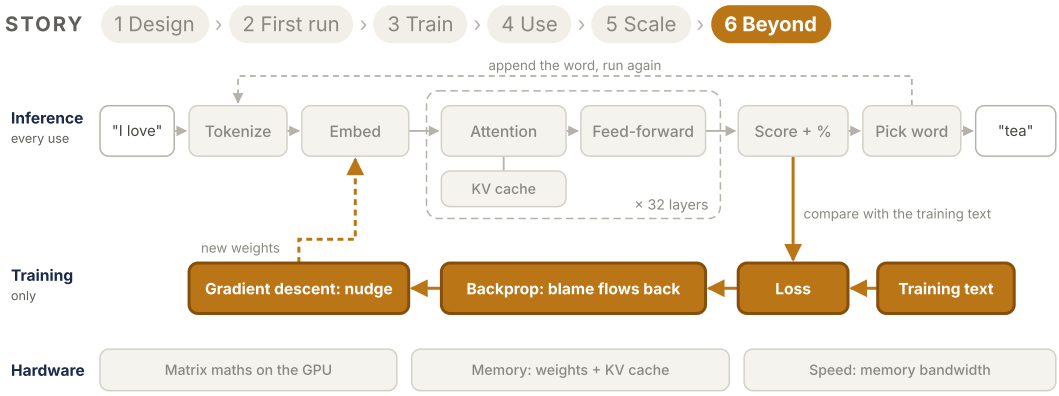
REALITY

Quality isn't proportional to bits. Our model: 8-bit identical, 4-bit close, 2-bit broken. Large models show the same shape: little loss down to a point, then a steep drop.

STAGE 6

Beyond pre-training: how today's large systems go further

A pre-trained model predicts text. Assistants that follow instructions, prefer good answers and use tools get three more layers: fine-tuning, reinforcement learning, and a harness of code around the model.



20 Fine-tuning: more training, on new text

Pre-training is expensive and done once. Fine-tuning reuses the trained weights and trains a little more on a smaller, targeted text.

How do you change a trained model's behaviour, and what does it cost?

COMES IN	WHO DOES THE WORK	HANDS ON
Trained weights (Ch 12) + new text	The same training loop (Ch 10–12)	Adjusted weights → use (Ch 13–15)

IN ONE LINE Fine-tuning is the same training loop on new text, starting from trained weights.

A café wants the model to recommend coffee. New text: **"I love coffee" × 3**. Same loss, same backprop, same update rule, learning rate 0.1. Only the starting point differs: trained weights instead of random ones.

Option 1: full fine-tuning (all 80 weights move)

Step	"I love" → tea	→ coffee	"I" → love / hate	Loss on the original text
0	67%	33%	77% / 23%	0.523
5	3%	97%	98% / 2%	1.412
10	1%	98%	99% / 1%	1.640
20	1%	99%	99% / 0%	1.905
40	0%	100%	100% / 0%	2.204

- Coffee jumps from 33% to 97% in just 5 steps.
- **Side effect:** "I" → hate falls from 23% to 0.2%, though we never asked for that. The new text never contains "hate", so training pushed it down. Loss on the original text rose from 0.523 to 2.204. This is **catastrophic forgetting**.

Option 2: LoRA (freeze the model, train a small add-on)

LoRA (low-rank adaptation) freezes every original weight and adds a small correction built from two thin matrices. We add one to the feed-forward write rows: a 3 × 1 column times a 1 × 4 row = **7 trainable numbers** instead of 80. The row starts at zero, so training begins from the unchanged model.

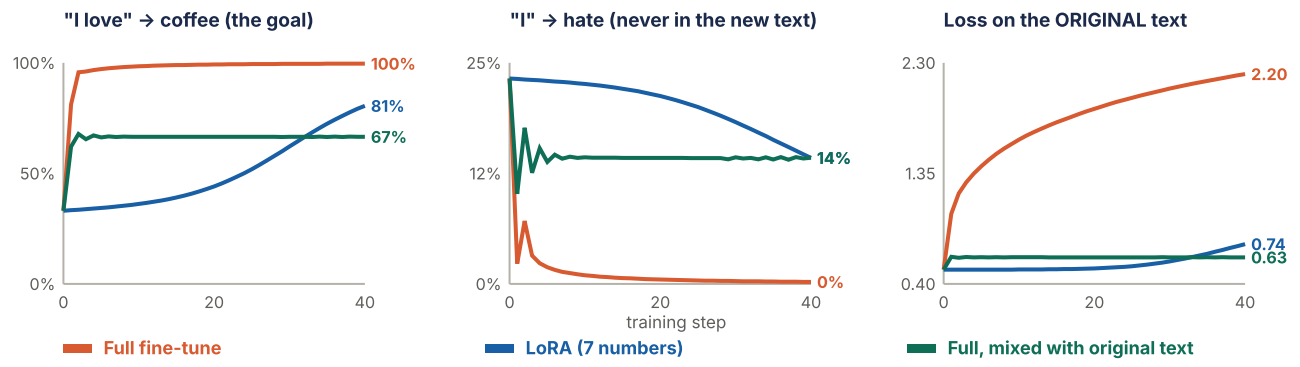
Step	"I love" → tea	→ coffee	"I" → love / hate	Loss on the original text
0	67%	33%	77% / 23%	0.523
5	65%	34%	77% / 23%	0.523
10	64%	36%	77% / 23%	0.524
20	56%	44%	79% / 21%	0.533
40	19%	81%	86% / 14%	0.743

- Coffee rises more slowly (81% after 40 steps), and the original text is damaged far less (loss 0.743 vs 2.204).
- Fewer trainable numbers means less to store and less to disturb. In **this** experiment that gave less change and less forgetting. It isn't a general guarantee: forgetting with LoRA depends on rank, where adapters are placed, learning rate and training length.

The maths of fine-tuning, in pictures

Three runs, 40 steps each, learning rate 0.1, all starting from the trained model: **full** fine-tuning on "I love coffee" × 3, **LoRA** on the same text, and full fine-tuning on the same text **mixed** with the original 4 sentences.

1 · THE GOAL AND THE SIDE EFFECTS, STEP BY STEP



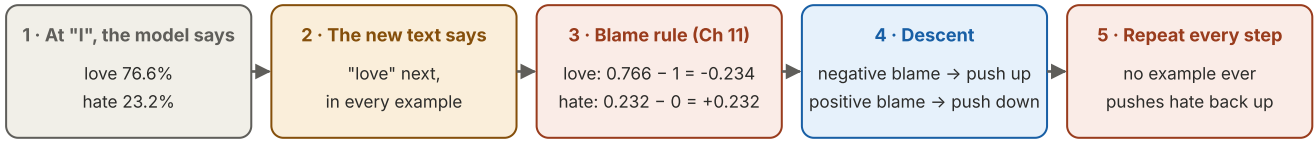
WHAT IT SHOWS

Left: all three raise coffee. Middle: full fine-tuning drives "I → hate" toward 0; the mixed run settles at 14%. Right: the original text's loss climbs from 0.52 to 2.20 with full fine-tuning, but stays near 0.63 when mixed.

WHY IT MATTERS

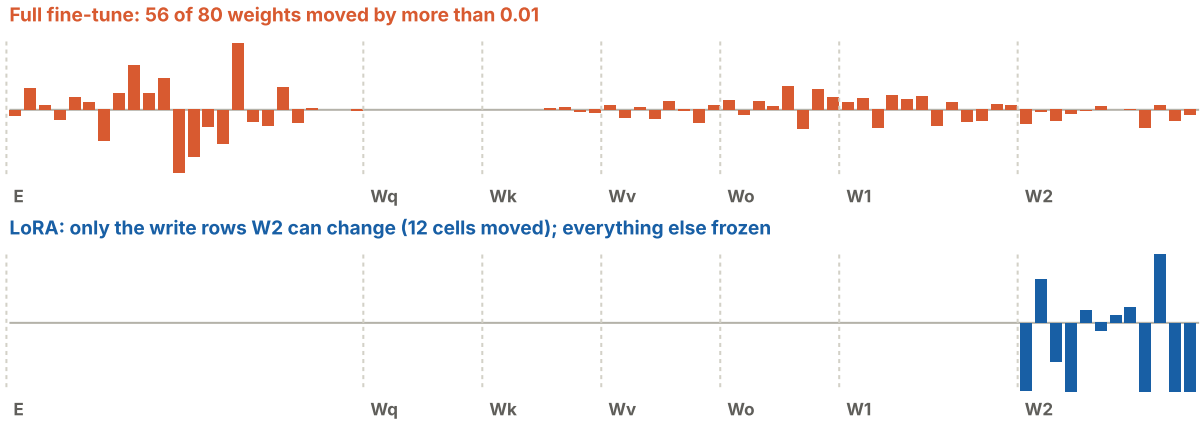
The goal and the damage come from the same nudges. You only see the damage if you measure the things you *didn't* train on.

2 · WHY "HATE" DISAPPEARS: THE BLAME RULE, APPLIED TO ONE-SIDED DATA



After 40 steps: "I" → hate 23.2% → 0.2%. Nobody asked for this: it is what the arithmetic does with one-sided data.

3 · WHAT EACH METHOD IS ALLOWED TO CHANGE



WHAT IT SHOWS

The change in every one of the 80 weights after 40 steps. Full fine-tuning touches almost every grid. LoRA can only change the write rows, through its 7-number add-on.

WHY IT MATTERS

Fewer weights allowed to move = less room for side effects, but also less room for the change you want (coffee reached 81% with LoRA vs 99.7% with full fine-tuning).

What it affects: every prompt, before and after

Measured on	Before	Full	LoRA	Mixed
"I love" → coffee (goal)	33.1%	99.7%	80.6%	66.6%
"I" → hate	23.2%	0.2%	14.2%	14.3%
"I hate" → rain	99.6%	94.5%	98.7%	99.6%
"I love tea" → rain (never trained)	94.5%	46.8%	89.0%	92.3%
Loss on the original text	0.523	2.204	0.743	0.629
Loss on the new text	0.686	0.003	0.186	0.281

- **Mixed data gives exactly the mixed text's odds:** coffee 4 of 6 = 67%, hate 1 of 7 = 14%. The model learns the proportions of whatever you feed it, so **the data mix is the specification**.
- **Unseen prompts drift too:** "I love tea" → rain was never trained in either run, yet full fine-tuning moved it from 95% to 47%.

Cost arithmetic: full vs LoRA

	Trainable numbers	Training memory (16 bytes per trainable + frozen weights)	What you save afterwards
Ours, full	80	80 × 16 = 1.3 KB	All 80 weights (640 bytes at 64-bit)
Ours, LoRA	7	Frozen 80 + 7 × 16 bytes	7 numbers (56 bytes)
8B, full	8.03 billion	≈ 128 GB, before working values: several GPUs	A whole new 16 GB model
8B, LoRA r = 16 on Q and V	**6.8 million (0.08%)**	**16 GB frozen + 6.8 M × 16 bytes ≈ 0.11 GB: one GPU**	**A ≈ 14 MB adapter (16-bit); swap per customer or task**

Compute per training token is similar for both: the backward pass still runs through the whole model (Ch 11). LoRA saves memory and storage, not most of the arithmetic.

Cautions: what to watch, with our numbers

Caution	What you see	Our numbers	What to do
Forgetting	Behaviour the new data doesn't show fades	"I → hate" 23.2% → 0.2%; original loss 0.52 → 2.20	Mix in original-style data; test old tasks every run
Over-certainty	Small, repetitive data gets memorised	coffee 99.7%; new-text loss 0.003: one sentence learned by heart	Fewer steps, varied examples, a held-out test set
Drift on unseen prompts	Answers change where you never trained	"I love tea" → rain 95% → 47%	Test prompts outside the fine-tuning set
Too gentle	A small add-on may underfit	LoRA coffee 34% at step 5, 81% at step 40	More steps, higher rank, more target grids
Learning rate / steps	Too much changes too fast	Full: coffee 33% → 97% in 5 steps	Small learning rate, early stopping (Ch 12)

INSIGHT

Fine-tuning is Chapters 10–12 again, starting from trained weights. Everything it changes follows from the blame rule: it pushes up what the new data shows and pushes down everything else. So **what you leave out of the data is as important as what you put in**.

LoRA in a few lines — Python

```
A = rng.normal(0, 0.5, size=(3, 1))   # small random column
B = np.zeros((1, 4))                  # starts at zero: no change at step 0
W2_used = W2_frozen + A @ B           # only A and B are trained (7 numbers)

# Real models, with the peft library:
from peft import LoraConfig, get_peft_model
model = get_peft_model(model, LoraConfig(r=16, target_modules=["q_proj", "v_proj"]))
```

IN A REAL MODEL (8B)

Rank-16 LoRA on the 8B model's query and value matrices trains ≈ 6.8 million numbers, about 0.08% of the model. Real post-training typically uses several rounds of supervised fine-tuning on curated examples, plus preference training (Ch 21).

SCALE IT UP: WHAT GROWS, AND WHAT IT COSTS

Knob	Toy → 8B model	Compute	Memory	Good	Bad
Full fine-tuning	80 → 8 billion trainable	Training compute ∝ tokens × 8 B	≈ 16 bytes per weight with Adam ≈ 128 GB+	Biggest possible change	Several GPUs; risk of forgetting
LoRA	7 → ≈ 7 million trainable	Backward pass still runs through the whole model	Frozen base at 2 bytes/weight (16 GB) + small adapter and its optimiser state	Fits on one GPU; adapters are megabytes and swappable	Smaller change; may underfit big shifts

INSIGHT

Fine-tuning teaches **behaviour and format** well: tone, style, "always answer in JSON". For facts that change often, retrieval (putting text in the prompt) is usually cheaper and easier to update.

WATCH OUT

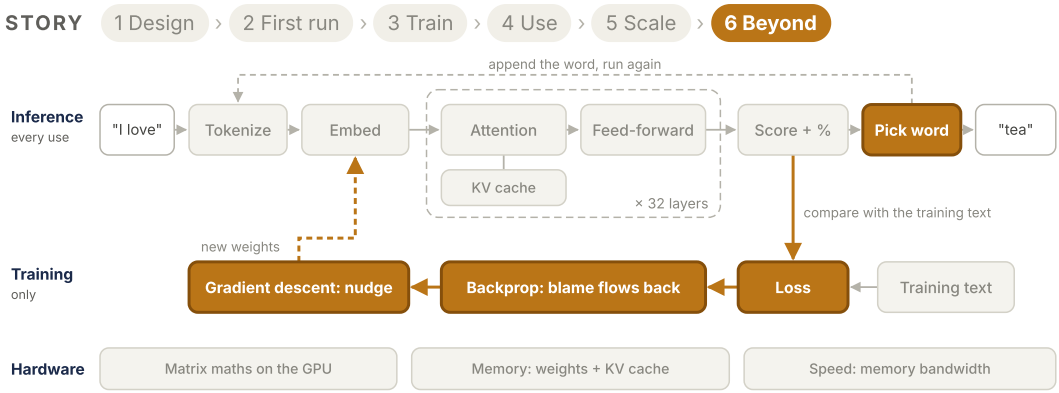
Fine-tuning data only shows what to do more of. Anything it leaves out may drift, as "hate" did here. Teams mix in some original-style data and test old behaviour after every run.

MYTH

Fine-tuning adds a new knowledge module.

REALITY

It nudges existing weights (or a small add-on) with the same loop as pre-training.



21 Reinforcement learning: rewarding answers, not copying text

Fine-tuning needs example answers. Reinforcement learning only needs a score for answers the model produced itself.

How do you train toward "better" when you can't write the perfect answer?

COMES IN	WHO DOES THE WORK	HANDS ON
Trained model + a reward rule	Reward, then the same backprop and nudge	A model that prefers rewarded answers → use

IN ONE LINE RL raises the share of rewarded answers; a penalty keeps the model close to where it started.

The loop changes in one place. Instead of comparing with a text, the model **tries** answers, a **reward** scores them, and the update raises the share of answers that scored above average.

#	What happens	Our numbers
1	The model generates answers (sampling, Ch 14)	4 tries after "I love": tea, coffee, tea, tea
2	A reward scores each one	café rule: coffee = 1, anything else = 0 → [0, 1, 0, 0]
3	**Advantage** = how much better than the group average	mean 0.25, spread 0.43 → [−0.58, 1.73, −0.58, −0.58]
4	Backprop + nudge: raise the share of above-average answers, lower the rest	coffee up, tea down

Step 3 is the group-relative scoring used by some reasoning-model training (e.g. GRPO, used in published reasoning-model training). Classic RLHF instead trains a **reward model** on human comparisons ("answer A is better than B") and optimises against it.

Our run: maximise reward, with and without a "stay close" penalty

An illustration of one simplified objective (reward − β × KL) on a single next-word decision. Production RL optimises whole answers with more elaborate algorithms, so these percentages show the mechanism, not how a real model's answers would shift.

The penalty is the **KL divergence**: a number that grows as the new percentages drift from the original model's. Objective = reward − β × KL. β sets how much drifting costs.

Step	β = 0: coffee	Loss on original text	β = 0.5: coffee	KL	Loss on original text
0	33.1%	0.523	33.1%	0.000	0.523
10	99.4%	1.749	78.5%	0.434	0.701
25	99.8%	2.136	78.5%	0.434	0.700
50	99.9%	2.493	78.5%	0.434	0.700
100	100.0%	2.927	78.5%	0.434	0.700
200	100.0%	3.451	78.5%	0.434	0.700
300	100.0%	3.796	78.5%	0.434	0.701

- **No penalty (β = 0):** coffee hits 100% and the model's fit to its original text collapses (loss 0.523 → 3.796). It maximised the reward and gave up everything else.
- **With penalty (β = 0.5):** coffee settles at **78.5%**. Theory predicts the balance point exactly: new share ∝ old share × e^(reward ÷ β), which gives **78.5%**. Our training landed on it. (β = 1 would give 57.4%.)

The RL step, and the closed-form check — Python

```
rewards = np.array([0, 1, 0, 0])                # one score per sampled answer
adv = (rewards - rewards.mean()) / rewards.std() # [-0.58, 1.73, -0.58, -0.58]
# update: weights += lr * sum(adv_i * gradient of log p(answer_i))

p_ref = np.array([.664, .333])                  # tea, coffee before RL
r, beta = np.array([0, 1]), 0.5
p_best = p_ref * np.exp(r / beta); p_best /= p_best.sum()   # coffee ~ 0.787
```

IN A REAL MODEL (8B)

Published recipes vary. Classic RLHF uses a learned reward model, PPO and a KL penalty. Many recent open models use Direct Preference Optimisation (DPO), which learns from preference pairs without a separate RL loop, often over several rounds. Reasoning models are often trained with **checkable rewards**: did the maths answer match, did the code pass its tests?

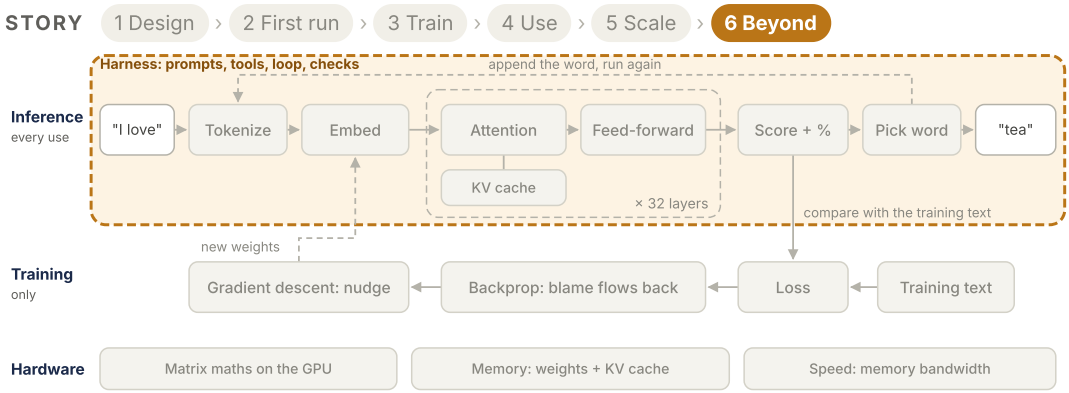
SCALE IT UP: WHAT GROWS, AND WHAT IT COSTS

Knob	Toy → 8B model	Compute	Memory	Good	Bad
Samples per prompt	4 → many per prompt, millions of prompts	Generating answers is inference: slow, token by token, and often the biggest cost of RL	Policy, reference model and sometimes a reward model in memory together	Learns from its own attempts; needs no perfect example answers	Expensive; unstable without care
Answer length	1 word → thousands of reasoning tokens	Longer answers cost more per sample	More KV cache per sample	Reasoning models learn to think longer when it raises reward	Longer, costlier answers at use time too

WATCH OUT: REWARD HACKING

The model maximises the **score**, not your intention. With β = 0 ours abandoned everything for the reward. Learned reward models can be fooled by answers that look good but aren't, which is one reason checkable rewards and KL penalties are used.

MYTH	REALITY
RL teaches the model new facts.	Mostly it reshapes which of the model's existing behaviours become likely. Here it moved shares between tea and coffee; it couldn't invent a drink it never learned.



22 The harness: the code wrapped around the model

What you use in a product is rarely the bare model. It's a loop of plain code that builds the prompt, calls tools, checks results and calls the model again.

What does the system around the model do, and what's the arithmetic of it?

COMES IN	→	WHO DOES THE WORK	→	HANDS ON
A trained model		Harness: ordinary code		Answers, actions, results → you

IN ONE LINE The harness is the software that turns a model's capabilities into a reliable product.

At each step the model does one computation: given tokens, output percentages for the next token. Trained models can use that to follow instructions, write structured tool calls and reason over several steps. The harness is the code that organises those abilities into a product:

Harness part	What it does	How it reaches the model
System prompt	Instructions, persona, rules	Tokens placed before your message
Retrieval (RAG)	Finds relevant documents	Pasted into the prompt as tokens
Tools	Search, calculator, code runner, database	Model writes a structured request; code runs it; the result is appended as tokens
Loop / agent	Repeats think → act → observe until done	Each turn re-sends the growing context
Checks	Tests, validators, filters, retries	Code decides whether to accept or retry
Routing	Picks a small or large model per request	Different model, same interface
Chat interface	Keeps the history, applies the chat template, streams the reply	The whole conversation, re-sent as tokens each turn

The chat interface: the harness you use every day

A chat window feels like a conversation. The model sees something simpler: **one long token stream**, rebuilt and re-sent on every turn.

#	What happens	Example
1	You type a message	"What goes with tea?"
2	The app adds the system prompt and **every earlier message**	system + turn 1 + reply 1 + … + your new message
3	A **chat template** joins them into one text, with special tokens marking who said what	see below
4	The model predicts the reply token by token; the app **streams** each token to your screen as it arrives	"Biscuits…"
5	It stops at the end-of-turn token; the app saves the reply into the history	ready for the next turn

```
<|begin_of_text|><|start_header_id|>system<|end_header_id|>
You are a helpful assistant.<|eot_id|><|start_header_id|>user<|end_header_id|>
What goes with tea?<|eot_id|><|start_header_id|>assistant<|end_header_id|>
```

One real open model's chat format. The model continues from the last line; the special tokens were learned in fine-tuning (Ch 20).

What it feels like	What actually happens
The model remembers our chat	The model's weights don't change during a chat, and it doesn't remember across independent requests. The app manages the history and re-sends it each turn; the serving system may cache parts of the computation (prefix caching), but logically the whole history is the input
It types as it thinks	Streaming: each token is shown as soon as it's picked (Ch 14)
A long chat "forgets" the start	History exceeded the context window, so the app dropped or summarised old turns
It remembers me across chats	A memory feature: the app stores notes and pastes them into the prompt

CHAT ARITHMETIC, THE 8B MODEL

```
Each turn (question + reply) ≈ 500 tokens
Turn 20 re-sends ≈ 10,000 tokens   → ≈ 1.2 GiB of KV cache
Context limit 131,072 tokens        → full after ≈ 260 turns; then the app must trim
```

Without prefix caching, every turn re-reads the whole history in prefill, which is why long chats get slower and costlier.

The arithmetic that shapes harness design

Question	Calculation	Result
Long multi-step tasks	Each step succeeds 95% of the time; all must succeed: 0.95 × 0.95 × …	5 steps: 77% · 10 steps: **60%** · 20 steps: 36%
Same, with better steps	0.99 per step	10 steps: 90% · 20 steps: 82%
Majority vote	Ask 5 times, take the majority; each answer right 60% of the time	5 votes: **68%** · 9 votes: 73%
Best of n with a checker	Each try right 30%; tests can tell which tries pass	1 try: 30% · 5 tries: **83%** · 10 tries: 97%
Cost of a 10-turn loop	Each turn adds 2,000 tokens and re-sends everything so far: 2,000 × (1 + 2 + … + 10)	**110,000** input tokens supplied, not 20,000. With prefix caching, much of the already-seen part needn't be recomputed, though it's still logically part of the input

- Reliability **multiplies** across steps, which is why harnesses add checks and retries after risky steps.
- Voting and best-of-n trade **more tokens** (cost, time) for accuracy. This is one form of "test-time compute". Majority voting over reasoning paths is called **self-consistency**.
- The voting and retry figures assume independent attempts. Real attempts make correlated mistakes, so real gains are smaller.

Chat template and a minimal tool-using loop — Python

```python
text = tok.apply_chat_template(messages, tokenize=False, add_generation_prompt=True)  # one
token stream

messages = [{"role": "system", "content": "Use tools when needed."},
            {"role": "user",   "content": "What is 1234 * 5678?"}]
while True:
    reply = model(messages)                          # the model: next-token prediction
    if reply.tool_call:                              # it asked for a tool, as text/JSON
        result = run_tool(reply.tool_call)           # plain code, e.g. a calculator
        messages += [reply, {"role": "tool", "content": str(result)}]
        continue                                     # call the model again
    break                                            # final answer
```

IN A REAL MODEL (8B)

Chat assistants, coding agents and research tools are a trained model (Ch 1–21) inside a harness like this. Speed and cost follow Chapters 15–18: every loop re-reads context, so the KV cache, prefix caching and batching matter even more for agents.

SCALE IT UP: WHAT GROWS, AND WHAT IT COSTS

Knob	Toy → 8B model	Compute	Memory	Good	Bad
Loop turns	1 → dozens	Each turn is a full model call	Context and KV cache grow every turn	Can finish multi-step tasks	Cost grows faster than the number of turns; errors compound
Samples per question	1 → n (voting, best-of-n)	n × the tokens	n × the cache, or n × the time	Higher accuracy where answers can be checked or compared	n × the cost

INSIGHT

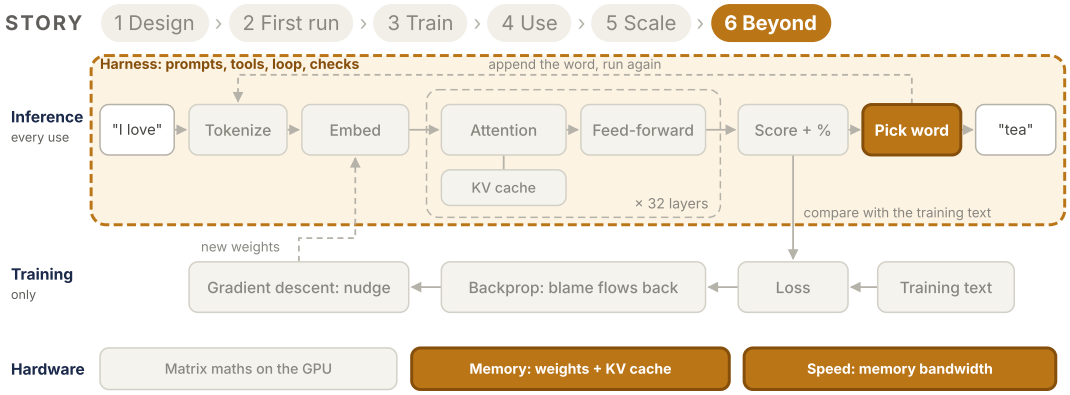
Model and harness contribute different parts. The model supplies the capabilities: language, instruction following, tool-call writing, reasoning. The harness makes them reliable and useful with retrieval, tools, checks and retries. Much of the practical difference between AI products comes from these engineering choices, which you can reason about with the arithmetic above.

MYTH

The model browses, runs code and remembers you.

REALITY

The model writes text, including tool requests it was trained to produce. The harness runs the browsing and code and stores the memory, then feeds the results back in as tokens.



23 Picking a model: the maths, put to work

You will probably never build a model. You will pick one, size the hardware, choose a fix, and decide when to trust it. Every one of those calls is arithmetic from this guide.

Given a real task, how do I choose the model, its size and its bits, and how did this guide prepare me for that?

COMES IN
Everything from Ch 1–22

→

WHO DOES THE WORK
You: six questions, each answered with a number

→

HANDS ON
A choice you can defend: model, size, bits, fix, checks

IN ONE LINE Picking a model is arithmetic you now know: fit, speed, knowledge, trust and cost.

What this guide set out to do, and the proof it did

The promise	What you now know	The number that shows it	Ch
You stop guessing	Every step is arithmetic you have seen: lookup, dot products, softmax, add. Training is measure → blame → nudge.	Random weights gave tea 18%; trained weights give 66.6%. Same steps, different numbers	4–13
You can explain "why is it slow?"	Each new token reads every weight. Memory speed, not maths, sets the pace.	16 GB ÷ 3.35 TB/s ≈ 4.8 ms per token → ceiling ≈ 209 tokens/s for the 8B model	15–18
You know when not to trust it	A share is a preference, not a check of truth. The model always produces an answer.	After "I love tea" (never in the text) it still says rain 95%	9, 14
You pick the right fix	Facts in the prompt go through attention; behaviour comes from weights; size and bits set memory and speed.	Fine-tuning moved coffee to 97% but "I → hate" fell 23% → 0.2%. 4-bit kept the answer; 2-bit flipped it	6, 17–20
You talk to engineers on the same page	Tokens, context, KV cache, quantisation, LoRA, decode speed: each is now a number you can check.	128 KiB of cache per token; 16 bytes per weight to train; 7 LoRA numbers vs 80	all

Six questions to pick a model

Question	How to answer it	Where the maths came from
1 · What must it know?	Facts that change → put them in the prompt (RAG). A fixed style or format → fine-tune.	Attention is the only step that reads the prompt (Ch 6). Weights change only by training (Ch 12, 20)
2 · Does it fit?	Memory = weights + KV cache. Weights = parameters × bytes. Cache = 128 KiB × tokens × users (8B, 16-bit).	Ch 17, 19
3 · Is it fast enough?	Writing speed ≤ bandwidth ÷ bytes read per token. Fewer bits or a smaller model → faster.	Ch 18, 19
4 · Is it good enough?	Test on your own examples. Bigger is usually more capable; low bits drift, then break.	Ch 13, 19
5 · Can I trust the output?	Multi-step work multiplies errors. Add checks, votes or tests in the harness.	0.95^{10} = 60%; majority of 5 at 60% → 68%; best of 5 with a checker at 30% → 83% (Ch 22)
6 · What will it cost?	You pay per token, and chats and agents re-send the history every turn.	10 turns × 2,000 tokens = 110,000 input tokens (Ch 22); output tokens priced higher

Worked example: a team assistant on one data-centre GPU (80 GB)

The task: answer questions about 2,000 internal documents that change weekly. 20 people at once, up to 8,192 tokens each.

#	What happens	Our numbers
1	**Know:** the documents change weekly, so retrieve the relevant passages into the prompt (RAG). Fine-tuning would need retraining every week, and it can make the model forget (Ch 20)	RAG, not fine-tuning
2	**Fit, 8B at 16-bit:** weights 16.1 GB; cache 20 users × 8,192 tokens × 128 KiB = 20 GiB (21.5 GB)	≈ 37.5 GB of 80: fits, room to spare
3	**Fit, a 70B model at 4-bit:** weights ≈ 40 GB; its cache is 320 KiB per token (80 layers × 8 K/V heads × 128 × 2 × 2 bytes), so 20 × 8,192 × 320 KiB = 50 GiB (53.7 GB)	≈ 94 GB: does **not** fit
4	**Fix the 70B:** store the cache in 8-bit (Ch 17, 19): 26.8 GB	≈ 67 GB: fits, but attention now reads rounded K and V
5	**Speed:** single-user ceilings ≈ 209 tokens/s (8B, 16-bit) vs ≈ 84 (70B, 4-bit: 40 GB ÷ 3.35 TB/s). Batching 20 users shares each weight read	8B is faster per user
6	**Decide by testing:** run both on 50 real questions with known answers. Pick the 8B unless the 70B is clearly better on *your* questions	A choice you can defend

INSIGHT

In step 3 the **cache**, not the weights, broke the budget. People size hardware from the model's name ("70B") and forget the per-user, per-token memory. You now know both halves of the sum.

From 80 weights to a real model: what scaling would take

Our model has **80** weights: 0.00000008 B, not 80B. Could we just train it on a library? No:

- **It can't read new words.** 6 rows in the embedding table means 6 words in and out. Any other word has no row.
- **It is already full.** Loss 0.523 against a best possible 0.520: its 80 numbers hold everything they can. On a library they would average over millions of patterns and learn little more than which words are common.
- **Size and text grow together.** A common rule of thumb for the best result per unit of training compute is ≈ **20 training tokens per weight**. Ours needs only ≈ 1,600. Real small models are often trained far past it (the 8B saw ≈ 1,870 tokens per weight) because extra training makes them better without making them dearer to use.

Weights	Tokens (≈ 20 per weight)	Training compute (6 × weights × tokens)	Time on 1 GPU at 40%	Training memory (16 bytes / weight)	Memory to use (16-bit)	Roughly what you get
80 (ours)	**1,600**	**7.7×10^{5}**	**under a second**	**1.3 KB**	**160 bytes**	**Our tea / coffee split**
1 M	20 M	1.2×10^{14}	under a second	16 MB	2 MB	Short, simple, mostly grammatical sentences (on simple text)
100 M	2 B	1.2×10^{18}	≈ 51 minutes	1.6 GB	200 MB	Fluent-sounding text, weak on facts
1 B	20 B	1.2×10^{20}	≈ 84 hours	16 GB	2 GB	Useful on narrow tasks after fine-tuning
8 B	160 B	7.7×10^{21}	≈ 5,413 hours	128.5 GB	16.1 GB	A capable general model
8 B, as actually trained	15 T	7.2×10^{23}	≈ 0.5 million hours	128.5 GB	16.1 GB	The 8B model in this guide

Time = compute ÷ (989 trillion × 40%); divide by the number of GPUs used in parallel. The last row's published figure is 1.46 million GPU-hours (Ch 12). "What you get" is a rough guide from published small-model experiments; text quality and training recipe matter as much as size. Money = GPU-hours × your hourly rate.

INSIGHT

The maths of this guide is the same at every size: lookup, dot products, softmax, loss, blame, nudge. Only the counts change, and the bill grows with **weights × tokens**. To get a better model you grow the *shape* (vocabulary, slots, layers, detectors) and the *text* together.

BEYOND MORE WEIGHTS

A real model also needs: a sub-word tokenizer so any word can be written (Ch 2); the parts ours leaves out, such as word positions, several heads and normalisation for stable deep stacks (Ch 6–8); clean, varied text; and post-training (Ch 20–21) to turn a text predictor into an assistant.

Estimating cost from these terms

Every cost is one of three sums, and each term in it is something this guide measured. Prices below are **made-up example rates** so the arithmetic is visible: put in your provider's real numbers.

How you run it	The sum	Terms from the guide
Pay per token (an API)	input tokens × input price + output tokens × output price	Tokens (Ch 2), history re-sent (Ch 22), output written one pass per token (Ch 14, 18)
Run it yourself	GPUs needed × price per GPU-hour × hours	Memory decides how many GPUs (Ch 17, 19); bandwidth and batching decide tokens per hour (Ch 18)
Train or fine-tune	6 × parameters × training tokens ÷ (GPU speed × utilisation) = GPU-seconds	The 6 × rule (Ch 11–12); 16 bytes per weight for memory (Ch 12)

1 · Pay per token: the team assistant

#	What happens	Our numbers
1	**Count tokens per question:** 5 retrieved passages × 500 tokens + the question ≈ 3,000 input tokens; the answer ≈ 300 output tokens (≈ 225 words, since 100 tokens ≈ 75 words)	3,000 in, 300 out
2	**Count per day:** 20 people × 30 questions	1.8 M in, 0.18 M out
3	**Multiply by rates** (example: 0.50 per million in, 2.00 per million out)	0.90 + 0.36 = **1.26 per day** ≈ 38 per month

Term you now know	What it does to the bill	Example
Chat history re-sent (Ch 22)	Every turn pays again for all earlier turns	10 turns × 2,000 tokens = 110,000 input tokens, not 20,000 (5.5×)
RAG passages (Ch 6)	Retrieved text is input tokens on every question	Sending 2 passages instead of 5 cuts input ≈ 50%
Output tokens (Ch 14, 18)	Each one is a full pass; usually priced higher	Ask for shorter answers
Hidden reasoning tokens	Billed as output even though you don't see them	2,000 hidden + 300 shown = 7.7× the output cost
Votes and retries (Ch 22)	Each try is a full call	Majority of 5 = 5× the cost, for 60% → 68% accuracy
Language (Ch 2)	Some languages need more tokens per word	Same text, more tokens, higher bill

2 · Run it yourself

- **How many GPUs:** from memory (Ch 17, 19). 8B at 16-bit + cache for 20 users ≈ 37.5 GB: one 80 GB GPU. A 70B at 16-bit is 140 GB of weights alone: at least two GPUs, so twice the hourly bill.
- **Tokens per hour:** one user's ceiling is ≈ 209 tokens/s (Ch 18). With 20 users batched, each weight read serves 20 tokens: ceiling ≈ 4,180 tokens/s, before cache reads. Real servers reach less; **measure it**.
- **Cost per million tokens** = price per GPU-hour ÷ (tokens per second × 3,600) × 1,000,000. Example: 3.00 per hour at a measured 1,000 tokens/s ≈ **0.83 per million tokens**.

- **Idle time counts:** you pay for the hour whether or not anyone asks. Busy half the day → each token costs double.

3 · Train or fine-tune

- **Fine-tune the 8B on 10 million tokens:** 6 × 8.03 B × 10 M ≈ 4.8×10^{17} operations. At 989 trillion/s and 40% utilisation: ≈ 1,200 s ≈ **20 GPU-minutes** of compute. Memory, not time, is the hurdle: full fine-tuning needs ≈ 128 GB (16 bytes per weight), so 2+ GPUs, or LoRA on one.
- **Pre-train the 8B on 15 trillion tokens:** ≈ 7.2×10^{23} operations → ≈ 0.5 million GPU-hours at 40%. The published figure is 1.46 million: real runs lose time to restarts, communication and extra stages. The formula gives the **size** of the bill, not the exact invoice.

Cost calculator (example rates) — Python

```python
# 1. Pay per token
q_in, q_out = 3000, 300                    # tokens per question
per_day = 20 * 30                          # people x questions
api = per_day * (q_in * 0.50 + q_out * 2.00) / 1e6
print(f"API: {api:.2f} per day")           # 1.26

# 2. Run it yourself
gpu_hour, tok_s = 3.00, 1000               # measured throughput
print(f"Self-host: {gpu_hour / (tok_s * 3600) * 1e6:.2f} per million tokens")  # 0.83

# 3. Train / fine-tune
flops = 6 * 8.03e9 * 10e6                  # params x tokens
print(f"Fine-tune: {flops / (989e12 * 0.4) / 60:.0f} GPU-minutes")         # ~20
```

INSIGHT

The cheapest saving is usually **fewer tokens**, not a cheaper model: shorter history, fewer retrieved passages, shorter answers. Every one of those is a term in the sums above.

Decision matrix: situation → choice → the guide's reason

Situation	Choose	Why, in this guide's numbers	Ch
Facts change often (docs, prices, policies)	RAG with a capable model	The prompt reaches the answer through attention; no retraining needed	6
Need a fixed tone, format or narrow skill	Fine-tune, ideally LoRA	7 trainable numbers moved coffee to 81%; full fine-tuning also caused forgetting	20
Prefer one answer over another ("helpful", "safe")	RL with a penalty for drifting	β = 0.5 gave coffee 78.5%; β = 0 collapsed to 100%	21
Must run on a laptop or stay private	8B at 4-bit	≈ 4.5 GB; 4-bit kept our answer, 2-bit flipped it	19
Many users, one GPU	8-bit weights; budget the cache	Cache = 128 KiB × tokens × users; it often decides how many users fit	17
Very long documents	Long-context model + cache budget, or RAG to send less	131,072 tokens = 16 GiB of cache per user, about the size of the 8B's weights	15, 17
Fast, short answers matter	Smaller model or fewer bits	Speed ≈ bandwidth ÷ bytes per token	18
Long multi-step tasks (agents)	Add checks and tests in the harness, not just a bigger model	95% per step → 60% over 10 steps	22
High-stakes answers	Verify outside the model	A confident share can be wrong: rain 95% after a sentence it never saw	14

WATCH OUT

None of these numbers replaces testing. They tell you **what to test and what it will cost**: whether it fits, how fast it can be, where it can go wrong. The final call comes from your own examples.

MYTH

The biggest model is always the best choice.

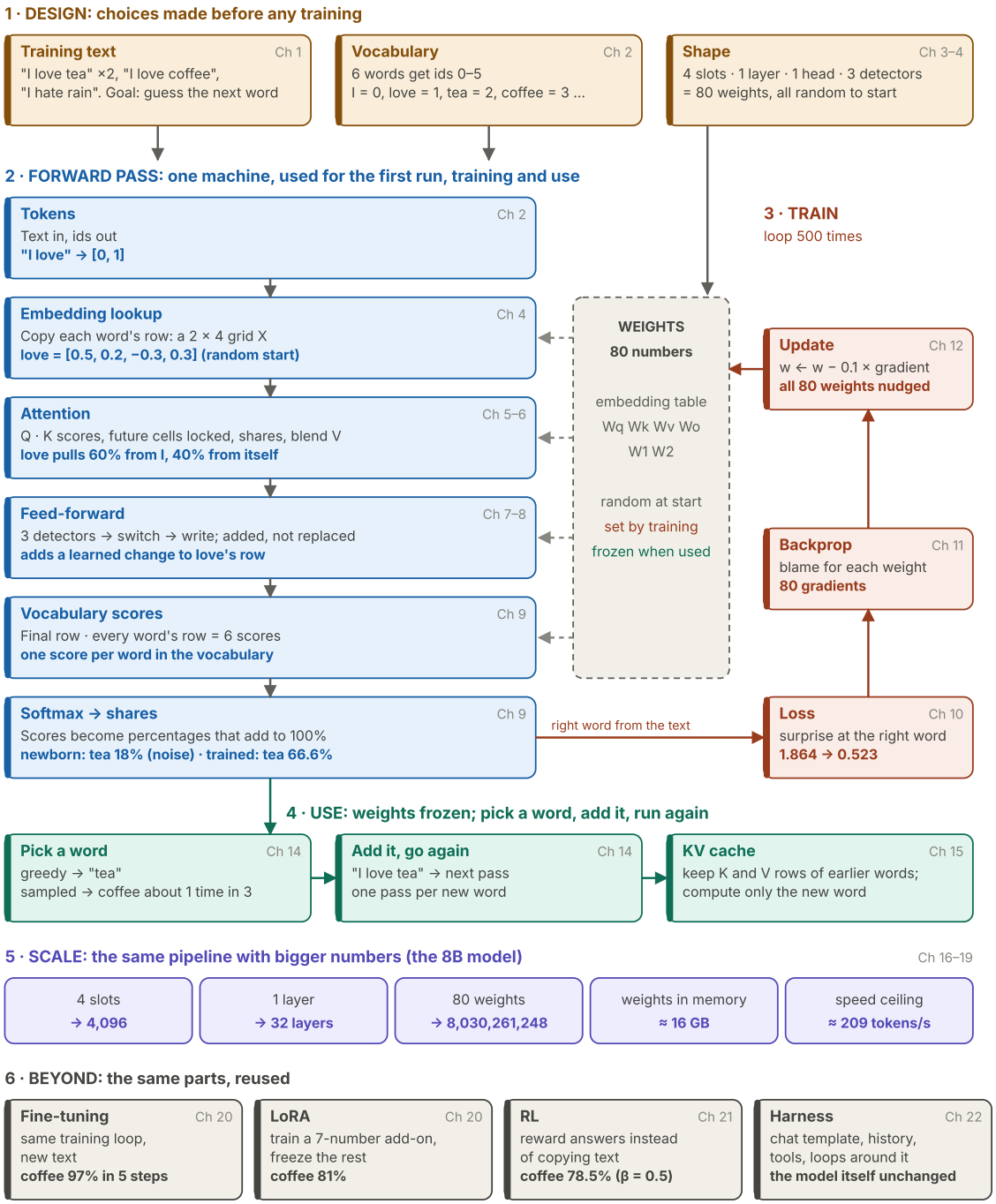
REALITY

Bigger usually means more capable, but also more memory, slower tokens and higher cost, and it may not fit beside its cache. The right model is the smallest one that passes *your* tests, at a speed and cost you can afford.

THE WHOLE GUIDE IN ONE LINE A model is numbers in fixed places, set by training and read on every token. Knowing that, you can reason about what it knows, what it costs, how fast it runs and when to check it.

The whole pipeline in one picture

Every chapter connected, top to bottom. All numbers are the tiny model's real ones; the scale row is the 8B model.



HOW TO READ IT

Blue is the forward pass: the same six steps every time the model runs. The grey dashed box is the 80 weights it reads. Red is the training loop, which runs only while training: it scores the guess, finds each weight's blame and nudges the weights, 500 times. Green is use: weights frozen, one pass per new word.

THE POINT

There is only one machine. A first run, training and a chat all go through the same blue steps. Training is the only thing that changes the weights, and scale only makes each number bigger. Fine-tuning and RL reuse the red loop; the harness wraps the green part.

APPENDIX

The whole story on one page

Every number from the tiny model, in order.

Stage	Step	Result	Ch
Design	Training text	4 sentences → targets: after "I love", tea 67% / coffee 33%	1
Design	Vocabulary	6 words, ids 0–5	2
Design	Shape	4 slots, 1 layer, 1 head, 3 detectors → 80 weights	3
First run	Random weights → "I love"	tea 18%, coffee 18%, rain 16%: noise	4–9
Train	Loss	1.864 (best possible 0.520)	10
Train	One gradient, two ways	−0.0123 (chain rule) = −0.0123 (nudge)	11
Train	500 steps, learning rate 0.1	loss 0.523; tea–coffee similarity 0.00 → 0.92	12
Use	Trained "I love"	tea 67%, coffee 33%, rain 0.2%	13
Use	"I love tea" → ?	rain 95%: confident, never trained	14
Use	KV cache	Same answer, only 1 new token computed	15
Scale	Same weights at 4-bit / 2-bit	loss 0.529 / 1.201; at 2-bit the answer flips to coffee	19
Beyond	Full fine-tune on "I love coffee"	coffee 97% in 5 steps; "I" → hate falls 23% → 0.2% (forgetting)	20
Beyond	LoRA, 7 trainable numbers	coffee 81%, less forgetting in this run	20
Beyond	RL, reward coffee, β = 0.5	coffee 78.5%, as theory predicts	21

Scaling knobs at a glance

Knob	Compute grows	Memory grows	You feel it as	Ch
Parameters	∝ parameters	∝ parameters × bytes	Which GPU, cost per token	3, 17
Width	≈ width²	≈ width²	Model size class	3
Context length	Attention scores ≈ length²	KV cache ∝ length	Long prompts slow and pricey	6, 15
Concurrent users	∝ users (batched)	KV cache ∝ users	Users per GPU	18
Output length	∝ tokens, one at a time	KV cache ∝ tokens	Wait time, output price	14
Bytes per number	Less to read → faster decode	∝ bytes	Fits smaller hardware	17
Training tokens	≈ 6 × parameters × tokens	Streamed, not on GPU	Training bill	1, 12

Glossary

- **Bandwidth**: bytes per second memory can deliver
- **Batch**: several requests processed together
- **Causal mask**: tokens may only look backwards
- **Context**: all tokens the model can see
- **Cross-entropy**: the loss; surprise at the actual next token
- **Decode**: writing, one token per pass
- **Gradient**: how loss changes as one weight moves
- **GQA**: query heads sharing K/V to shrink the cache
- **Head**: one independent attention calculation
- **Inference**: using fixed weights to answer
- **Learning rate**: size of each nudge

- **Logit**: raw score before softmax
- **MLA**: compressing K/V into a smaller row
- **Parameter**: one learned number; "8B" = 8 billion of them
- **Bit / byte**: one 0/1 switch / 8 bits; bits per weight set the size, not the count
- **Prefill**: reading the prompt in one pass
- **Quantisation**: fewer bits per number
- **Residual shortcut**: adding a change instead of replacing
- **Top-p**: sample only from the top words covering p%
- **Chat template**: format that joins messages into one token stream
- **Fine-tuning**: more training on targeted text
- **Harness**: code around the model (chat, tools, loops)
- **KL penalty**: cost for drifting from the original model
- **LoRA**: train a small add-on, freeze the model
- **Reward hacking**: maximising the score, not the intent

Sources and method

- **Tiny model:** every number computed with the companion Python package **tlm** (NumPy). Run `python -m tlm guide` to reproduce them, and `python -m tlm quant` for Chapter 19. Random start: seed 1, spread 0.5, rounded to 1 decimal. Plain gradient descent, learning rate 0.1, 500 full-batch steps on the 8 predictions. Gradients are exact (analytic backprop), checked against nudging each weight (Ch 11) and covered by the package tests. Displayed values are rounded.
- **Real-model figures:** taken from the publicly released configuration, model card and technical report of the open 8B model used throughout.
- **Hardware figures:** from the published datasheet of the data-centre GPU used in the examples.
- **Rules of thumb** (16 bytes per weight for training, 6 × parameters × tokens, tokens per word) and **post-training methods** (fine-tuning, LoRA, RLHF, DPO, group-relative RL, self-consistency): standard results from published research.
- **Code and guide:** github.com/nikhil-rupanawar/tlm_education